Module 1: Formal Syntax and Semantics

Topics

1. Introduction
2. Formal Syntax
3. Static Semantics
4. Dynamic Semantics
5. Syntax Translation

I. Introduction

Although our primary focus in this module is formal syntax and semantics, we begin with several introductory topics, which include the criteria by which we plan to evaluate the features of programming languages, an introduction to language translators, and a brief history of programming languages.

A. Evaluating Language Design

Throughout these modules, we will compare the ways in which different programming languages implement various language features. As a part of the comparison, we will often make judgments regarding which implementation is more desirable. To be able to make such judgments, we must establish a set of criteria to use.

Most language critics would agree that programming languages should be easy to read, easy to write, and capable of producing error-free programs. There is less agreement, however, on the way in which specific language features promote these three goals. What complicates language evaluation is that when we look at more specific language attributes such as simplicity or expressivity—both desirable attributes—what enhances one may lessen the other. Specifically, a language that is too simple may be less expressive.

As we examine language features, we will pay close attention to relationships between them. One of the most undesirable relationships is feature interaction that creates ambiguity, or worse, the possibility of incorrect programs. The latter occurs when two or more features result in programs that would reasonably be expected to do one thing but instead do something else, possibly something that is not at all meaningful. The following statement in either C or C++ illustrates an example of this kind of feature interaction:

```
if (x = 0)
  ...
```

Three features contribute to the misinterpretation of this statement. The first is the similarity of the assignment operator and the relational operator for equality and the fact that in other languages this operator is a relational operator. The second is the fact that assignments are expressions, not statements, as they are in some languages. The third feature is the fact that C and C++ have no true Boolean data type. In module 2, when we discuss expressions, we will see how all three features must be present to create this misinterpretation.

Another important relationship between language features is whether or not they are *orthogonal*. Features are orthogonal when all possible combinations of the two features are allowed. An example of two orthogonal features in C or C++ is whether integral data types are signed or unsigned. Table 1.1 illustrates the orthogonality of these two features:

Table 1.1
Orthogonality of Data-Type Size and Signedness

	char	short	int	long
unsigned	√	√	√	√
signed	√	√	√	√

If, for some reason, some combinations of these two features were not permitted, we would regard them as not orthogonal.

Another important relationship between features is whether two features are redundant, sometimes referred to as *feature multiplicity*. The C or C++ while and for statements are redundant features. These statements are completely interchangeable, meaning that wherever one could be used the other could be used instead. In other cases of feature multiplicity, one feature may be more general than the other. The latter situation is usually more desirable. We will explore why that is so when we encounter our next instance of redundant features.

As we explore various language features, we may find it necessary to use other criteria for comparison. When making judgments about the quality of language design, there is considerable room for opinion, no matter how specific we try to be with our evaluation criteria. Bear this in mind because it is not the case with many aspects of computer science.

B. Language Translators

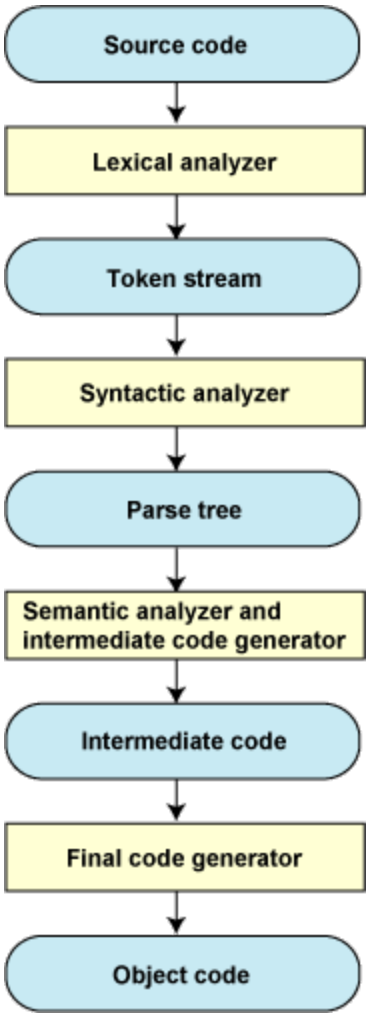
The design of programming languages is intimately connected with the design of the translators that translate programs written in a particular language into a lower-level language. Consequently, understanding the way in which language-design decisions influence the complexity of language translators requires some understanding of how translators are built. The decision of whether a language will support nested subprograms greatly influences the complexity of its compiler, which is one kind of translator. In module 3, we will see how much more complex a compiler must be when the language includes such subprograms.

We will discuss three types of translation mechanics: full compilation, full interpretation, and hybrids of the two.

Compiled Languages

C, C++, Pascal, Ada, and many other languages are fully compiled. Compilers for such languages accept the source code and produce object code. Figure 1.1 illustrates the phases of such a compiler.

Figure 1.1
Phases of a Compiler



Most compilers consist of two passes. The first pass, which transforms the source code to intermediate code, is machine independent. The second pass, which produces the object code, is not. Most compiled languages support separate compilation. For programs that consist of multiple source files, the object files corresponding to those source files must be linked together with a program called a *linker*. Regardless of whether there are multiple source files, the object code must be linked with the object library, which contains common code used by all programs, such as code to perform input and output.

Interpreted Languages

Languages that have a relatively simple syntax and little or no data typing tend to be fully interpreted. LISP, a language we will study in module 5, is one language that is generally interpreted. Early versions of BASIC are another example. The difference between an interpreter and a compiler is that an interpreter translates and executes the program, whereas a compiler only performs the translation.

There are tradeoffs between these two alternatives. The advantage of full compilation is that the programs will run faster because the translation is only done once, not each time the program is run. Debugging programs written in fully interpreted languages was advantageous—especially when compilers were slow. Because most compilers today run so fast, the benefit of full interpretation when debugging has diminished.

Hybrid Languages

Finally, some languages are partly compiled and partly interpreted. This technique was used with early versions of Java. The Java compiler translates the source into intermediate code, which for Java is called *bytecode*. The interpreter, which is referred to as the *Java Virtual Machine* (JVM), translates and executes the bytecode. More recently, just-in-time (JIT) compilers have replaced the interpreter. A JIT compiler, itself, is a kind of hybrid. It translates and executes like an interpreter, but like a compiler, it ensures that the translation is done only once.

C. History of Programming Languages

Although the history of programming languages is relatively short, given the number of programming languages that have been developed during the past half-century, it is a rich history. Rather than attempt to describe all or even a significant number of the languages that have been developed, we instead elect to describe several significant steps in the evolution of languages. This characterization will be somewhat chronological, but not entirely so. Our focus instead will be on the level of abstraction of languages.

Machine-Level Languages

Historically, unstructured imperative languages were the languages used first. *Machine language*, which consists of pure binary numeric code, and *assembly language*, a symbolic equivalent of machine language, are examples. Such languages are the least abstract, which means they are closest to the hardware. They are the languages into which all higher-level languages must eventually be translated. What is important to understand about such languages is that they have no nesting—either at the level of expressions or the level of statements. Each line of code in such languages has at most one operator, and control flow is achieved with unconditional or conditional *goto*-like instructions.

High-Level Imperative Languages

What separates high-level languages from machine language is the presence of nesting, which introduces a greater degree of abstraction to programs written in such languages. Nesting really entered programming languages in two steps, although historically these events happened quite close together in time. Fortran introduced nested expressions, which represent mathematical formulas, but the earliest versions of Fortran permitted almost no nesting of statements.

Structured Imperative Languages

ALGOL, which was developed shortly after Fortran, allowed fully nested statements. This ability makes it possible to program without *goto* statements, although it took almost 30 years before popular languages actually removed such statements.

Declarative Languages

Declarative languages, which include both functional and logic languages, are the most abstract. They are closest to mathematics and furthest from the hardware. Functional languages are based on a branch of mathematical logic called *lambda calculus.* Logic languages are based on a branch called *predicate calculus*.

Although most imperative languages to some degree support recursion and function parameters, declarative languages rely on recursion and higher-order functions in place of iteration, variables, and assignments. Programs written in declarative languages specify definitions of the program more than algorithms for their solution.

Because declarative languages differ fundamentally from imperative ones, we will wait until module 5, when we explore them more fully, to try to characterize all the ways in which they differ from imperative languages.

Object-Oriented Languages

Object-orientation incorporated abstraction into programming languages, but a kind that differs from what we have thus far discussed. The movement toward object-orientation early on was often referred to as *data abstraction*.

Object-orientation is orthogonal to whether a language is imperative or declarative. Table 1.2 illustrates this fact by showing one language in each of the four categories.

Table 1.2
Orthogonality of Object-Orientation and Imperative vs. Declarative

	Non-Object-Oriented	**Object-Oriented**
Imperative	C	C++
Declarative	LISP	CLOS

Object-orientation shifts the focus of design from control structure to data structure, although the data units are objects, which are compound data structures, not primitive variables.

II. Formal Syntax

We now turn to the primary topic for this module—formal syntax and semantics. Like natural languages, programming languages have syntax, which is the grammar, and semantics, which is the meaning. The first two phases of the compiler determine whether the program supplied to the compiler is syntactically correct. Syntax is sometimes divided into two levels, the *lexical level*, which is checked by the lexical analyzer, and the *textual level*, which is checked by the syntactical analyzer.

A. Regular Expressions

Specifying the syntax at the lexical level is synonymous with defining the syntax of the tokens of the language. Regular expressions are generally used to specify syntax at this level. It is easiest to explain how regular expressions are used with an example. Shown below is the regular expression that defines the identifier token in Ada.

```
[a-zA-Z](_?[a-zA-Z0-9])*
```

In Ada, identifiers must begin with a letter followed by zero or more letters or digits. Underscores are permitted, but leading, trailing, and consecutive underscores are prohibited. Clearly, the definition that uses a regular expression is more concise than the equivalent English definition and usually more precise because English can often be unclear or ambiguous. Of course, to understand the regular expression, it is important to understand the following metasymbols

- The ? indicates optional.
- The * means zero or more.
- The - specifies a range of allowable characters.
- The square brackets define a collection of characters from which one can be chosen.
- The parentheses act as a grouping mechanism.

Although the language of regular expressions is not a programming language, it is a declarative language, in that it is used to define language syntax, and it is a metalanguage

Regular expressions are a less powerful language-defining mechanism than the context-free grammars that we will encounter shortly. In particular, regular expressions cannot be used to describe syntax that involves nesting. Consequently, most programming-language token definitions do not permit nesting. In particular, comments cannot be nested in C, as the unsuccessful attempt below illustrates:

```
/* /* An attempt to nest comments */ */
```

In the above example, the comment ends at the first */.

B. Backus-Naur Form

To describe the syntax of a programming language at the textual level, a more powerful metalanguage called *Backus-Naur Form* (BNF) is used. Using BNF, we write a grammar that consists of a collection of productions. The productions consist of terminal symbols, nonterminal symbols, and punctuation metasymbols.

Every production contains the metasymbol → that separates the left-hand side of the production from the right. That symbol is read as "can be replaced by" or "is defined as." The left-hand side of every production consists of a single nonterminal, which represents the nonterminal being defined, and the right-hand side consists of a collection of nonterminals and terminals. As an example of a production, consider the production that defines the syntax of a while statement in C, C++, or Java:

```
<while_stmt> → while (<exp>) <stmt>
```

The reserved word while and the parentheses are terminals, whereas <exp> and <stmt> are nonterminals.

The metasymbol | can be used to condense multiple productions with a common left-hand side into a single production. The metasymbol | separates various choices for the right-hand side. Extended BNF allows additional metasymbols.

Although extended BNF allows the metasymbols {} to be used for zero or more occurrences, in standard BNF, recursive productions are needed to define components that occur multiple times.

Shown below is an example of a recursive production that is used to define statement lists in Pascal. Pascal is unusual among languages in that the semicolon is a statement separator rather than a statement terminal, which the following production illustrates:

```
<stmt_list> → <stmt_list> ; <stmt> | <stmt>
```

The above production is left recursive or head recursive. In this case, a right-recursive or tail-recursive production could also have been used. Notice that, like recursive methods, recursive productions must have a base case—a nonrecursive choice.

To determine whether a program is syntactically correct, the parser phase of a compiler attempts to construct a parse tree for the program. The ability to construct a parse tree verifies syntactic correctness. Because the parser phase of

most compilers is not a separate pass, only a small portion of the parse tree is actually maintained at any point in time. Parse trees serve a second purpose, which is that they are used to generate intermediate code.

It is possible to write more than one grammar to define any language. As an example, we could define the Pascal statement lists that we discussed earlier with the following right-recursive production:

```
<stmt_list> → <stmt> ; <stmt_list> | <stmt>
```

Although these two productions define the same syntax for statement lists, the parse trees that would be constructed in the two cases would be different. If all we cared about was using parse trees to verify syntactic correctness, either grammar could be used, but recall we stated earlier that parse trees are also used to generate intermediate code. To generate proper intermediate code, it is important to define a grammar that produces the correct parse tree.

Expression Grammars

Ensuring that our grammar produces the correct parse tree is especially important for expressions. Therefore, next we will consider a variety of different grammars that we might use to define expressions and consider how that choice affects the parse trees that are produced. Let's begin with a relatively simple grammar for expressions in a typical programming language—one that includes the exponential operator.

```
<exp> → <exp> <op> <fact> | <fact>
<fact> → <id> | <cons>
<id> → x | y
<cons> → 1 | 2
<op> → + | - | * | / | **
```

The last three productions of our grammar define the syntax of the identifier, integer literal constant, and operator tokens. Earlier, we mentioned that it is common to use regular expressions for this purpose. It is worth noting, however, that BNF can define any language defined by regular expressions, although the reverse is not true. Notice, too, that our definition for the identifier token is much

more limited than our previous definition. We have elected to use a grammar for both the lexical and textual levels of syntax in this case so that the leaves of our parse tree will be lexemes rather than just tokens.
Figure 1.2 illustrates the construction of the parse tree for one expression. The parse tree is constructed from the top down using a left-most derivation.

Figure 1.2
Construction of a Left-to-Right Parse Tree

0:00 / 2:41

Viewing the completed tree from the bottom up, it grows from left to right. It must grow in this fashion because the only recursive production in this grammar is left-recursive. We could change that production to be right-recursive without changing the language, but if we made that change, the parse tree for this expression would grow from right to left.

As we have been emphasizing, it is necessary to have a grammar that will produce parse trees that generate the proper code. If we use the above parse tree to generate intermediate code, we would emit instructions in postorder, giving the following intermediate code:

```
t1 = x + 1
t2 = t1 * y
t3 = t2 ** 2
```

The variables t1, t2,and t3 are temporary variables. We will see more examples using temporary variables later in this module when we discuss operational semantics.

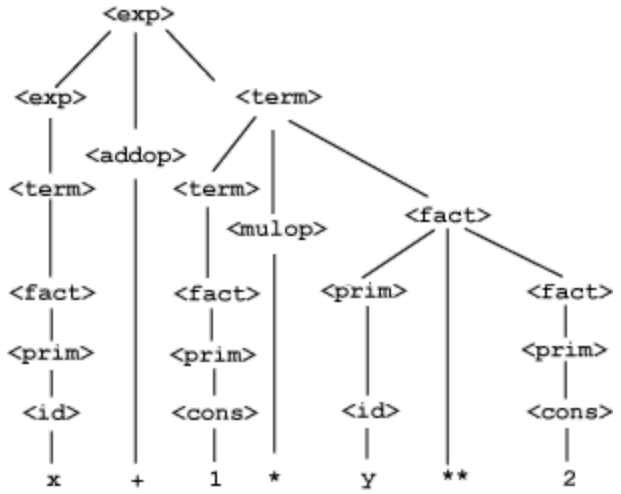
In most programming languages, not all operators have the same precedence—a characteristic that expressions borrow from mathematical formulas. Clearly, the code generated from that parse tree does not follow the usual precedence rules.

By modifying the grammar, we can ensure that a parse tree will be produced that will generate code consistent with the precedence rules. Consider the following expression grammar:

```
<exp> → <exp> <addop> <term> | <term>
<term> → <term> <mulop> <fact> | <fact>
<fact> → <prim> ** <fact> | <prim>
<prim> → <id> | <cons>
<id> → x | y
<cons> → 1 | 2
<addop> → + |-
<mulop> → * | /
```

Figure 1.3 illustrates the parse tree for the same expression used previously. Notice, however, that the parse tree that is constructed will now facilitate generating the proper code.

Figure 1.3
Construction of a Parse Tree According to Operator Precedence



One other thing to notice is that the parse trees generated by this grammar will also properly reflect the associativity rules—the adding and multiplying operators being left associative and exponentiation being right associative—because the recursive rules for the expression and term are left recursive and the recursive rule for a factor is right recursive.

One aspect of typical expression syntax we have thus far omitted in our grammars is the possibility of parentheses. Most languages allow, but do not require, parenthesizing subexpressions, in which case the parentheses override the rules of precedence and associativity.

Adding the following production to our previous grammar will accomplish this goal:

```
<prim> → (<exp>)
```

With the addition of this production, our grammar now contains indirectly recursive productions: an expression can be a term, a term can be a factor, a factor can be a primary, and, finally, a primary can contain an expression.

Ambiguous Grammars

There is one other characteristic of certain grammars that you should understand, which is whether a grammar is ambiguous. None of the grammars we have examined thus far have been ambiguous. Although it is easy to modify them to make them ambiguous, we should caution you that we are doing so only to illustrate ambiguous grammars, not because such grammars are desirable. In fact, we prefer to avoid them.

Consider the following modified expression production of our first expression grammar:

```
<exp> → <exp> <op> <exp> | <fact>
```

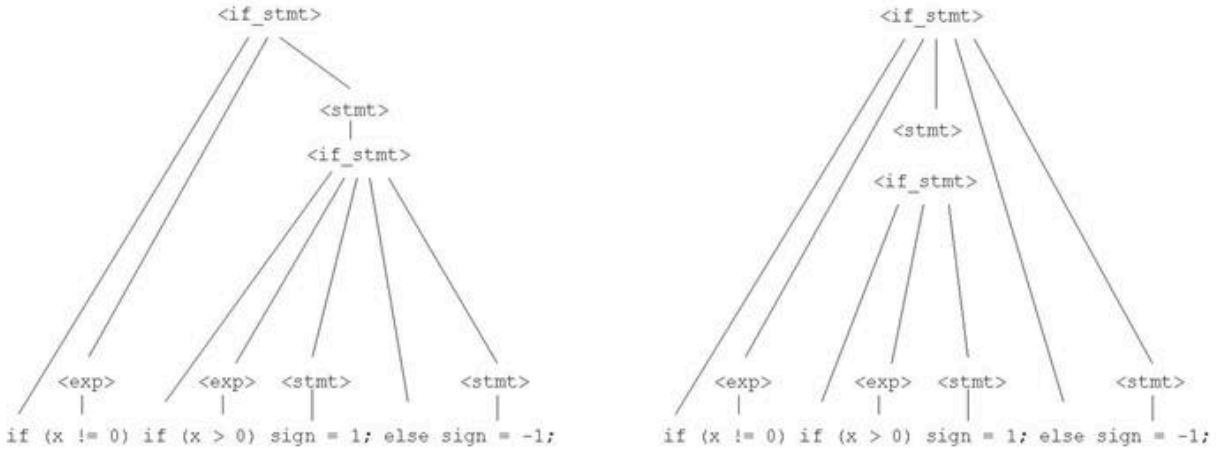
Notice that this production is both left- and right-recursive, which is the cause of the ambiguity. Consequently, with this grammar, parse trees that grow from left to right or right to left, as well as any combination of the two, can be constructed. There are other ambiguous grammars, in which the cause of the ambiguity is more subtle. One of the most important such productions that occurs in grammars of many programming languages is the one that defines the syntax of the if statement, as shown below.

```
<if_stmt> → if (<exp>) <stmt> [else <stmt>]
```

Note also that the above production uses the square bracket metasymbols of extended BNF that denote *optional*. They surround the else clause in the above production.

Figure 1.4 illustrates the ambiguity caused by this production, showing two parse trees for the same statement. Note that for simplicity, the parse trees in this diagram are partial. We have omitted the intermediate details for assignments and expressions to avoid overcomplicating what we are trying to illustrate.

Figure 1.4
Ambiguity of if Statement Grammar



Although it is possible to write a grammar that is not ambiguous, altering the language syntax is the better solution as we will see in module 2 when we discuss the dangling else problem.

III. Static Semantics

For a program to compile, it must satisfy both the syntax rules of the language and the static semantic rules. Whereas the syntax rules verify that a program has correct grammar, the static semantic rules ensure that the program is meaningful.

A. Static Semantic Rules in English

To help you understand this distinction, let's consider the analogous requirements in English. Although you may never have thought about it, English sentences must satisfy both syntax and static semantic rules to be correct sentences. To illustrate this point, consider the following sentence:

The idea ate the house.

Syntactically, this sentence is fine. It is a declarative sentence containing a transitive verb and a direct object, but the sentence is not meaningful. The reason is that although *idea* is a noun, it is not a physical object and, moreover, not an animate one. Furthermore, *house* is also a noun, but not an edible object.

These same two requirements apply at the lexical level—both in English and in programming languages. Let's consider how they apply in English. At the lexical level, there are syntax rules governing how letters form words that are very similar to the regular expression rule that we encountered earlier, which defines the rules for defining identifiers. In English, the rules involve the alternation of consonants and vowels, although certain consonant strings such as *str* are permitted, whereas many others such as *tzl* are not. Other more arcane rules also exist, such as the requirement that the consonant *q* be followed exclusively by the vowel *u*.

Just because a string of English characters follows the syntax rules for English words does not make the string a word. The string *fap* fulfills the syntax rules but not the static semantic ones because that character string has no meaning. To have

meaning, it must be in the dictionary, which is the primary static semantic requirement at the lexical level in English.

B. Static Semantic Rules in Programming Languages

Having considered the significance of static semantics in English, let's now consider how these rules apply to programming languages beginning at the lexical level. In programming languages, at the lexical level, the only static semantic rule applies to identifiers. For an identifier to be meaningful, it must be in the compiler's symbol table, which means that in most languages it either must be predefined or have been defined within the program being compiled.

Next, we consider such rules at the textual level. Let's return to an example we presented earlier when we were discussing feature interaction. The example we used was:

```
if (x = 0)
...
```

We noted that in C and C++, such a statement will compile but will be misinterpreted. The same statement in Java will produce a compilation error, not because it violates any syntax rule, but because it violates a rule that pertains to static semantics. The rule in Java is that the expression in an if condition must have type boolean. Consequently, there are statements with the same syntax that will compile, for example:

```
boolean b;
if (b = true)
...
```

Many rules involving static semantics are related to type checking. Others pertain not to the type of identifiers but to their categories. As an example of the latter, consider the following assignment in C, C++, or Java:

```
PI = 3.14159;
```

Based on the commonly used lexical convention, we infer that PI is a declared constant, so this assignment will not compile. It is because of the rule that the identifier on the left-hand side of the assignment must be the name of a variable and cannot be the name of a constant.

It is the semantic-analyzer phase of the compiler that checks these rules and generates error messages when they are violated. The semantic analyzer also frequently checks rules, sometimes referred to as *context rules*, that are really rules of syntax but awkward to define without excessively complicating the BNF. An example of such a rule is that an EXIT statement in Ada can only appear inside a loop and in no other context.

C. Expressing Static Semantic Rules with Attribute Grammars

BNF grammars can be extended by adding additional rules that involve annotations with attribute information that incorporate static semantics. These extended grammars are known as *attribute grammars*. They contain two additional kinds of rules—one kind to define how attribute information is carried up the parse tree and another kind that specifies under what conditions compilation errors are to be generated.

Because the former kind of rule is mainly of interest to those building compilers, we address only the latter kind of rule, called *predicates*, which are of interest to both compiler writers and compiler users. As our example of a predicate, we provide the one that would be needed to express the rule associated with the required type of the expression inside a Java if statement:

```
expression.type == boolean
```

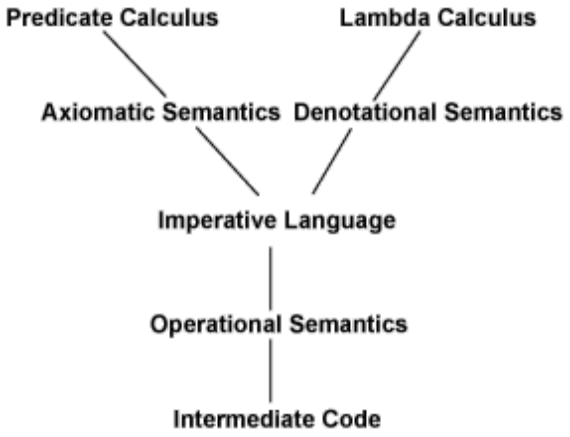
Clearly, the attribute .type refers to the type of the expression. Unfortunately, not all error conditions that are typically generated by the semantic analyzer can be easily expressed with attribute grammars.

IV. Dynamic Semantics

Now that we have discussed the rules that define whether or not a program is meaningful, we consider meaning from another perspective—the way in which the meaning of a program can be described, which is referred to as *dynamic semantics*.

The only way that the meaning of a language can be described is with another language. Each of the various models for dynamic semantics essentially uses some other language. Figure 1.5 illustrates the languages used in each of the three models.

Figure 1.5
The Three Dynamic Semantic Models



In our diagram, we placed axiomatic and denotational semantics above the imperative language whose meaning they describe, using predicate and lambda calculus respectively. Because both predicate and lambda calculi are branches of mathematical logic, they are more abstract than an imperative language. We placed operational semantics below because it defines the meaning of an imperative language by using the lower-level intermediate code into which a compiler would translate it.

A. Operational Semantics

Operational semantics describes the meaning of the statements in a programming language by describing what the statements do. Providing a flowchart for an if statement is really an operational semantic description of its meaning. A less

graphical way to convey the same idea is to define a small set of intermediate code instructions. We need only a small number of instructions. The first set consists of four instructions needed for computation.

```
a ← b                      assignment with no operator
a ← <un_arith_op> b        assignment with a unary arithmetic operator
a ← b <bin_arith_op> c     assignment with a binary arithmetic operator
a ← b(c)                   function call
```

What is most important is that these instructions do not allow nested subexpressions. Each instruction contains no more than a single operator. Assignments containing nested expressions in high-level languages must be translated into these single-instruction assignments.

Figure 1.6 illustrates the generation of intermediate code for an assignment in C or C++ to a nested expression that computes one root of a quadratic equation from its three coefficients.

Figure 1.6
Translation of Expression That Calculates One Root
of a Quadratic Equation into Intermediate Code

0:00 / 1:39

The second set consists of two instructions that perform transfer of control:

```
if a <rel_op> b goto L
go to L
```

These two instructions are usually referred to as a conditional and an unconditional *goto*, respectively. They allow us to define the behavior of high-level control statements that can be nested. Table 1.3 shows how an if and a while statement can be translated into intermediate code.

Note: The symbol ¬ represents negation.

Table 1.3
Translation of Structured Statements into Intermediate Code

High-Level Statement	Intermediate Code

if (<exp>)
  <stmt$_1$>
else
  <stmt$_2$>
	if ¬<exp> goto L_1
	  <stmt$_1$>
	  goto L_2
	L_1 <stmt$_2$>
	L_2 ...

while (<exp>) <stmt>	L_1 if ¬<exp> goto L_2 <stmt> goto L_1 L_2 ...

To illustrate how these translations would be applied, consider the following code segment written in C or C++ that determines whether the value in n is a prime number:

```
i = 2;
while (i <= srqt(n))
{
  if (n % i == 0)
    isPrime = false;
  i++;
}
```

Shown below is the intermediate code that corresponds to the above code segment:

```
      i = 2;
      isPrime = true
      t1 = sqrt(n)
L1    if (i > t1) goto L3
      t2 = n % i
      if (t2 != 0) goto L2
      isPrime = false
L2    i = i + 1
      goto L1
L3    ...
```

The actual target of the *goto* statements must be filled in once the destination statements have been generated. This process is known as *backpatching*.

B. Axiomatic Semantics

Axiomatic semantics defines the meaning of programs using assertions written in predicate calculus called preconditions and postconditions. As such, it defines what must be true before and after a program executes but does not specify how it is to be accomplished. Axiomatic semantics was developed as a part of an effort to prove programs correct. The kind of statement that we prove using axiomatic semantics is one like the following:

{P} S {Q}

This statement claims that if the precondition P is true and program S is executed, postcondition Q will be true.

Now let's consider how axiomatic semantics defines the meaning of statements in an imperative language. We express the meaning of an assignment statement, using the following substitution rule:

$\{Q_{v \rightarrow e}\}\ v = e\ \{Q\}$

In the above rule, $Q_{v \rightarrow e}$ represents the assertion Q having substituted every occurrence of *v* with *e*. Using the substitution rule, we can compute a precondition given a postcondition. In figure 1.7, we demonstrate how a precondition can be computed for the standard three-line code sequence that swaps two variables.

Figure 1.7
Computing the Precondition of a Swap-Code Segment
Using the Substitution Rule

0:00 / 0:58

Compound statements are defined using inference rules. The inference rule for an if statement requires that the expected two cases be proven to draw an inference about the entire statement.

$$\frac{\{P \wedge B\}\ S_1\ \{Q\},\ \{P \wedge \neg B\}\ S_2\ \{Q\}}{\therefore\ \{P\}\ \texttt{if}\ (B)\ S_1\ \texttt{else}\ S_2\ \{Q\}}$$

To illustrate the use of the inference rule for an if statement, let's embed the swap-code segment that we discussed earlier inside an if statement to construct a C++ function that sorts two values into ascending order. That function is shown below with the precondition, postcondition, and all the intervening assertions, numbered and highlighted.

```
// 1 {true}
void sortTwo(int& x, int& y)
{
  if (x > y)
  {
    // 2 {x > y}
    t = x;
    // 3 {t > y}
    x = y;
    // 4 {t > x}
    y = t;
    // 5 {y > x}
  }
  // 6 {x <= y}
  else
    // 7 {x <= y}
    ;
  // 8 {x <= y}
}
// 9 {x <= y}
```

Notice that the precondition is {true}, which is the weakest possible precondition. It is always desirable to state the weakest precondition for a given program, although that weakest precondition is sometimes an assertion stronger than simply {true}.

When we wish to prove the correctness of a function such as sortTwo, we work from the inside out, first establishing that if assertion 2, {x > y}, is true before the three-line swap, assertion 5, {y > x} will be true afterward. We establish that fact using the same technique of substitution that we used earlier. Unfortunately, assertion 5 is not exactly what we need. What we need is assertion 6, {x <= y}. That requires an additional inference rule, called the *rule of consequence,* shown below:

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q' \Rightarrow Q}{\therefore\ \{P'\}S\{Q'\}}$$

Here is how we apply the above rule to establish the assertion, which is numbered 6.

$$\frac{\{x > y\}\ \texttt{swap}\ \{y > x\},\ y > x \Rightarrow x <= y}{\therefore\ \{x > y\}\ \texttt{swap}\ \{x <= y\}}$$

For the clause Q' ⇒ Q in the rule of consequence, we used an implication we know to be true from arithmetic:

y > x ⇒ x <= y

We did not need the clause P' ⇒ P.

To complete the proof, we must apply the inference rule for an if statement, which we introduced earlier. The application of that rule is shown below:

$$\frac{\{\text{true} \wedge x > y\}\ \texttt{swap}\ \{x <= y\},\ \{\text{true} \wedge x <= y\}\ ;\ \{x <= y\}}{\therefore\ \{\text{true}\}\ \texttt{if}\ (x > y)\ \texttt{swap else}\ ;\ \{x <= y\}}$$

We have completed the proof and can now state {true} sortTwo {x <= y}.

The inference rule for a while statement requires that we find a loop invariant, which in the following inference rule is the assertion P.

$$\frac{\{P \wedge B\}\ S\ \{P\}}{\therefore\ \{P\}\ \texttt{while}\ (B)\ S\ \{P \wedge \neg B\}}$$

Finding loop invariants is often the most challenging aspect of correctness proofs. Because proofs involving loops can become quite complicated, we omit them.

Perhaps the most important idea to understand about axiomatic semantics is that it defines the meaning of a program in a language that is more abstract than the way in which it is expressed in an imperative language. An axiomatic statement of meaning suggests no algorithm. As an illustration of that point, consider the fact that a bubble sort, quick sort, heaps sort, and so on would all have identical pre- and postconditions.

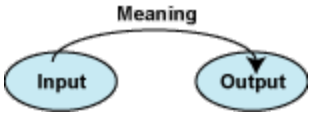
The fact that both axiomatic semantics and logic programming languages are based on predicate calculus suggests a strong connection between them. We will get a better sense of that connection in module 5.

C. Denotational Semantics

As we mentioned in our introduction to dynamic semantics, denotational semantics, like axiomatic semantics, is based on a branch of mathematical logic—in this case, lambda calculus. Lambda calculus is also referred to as *recursive function theory*.

The meaning of a program from the perspective of denotational semantics is a mathematical function that maps the input of the program as the domain to the output as the codomain. Figure 1.8 illustrates this idea.

Figure 1.8
The Meaning of a Program as a Mathematical Function



The meaning of individual statements of an imperative language can be viewed as functions also, but the domain and codomain of those functions are the state of the machine. Understanding this view of statement meaning will help you later to understand the distinction between imperative and declarative languages. When we encounter functional languages in module 5, we will see that programs in functional languages, unlike programs in imperative languages, are stateless.

To help explain this view of program meaning, we will begin with a computational mechanism involving only assignment statements, similar to what we did with axiomatic semantics. When no input or output is involved, we can capture the denotational meaning of a block of code using a concurrent assignment.

Figure 1.9 illustrates this method of computing a concurrent assignment, again using the three-line swap-code segment as our example.

Figure 1.9
Computing the Concurrent Assignment of a Swap-Code Segment

0:00 / 0:58

If we are dealing with three integer variables, the state space at that point in the program is the set:

$$\mathbf{Z} \times \mathbf{Z} \times \mathbf{Z}$$

The meaning of each assignment then is a function a:

$$\mathbf{Z} \times \mathbf{Z} \times \mathbf{Z} \rightarrow \mathbf{Z} \times \mathbf{Z} \times \mathbf{Z}$$

which is exactly what is represented by a concurrent assignment of three variables. Suppose that a_1, a_2, and a_3 represent the functional meanings of three successive assignments, such as the ones in our three-line swap. The meaning of the whole block is the composition of the three functions:

$$a_1 \circ a_2 \circ a_3$$

This function composition is exactly what we computed using the technique of symbolic execution illustrated in figure 1.10.

Function composition is exactly how the meaning of a statement list is defined. Assume that *M* is the function that converts a statement to its functional meaning, and that S_1 S_2 ... S_n are the component statements of a statement list, and that *s* is the state. Then, we can define the meaning of a statement list as follows:

$$M(S_1\ S_2 \ldots S_n)\ (s) = M(S_n) \circ \ldots \circ M(S_2) \circ M(S_1)\ (s)$$

Next, let's consider how we can define the function meaning of an if statement.

$$M(\text{if (B) } S_1 \text{ else } S_2)\ (s) = \begin{cases} M(S_1) \text{ if } B(s) \\ M(S_2) \text{ otherwise} \end{cases}$$

This definition of the meaning of an if statement should be easier to grasp than the meaning of the statement list. It says that the meaning of an if statement is the meaning of the first statement S_1 if B is true in state *s*, and it is the meaning of the second statement S_2 otherwise.

If we combine our technique of computing a concurrent assignment for a sequence of assignments with the definition of the meaning of an if statement, we can state the functional meaning of the C++ function sortTwo, which we used as an example in our discussion of axiomatic semantics, as follows:

$$M(\text{sortTwo}) = \begin{cases} x, y = y, x, \text{ if } x > y \\ x, y = x, y, \text{ otherwise} \end{cases}$$

There are two aspects of this definition that warrant comment. First, we have dropped the local temporary variable t from our concurrent assignment. From the perspective of a whole C++ function, the mathematical function that defines its meaning needs only to involve the parameters, never the local variables. Although the local variables are a part of intermediate states internal to the function, they are not a part of the initial or final states of the function as a whole.

The second aspect is the concurrent assignment in the "otherwise case." The code that was executed in that case was the null statement. Its meaning is the identity function, which is a concurrent assignment in which each *n*-tuple is mapped to

itself.

We conclude by defining the functional meaning of a while statement, which is shown below:

$$M(\text{while (B) S})(s) = \begin{cases} M(\text{while (B) S}) \circ M(\text{S})(s) & \text{if B}(s) \\ s, & \text{otherwise} \end{cases}$$

As was true in the case of axiomatic semantics, understanding the functional definition of the meaning of the while statement is a bit more difficult, so we will forgo any examples, and just make a few important observations about this definition. It is a recursive definition with two cases—the first being the recursive case and the second being the base case. In the recursive case, the meaning involves composition of the function defining the whole loop with the meaning of the statement that constitutes the body of the loop. Finally, observe that the function being defined in the base case is the identity function.

We recognize that dynamic semantics is a rather abstract topic. The language of mathematical logic used in both cases is a declarative language. As with declarative programming languages, recursion must be used to express what would be expressed iteratively in an imperative language.

Finally, note that both denotational semantics and functional programming languages are based on lambda calculus. In fact, denotational semantics is sometimes called *functional semantics*. In the case study for module 5, we will present an interpreter for an imperative language that is written in a functional language. For each statement of that imperative language, the interpreter will contain a function that essentially defines the functional meaning of that statement. At that time, we will encourage you to reread this section to see how closely related the two really are.

V. Syntax Translation

As we discussed earlier in this module, the first two phases of the compiler deal with verifying that the program submitted has correct syntax. The scanner or lexical analyzer handles the lexical level that breaks the character stream into lexemes and

categorizes those lexemes into tokens. The parser or syntactic analyzer handles the textual level, which includes each of the remaining levels from expressions to statements to subprograms, and in object-oriented languages, classes or some similar construct.

For a variety of reasons, it is customary to divide syntax into these two levels. We mentioned one of those earlier, which is that because it is easy to restrict token syntax to disallow nesting, token syntax can be defined with regular expressions—a less powerful defining mechanism than BNF. Another important difference between these two levels in most high-level languages is the role of white space, which includes spaces, tabs, and new-line characters. Whereas any amount of white space is permitted between lexemes without affecting the program meaning, white space cannot be arbitrarily inserted between characters within tokens without changing the program.

A. Scanners

In most modern compilers, scanners are not hand-coded. Because the theory that surrounds regular expressions and finite state automata is so well understood, there are programs, called *lexical analyzer generators,* that accept the regular expression definition of the tokens of a programming language and produce the transition table for the finite state automata that accepts those tokens. One of the most popular lexical analyzer generators is one called lex that is a part of the UNIX operating system.

It is, of course, possible to hand-code a lexical analyzer in a high-level language such as Java. In the case study for this module, we will examine an example of one such scanner.

B. Parsers

Parsers are, of necessity, more complicated programs than scanners because they must be able to recognize languages whose syntax involves nesting, as is universally true among modern high-level languages. Nonetheless, like recognizing languages defined with regular expressions, the problem of recognizing context-free languages is so well understood that programs called *syntactic analyzer*

generators exist that produce parse tables that are used by a very simple algorithm. These algorithms implement a machine called a *pushdown automaton*. One such generator, named yacc, is provided with the UNIX operating system.

Types of Parsers

There are fundamentally two kinds of parsers. The first kind is the *top-down parser,* which constructs the parse tree from the root start symbol to the terminal symbols that are the leaves. Figure 1.3, which we encountered earlier, constructed the parse tree in this fashion. The second kind is the *bottom-up parser,* which begins with the leaves and works toward the root. Bottom-up parsers are more powerful than top-down ones, which means they are capable of recognizing a larger number of grammars.

We intend to focus exclusively on top-down parsers, which although less powerful, are somewhat simpler. We leave bottom-up parsers, of which yacc is an example, to a more advanced course on language translation.

The problem of efficiently parsing languages that are defined by arbitrary grammars is difficult because for some grammars, it may be impossible at each step to know the correct production to choose, making it necessary to backtrack. Because of the inefficiency incurred by backtracking, the efficiency of parsing algorithms that can work with any grammar is $O(n^3)$, where n is the length of the program to be parsed. By restricting allowable grammars, algorithms can be developed that are $O(n)$, which is clearly far more desirable.

The family of grammars that can be efficiently parsed with top-down algorithms are called *LL(1) grammars*. Two restrictions must be imposed for a grammar to be able to be parsed efficiently top-down:

1. No left-recursive productions are permitted.
2. In a given state, we can always know what production to choose by looking at the next token.

There are two kinds of top-down parsers—those that use parse tables generated by a syntactic analyzer generator and hand-coded parsers called *recursive descent parsers*. Our focus will be exclusively on the latter.

Recursive Descent Parser

The rules for constructing a recursive descent parser are quite simple. There must be a function for each production, which will be recursive if the production is recursive. When writing the function to parse a particular production, the symbols on the right-hand side are processed one-by-one. If the symbol is a nonterminal, a call is made to the function that parses the production with that nonterminal on the left-hand side. If it is a terminal symbol, the function verifies that the next token in the input stream is that token. If so, it consumes it; if not, a syntax error is generated.

Clearly, an example best illustrates how a recursive descent parser is constructed. Let's consider a few productions from the grammar of Pascal.

```
<compound_stmt> → begin <stmt_list> end
<stmt_list> → <stmt> ; <stmt_list> | <stmt>
<stmt> → <compound_stmt> | <if_stmt>
<if_stmt> → if <exp> then <stmt> [else <stmt>]
```

Let's begin with a few observations about the above grammar. It contains a pair of indirectly recursive productions. An if statement contains statements, and a statement can be another if statement. Note, also, that the first choice on the right-hand side of the statement-list production is right-recursive.

Now let's examine the Java methods that would be needed to parse the first three productions, beginning with the compound statement.

Move your mouse over the highlighted parts of the method to see the explanations of those parts.

```
boolean compoundStmt()
{
    if (token == BEGIN)
    {
        token = getNextToken();
        if (stmtList())
            if (token == END)
            {
                token = getNextToken();
                return true;
            }
    }
    return false;
}
```

Next, let's consider the method that parses statements.

Move your mouse over the highlighted part of the method to see the explanations of that part.

```
boolean stmt()
{
    if (token == IF)
        return ifStmt();
    if (token == BEGIN)
        return compoundStmt();
    return false;
}
```

Finally, let's consider the method that parses a statement list.

Move your mouse over the highlighted parts of the method to see the explanations of those parts.

```
boolean stmtList()
{
    if (stmt())
        if (token == ';')
        {
            token = getNextToken();
            return stmtList();
        }
        else
            return true;
    return false;
}
```

In the way we have written the parser, the method for parsing the complete program will ultimately return true or false—either the program parsed, or it contained a syntax error. Parsers in production compilers must do more than just report success or failure. They must locate and identify the type of error, then recover from the errors to attempt to find other errors. Error recovery is one of the most challenging aspects of writing a parser. In module 3, we will discuss how the design of the grammar of a language can make the recovery process easier.

In the case study for this module, we present a complete program that contains a recursive descent parser.

Return to top of page

© 2026 University of Maryland Global Campus

All links to external sites were verified at the time of publication. UMGC is not responsible for the validity or integrity of information located at external sites.

Module 1: Formal Syntax and Semantics

C Program Formatter Written in Java

As our case study for this module, we have chosen a program written in Java that contains a hand-coded scanner and a recursive descent parser. The purpose of this program is to format a C program. The formatting that is to be performed involves properly indenting statements according to their depth of nesting, providing uniform and consistent spacing between tokens, and writing the formatted program to an output file suitable for printing on a fixed-size page. To achieve that requirement, the program lines must not exceed a specified number of characters, and there must be a maximum number of lines per page.

A feature that indents programs automatically is incorporated in many current integrated development environments. In such new environments, the indentation is often done as the program is input. Some environments allow the user to specify how the braces will be placed, where blank lines will be inserted, and so on.

To illustrate how this program will behave, we provide a sample C program that determines the first four perfect numbers, which will be input to the formatter. That program is shown below:

```
#include <stdio.h>
#define NUMBER_TO_FIND 4
void main()
{
int candidate=2, numberFound = 0, sumOfFactors, divisor;
while (numberFound < NUMBER_TO_FIND)
{
sumOfFactors = 0;
for (divisor = 1; divisor <= candidate / 2; divisor++)
if (candidate % divisor == 0)
sumOfFactors += divisor;
if (candidate == sumOfFactors)
```

```
{
printf("%d\n", candidate);
numberFound++;
}
candidate++;
}
}
```

We have intentionally used very poor formatting so that the effect of the formatter will be clearer. When the complete formatter program is run with the above file as input, the output looks as follows:

```
primes
PAGE 1

#include <stdio.h>
#define NUMBER_TO_FIND 4

primes
PAGE 2
void main()

{
int candidate = 2, numberFound = 0, sumOfFactors, divisor;

while (numberFound < NUMBER_TO_FIND)
{
sumOfFactors = 0;
for (divisor = 1; divisor <= candidate / 2; divisor++)
if (candidate % divisor == 0)
sumOfFactors += divisor;
if (candidate == sumOfFactors)
{
printf("%d\n", candidate);
numberFound++;
```

```
}
candidate++;
}
}
```

Next, let's examine the code for this program, beginning with the class that contains the main method. As with any well-designed object-oriented program, the main method is quite short. It creates three *singleton objects*—the scanner, the parser, and the output object—that constitute the program. The code for the main class is shown below.

```
import java.io.*;

class Main
{

// The main function for the C formatter program. It creates
// the three primary objects, an output object, a lexer object,
// and a formatter object. It then calls the file method of the
// formatter object to perform the formatting.

private static final BufferedReader stdin =
new BufferedReader(new InputStreamReader(System.in));

public static void main(String[] args) throws IOException
{
String fileName;
System.out.print("Enter file name without .c: ");
fileName = stdin.readLine();
Output output = new Output(fileName);
Lexer lexer = new Lexer(fileName, output);
Format format = new Format(lexer, output);

format.file();
lexer.close();
```

```
output.close();
}
}
```

As we discussed in the module commentary, the tokens are best defined using an enumerated type, which is what we have elected to do. That enumerated type is shown below:

```
// Token Definitions

enum Token {NOT_FOUND, UPPER_CASE_IDENTIFIER, CONSTANT, COMMENT,
COMPILER_DIRECTIVE, ASSIGNMENT_OPERATOR,
PRE_OR_POST_UNARY_OPERATOR, STRUCTURE_OPERATOR,
UNARY_OR_BINARY_OPERATOR,UNARY_OPERATOR, BINARY_OPERATOR,
TERNARY_OPERATOR, COLON, LEFT_PARENTHESIS, RIGHT_PARENTHESIS,
LEFT_BRACKET, RIGHT_BRACKET, LEFT_BRACE, RIGHT_BRACE, SEMICOLON,
COMMA, STRING, SC_SPECIFIER, BREAK, CASE, TYPE_SPECIFIER,
CONTINUE, DEFAULT, DO, ELSE, FOR, GOTO, IF, RETURN, SIZEOF,
STATUS, STRUCT, SWITCH, UNION, WHILE, IDENTIFIER, FIRST, NONE,
END_OF_FILE}
```

Next, let's examine the class that defines the scanner object. This scanner is hand-coded, rather than being generated by a lexical analyzer that has been provided the regular expression definitions of the tokens. As you will see, most of the methods are private. The primary public method is getNextToken, which is called repeatedly by the parser.

```
// The Lexer class contains a lexical analyzer that returns Tokens
// from the input file on each call to getNextTokens. The class
// also contains functions that adjust the spacing based on the
// type and context of the Tokens.

import java.io.*;

class Lexer
```

```
{
public static final int SUPPRESS_NEITHER_SPACE = 0,
SUPPRESS_LEADING_SPACE = 1, SUPPRESS_TRAILING_SPACE = 2;
private int i = 0, spacing;
private char character;
private String line = "";
private BufferedReader file;
private Output output;
private Token currentToken, lastToken;
private String currentLexeme, lastLexeme;

// Constructor initializes private data members and opens the input
// file.

public Lexer(String fileName, Output output)
throws FileNotFoundException
{
file = new BufferedReader(new FileReader(fileName + ".c"));
character = nextChar();
currentLexeme = "";
lastToken = Token.NONE;
this.output = output;
}

// Closes input file

public void close() throws IOException
{
file.close();
}

// adjustSpacing will set bits in the spacing word to indicate the
// type of spacing adjustment to be done, LEADING OR TRAILING.

public void adjustSpacing(int spacingValue)
```

```
{
spacing |= spacingValue;
}

// checkDeclarationSpacing sets leading or trailing space bits in
// the variable "spacing" according to the type of current token.

public void checkDeclarationSpacing(Token current)
{
if (currentToken == Token.UNARY_OR_BINARY_OPERATOR)
spacing |= SUPPRESS_TRAILING_SPACE;
else if (currentToken == Token.LEFT_PARENTHESIS)
spacing |= SUPPRESS_LEADING_SPACE;
}

// checkExpressionSpacing sets leading or trailing space bits in
// the variable "spacing" according to the types of current and
// last tokens.

public void checkExpressionSpacing(Token current, Token previous)
{
if (current == Token.PRE_OR_POST_UNARY_OPERATOR)
if (previous == Token.IDENTIFIER ||
previous == Token.RIGHT_BRACKET ||
previous == Token.RIGHT_PARENTHESIS)
spacing |= SUPPRESS_LEADING_SPACE;
else
spacing |= SUPPRESS_TRAILING_SPACE;
else if (current == Token.UNARY_OR_BINARY_OPERATOR)
if (previous != Token.IDENTIFIER &&
previous != Token.RIGHT_BRACKET
&& previous != Token.RIGHT_PARENTHESIS && previous !=
Token.PRE_OR_POST_UNARY_OPERATOR)
spacing |= SUPPRESS_TRAILING_SPACE;
}
```

```
// getNextToken returns the next token in the input file and
// displays the previous token. Comment and preprocessor tokens
// are skipped.

public Token getNextToken()
{
if (lastToken != Token.NONE)
{
currentToken = lastToken;
lastToken = Token.NONE;
return currentToken;
}
output.outputToken(currentLexeme, spacing);
spacing = SUPPRESS_NEITHER_SPACE;
lastLexeme = currentLexeme;
do
{
currentLexeme = "";
while (character != 0 && Character.isWhitespace(character))
character = nextChar();
if (character == 0)
{
output.endLine(false);
return Token.END_OF_FILE;
}
if (Character.isUpperCase(character))
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '_')
{
currentLexeme += character;
character = nextChar();
}
currentToken = Token.UPPER_CASE_IDENTIFIER;
```

```
}
else if (Character.isLetter(character) || character == '_')
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '_')
{
currentLexeme += character;
character = nextChar();
}
currentToken = testToken(currentLexeme);
}
else if (Character.isDigit(character))
{
while (Character.isLetter(character) ||
Character.isDigit(character) || character == '.')
{
currentLexeme += character;
character = nextChar();
}
currentToken = Token.CONSTANT;
}
else if ((currentToken =
testOperator()) != Token.NOT_FOUND)
;
else if ((currentToken =
testSeparator()) != Token.NOT_FOUND)
;
else
currentToken = Token.NOT_FOUND;
}
while (currentToken == Token.COMMENT ||
currentToken == Token.COMPILER_DIRECTIVE);
return currentToken;
}
```

```
// Puts back the last token that was gotten.

public void putLastToken()
{
lastToken = currentToken;
}

// Returns the lexeme corresponding to the last token.

public String getLastLexeme()
{
return lastLexeme;
}

// Returns the next character in the input buffer.

private char nextChar()
{
try
{
if (line == null)
return 0;
if (i == line.length())
{
line = file.readLine();
i = 0;
return '\n';
}
return line.charAt(i++);
}
catch (IOException exception)
{
return 0;
}
}
```

```
// testOperator will return the token type if it is an operator.
// Otherwise, it returns NOT_FOUND. Spacing is set for some
// operators. Comments are ignored.

private Token testOperator()
{
char lastCharacter;

switch(character)
{
case '+':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '+')
{
currentLexeme += character;
character = nextChar();
return Token.PRE_OR_POST_UNARY_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '-':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
```

```
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '-')
{
currentLexeme += character;
character = nextChar();
return Token.PRE_OR_POST_UNARY_OPERATOR;
}
else if (character == '>')
{
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE |
SUPPRESS_LEADING_SPACE;
return Token.STRUCTURE_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '*':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
case '%':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
```

```
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '>':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else if (character == '>')
{
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '<':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
```

```
character = nextChar();
return Token.BINARY_OPERATOR;
}
else if (character == '<')
{
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '&':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '&')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.UNARY_OR_BINARY_OPERATOR;
```

```
case '|':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
else if (character == '|')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.BINARY_OPERATOR;
case '=':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.BINARY_OPERATOR;
}
else
return Token.ASSIGNMENT_OPERATOR;
case '!':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
```

```
return Token.BINARY_OPERATOR;
}
else
{
spacing = SUPPRESS_TRAILING_SPACE;
return Token.UNARY_OPERATOR;
}
case '/':
currentLexeme += character;
character = nextChar();
if (character == '=')
{
currentLexeme += character;
character = nextChar();
return Token.ASSIGNMENT_OPERATOR;
}
if (character == '*')
{
currentLexeme += character;
character = nextChar();
do
{
lastCharacter = character;
character = nextChar();
}
while (character != '/' || lastCharacter != '*');
character = nextChar();
return Token.COMMENT;
}
else
return Token.BINARY_OPERATOR;
case '~':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
```

```
return Token.UNARY_OPERATOR;
case '.':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE |
SUPPRESS_LEADING_SPACE;
return Token.STRUCTURE_OPERATOR;
case '?':
currentLexeme += character;
character = nextChar();
return Token.TERNARY_OPERATOR;
default:
return Token.NOT_FOUND;
}
}

// testSeparator will return the token type if it is a separator,
// otherwise, it returns NOT_FOUND. Compiler directives are
// printed out as they are found.

private Token testSeparator()
{
switch (character)
{
case ':':
currentLexeme += character;
character = nextChar();
return Token.COLON;
case '(':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
return Token.LEFT_PARENTHESIS;
case ')':
currentLexeme += character;
```

```
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_PARENTHESIS;
case '[':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE | SUPPRESS_TRAILING_SPACE;
return Token.LEFT_BRACKET;
case ']':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_BRACKET;
case '{':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_TRAILING_SPACE;
return Token.LEFT_BRACE;
case '}':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.RIGHT_BRACE;
case ';':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.SEMICOLON;
case ',':
currentLexeme += character;
character = nextChar();
spacing = SUPPRESS_LEADING_SPACE;
return Token.COMMA;
case '#':
while (character != '\n' && character != 0)
```

```
{
currentLexeme += character;
character = nextChar();
}
output.endLine(false);
output.outputDirective(currentLexeme);
return Token.COMPILER_DIRECTIVE;
case '\'':
currentLexeme += character;
character = nextChar();
while (character != '\'')
{
if (character == '\')
{
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
return Token.CONSTANT;
case '"':
currentLexeme += character;
character = nextChar();
while (character != '"')
{
if (character == '\')
{
currentLexeme += character;
character = nextChar();
}
currentLexeme += character;
character = nextChar();
```

```
}
currentLexeme += character;
character = nextChar();
return Token.STRING;
default:
return Token.NOT_FOUND;
}
}

// testToken will return the token type if it is a token.
// Otherwise,it returns IDENTIFIER.

private Token testToken(String lexeme)
{
switch (lexeme.charAt(0))
{
case 'a':
if (lexeme.equals("auto"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'b':
if (lexeme.equals("break"))
return Token.BREAK;
else
return Token.IDENTIFIER;
case 'c':
if (lexeme.equals("case"))
return Token.CASE;
else if (lexeme.equals("char"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("continue"))
return Token.CONTINUE;
else
return Token.IDENTIFIER;
```

```
case 'd':
if (lexeme.equals("default"))
return Token.DEFAULT;
else if (lexeme.equals("double"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("do"))
return Token.DO;
else
return Token.IDENTIFIER;
case 'e':
if (lexeme.equals("else"))
return Token.ELSE;
else if (lexeme.equals("entry"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("extern"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'f':
if (lexeme.equals("for"))
return Token.FOR;
else if (lexeme.equals("float"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'g':
if (lexeme.equals("goto"))
return Token.GOTO;
else
return Token.IDENTIFIER;
case 'i':
if (lexeme.equals("if"))
return Token.IF;
else if (lexeme.equals("int"))
return Token.TYPE_SPECIFIER;
```

```
else
return Token.IDENTIFIER;
case 'l':
if (lexeme.equals("long"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'r':
if (lexeme.equals("register"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("return"))
return Token.RETURN;
else
return Token.IDENTIFIER;
case 's':
if (lexeme.equals("short"))
return Token.TYPE_SPECIFIER;
else if (lexeme.equals("sizeof"))
return Token.SIZEOF;
else if (lexeme.equals("static"))
return Token.SC_SPECIFIER;
else if (lexeme.equals("status"))
return Token.STATUS;
else if (lexeme.equals("struct"))
return Token.STRUCT;
else if (lexeme.equals("switch"))
return Token.SWITCH;
else
return Token.IDENTIFIER;
case 't':
if (lexeme.equals("typedef"))
return Token.SC_SPECIFIER;
else
return Token.IDENTIFIER;
case 'u':
```

```
if (lexeme.equals("union"))
return Token.UNION;
else if (lexeme.equals("unsigned"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'v':
if (lexeme.equals("void"))
return Token.TYPE_SPECIFIER;
else
return Token.IDENTIFIER;
case 'w':
if (lexeme.equals("while"))
return Token.WHILE;
else
return Token.IDENTIFIER;
default:
return Token.IDENTIFIER;
}
}
}
```

The next class is the Format class, which defines the formatter. It uses a recursive descent parser. It repeatedly calls the scanner to get another token as needed, and it calls the output object to specify indentation and to output tokens and new lines when appropriate.

The methods to format the if, while, do, and for statements have been omitted. Your instructor may elect to assign you a programming project that entails completing those methods so you will understand better the process of recursive descent parsing.

You should note that the code provided here will not work properly on the sample C program that determines the first four perfect numbers because the formatting for the if, while, for, and do statements is not implemented. Once you have added

this code to the Format class, the case-study example will perform the correct formatting.

```
// The Format class contains the methods necessary for formatting a
// C program. Only the file method is public. Recursive descent
// parsing is used to parse the program and perform the formatting.

class Format
{
private Lexer lexer;
private Output output;
private Token token;

// The constructor establishes the input lexer and the output
// private data members.

public Format(Lexer lexer, Output output)
{
this.lexer = lexer;
this.output = output;
}

// file is the only public method. External
// declarations are formatted until a function is found, at
// which time functionBody is called to format it.

public void file()
{
token = lexer.getNextToken();
while (token != Token.END_OF_FILE)
if (externalDeclaration())
functionBody();
}

// functionBody formats the declarations and statements in a
```

```
// function body.

private void functionBody()
{
output.endLine(true);
while (token == Token.TYPE_SPECIFIER ||
token == Token.SC_SPECIFIER ||
token == Token.STRUCT || token == Token.UNION ||
token == Token.UPPER_CASE_IDENTIFIER)
parameterDeclaration();
output.indent();
output.endLine(false);
output.skipLine();
compoundStatement();
output.unindent();
output.endLine(false);
output.endPage();
}

// compoundStatement formats a multiple statement block

private void compoundStatement()
{
int noOfDeclarations= 0;
token = lexer.getNextToken();
output.endLine(false);
while (token == Token.TYPE_SPECIFIER ||
token == Token.SC_SPECIFIER ||
token == Token.STRUCT || token == Token.UNION ||
token == Token.UPPER_CASE_IDENTIFIER)
{
declaration();
noOfDeclarations++;
}
if (noOfDeclarations > 0)
```

```
output.skipLine();
while (token != Token.RIGHT_BRACE)
statement();
token = lexer.getNextToken();
output.endLine(false);
}

// statement determines the type of statement and calls the
// appropriate function to format it.

private void statement()
{
if (token == Token.IDENTIFIER)
if ((token = lexer.getNextToken()) == Token.COLON)
token = lexer.getNextToken();
else
{
token = Token.IDENTIFIER;
lexer.putLastToken();
}
switch (token)
{
case LEFT_BRACE:
compoundStatement();
break;
case SWITCH:
switchStatement();
break;
case BREAK:
case CONTINUE:
verifyNextToken(Token.SEMICOLON);
output.endLine(false);
break;
case RETURN:
if ((token = lexer.getNextToken()) != Token.SEMICOLON)
```

```
{
expression(Token.SEMICOLON);
token = lexer.getNextToken();
}
else
token = lexer.getNextToken();
output.endLine(false);
break;
case GOTO:
verifyNextToken(Token.IDENTIFIER);
verifyNextToken(Token.SEMICOLON);
output.endLine(false);
break;
default:
expression(Token.SEMICOLON);
token = lexer.getNextToken();
output.endLine(false);
}
}

// switchStatement formats a switch statement.

private void switchStatement()
{
verifyNextToken(Token.LEFT_PARENTHESIS);
expression(Token.RIGHT_PARENTHESIS);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
verifyCurrentToken(Token.LEFT_BRACE);
output.endLine(false);
while (token == Token.CASE || token == Token.DEFAULT)
{
if (token == Token.CASE)
{
```

```
expression(Token.COLON);
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
while (token != Token.CASE && token !=
Token.DEFAULT && token != Token.RIGHT_BRACE)
statement();
output.unindent();
}
else
{
expression(Token.COLON);
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
output.endLine(false);
output.indent();
while (token != Token.CASE && token != Token.DEFAULT &&
token != Token.RIGHT_BRACE)
statement();
output.unindent();
}
}
verifyCurrentToken(Token.RIGHT_BRACE);
output.endLine(false);
output.unindent();
}

// externalDeclarations formats external declarations such as
// global variables and function prototypes. It returns if
// it encounters a function heading.

private boolean externalDeclaration()

{
```

```
int braceCount = 0;
boolean indentAtSemicolon = false;
Token lastToken = Token.NOT_FOUND;
while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
lastToken = token;
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
lastToken = token;
token = lexer.getNextToken();
indentAtSemicolon = true;
braceCount--;
}
else if (token == Token.LEFT_PARENTHESIS)
{
lastToken = token;
token = lexer.getNextToken();
}
else if (token == Token.RIGHT_PARENTHESIS)
{
lastToken = token;
token = lexer.getNextToken();
if (token != Token.SEMICOLON)
return true;
}
else if (token == Token.ASSIGNMENT_OPERATOR)
```

```
while (token != Token.SEMICOLON)
{
lastToken = token;
token = lexer.getNextToken();
lexer.checkExpressionSpacing(token, lastToken);
}
else if (token == Token.SEMICOLON)
{
lastToken = token;
token = lexer.getNextToken();
if (braceCount > 0)
output.endLine(false);
if (indentAtSemicolon)
{
output.indent();
indentAtSemicolon = false;
}
}
else
{
lastToken = token;
token = lexer.getNextToken();
}
}

token = lexer.getNextToken();
output.endLine(false);
if (indentAtSemicolon)
output.indent();
return false;
}

// parameterDeclaration formats parameter declarations.

private void parameterDeclaration()
```

```
{
int braceCount = 0;
while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
token = lexer.getNextToken();
output.indent();
braceCount--;
}
else if ((braceCount > 0 ) && (token == Token.SEMICOLON))
{
token = lexer.getNextToken();
output.endLine(false);
}
else
token = lexer.getNextToken();
}
token = lexer.getNextToken();
output.endLine(false);
}

// declaration formats local declarations.

private void declaration()
```

```
{
int braceCount = 0;
boolean indentAtSemicolon = false;

while ((braceCount > 0) || (token != Token.SEMICOLON))
{
lexer.checkDeclarationSpacing(token);
if (token == Token.LEFT_BRACE)
{
output.endLine(false);
output.indent();
token = lexer.getNextToken();
output.endLine(false);
braceCount++;
}
else if (token == Token.RIGHT_BRACE)
{
token = lexer.getNextToken();
indentAtSemicolon = true;
braceCount--;
}
else if (token == Token.SEMICOLON)
{
token = lexer.getNextToken();
if (braceCount > 0)
output.endLine(false);
if (indentAtSemicolon)
{
output.indent();
indentAtSemicolon = false;
}
}
else if (token == Token.ASSIGNMENT_OPERATOR)
expression(Token.SEMICOLON);
```

```
else
token = lexer.getNextToken();
}
token = lexer.getNextToken();
output.endLine(false);
if (indentAtSemicolon)
output.indent();
}

// expression formats an expression. The delimiting token must
// be provided.

private void expression(Token terminator)
{
Token lastToken;

lastToken = Token.NOT_FOUND;
while (token != terminator)
{
lexer.checkExpressionSpacing(token, lastToken);

if (token == Token.LEFT_PARENTHESIS)
{
if (lastToken == Token.IDENTIFIER ||
lastToken == Token.UPPER_CASE_IDENTIFIER)
lexer.adjustSpacing(Lexer.SUPPRESS_LEADING_SPACE);
token = lexer.getNextToken();
expression(Token.RIGHT_PARENTHESIS);
}
lastToken = token;
token = lexer.getNextToken();
}
}

// Gets the next token and then verifies that the supplied token is
```

```
// the required token.

private void verifyNextToken(Token requiredToken)
{
token = lexer.getNextToken();
verifyCurrentToken(requiredToken);
}

// Verifies that the supplied token is the current token.
// Displays an error message if it is not.

private void verifyCurrentToken(Token requiredToken)
{
if (token != requiredToken)
output.outputError("MISSING " + requiredToken.name());
else
token = lexer.getNextToken();

}
}
```

The final class is Output. The singleton object of this class defined in main performs the necessary page formatting of the output. It ensures that each line is no longer than some maximum length and that each page contains no more than a specific number of lines. It also maintains the current indentation level so lines can be properly indented. It also provides the appropriate spacing between tokens.

```
// The Output class controls the formation of lines and pages in the
// output. It also provides explicit functions for controlling the
// indentation and forcing new lines and pages.

import java.io.*;

class Output
{
```

```
private static final int INDENT_INCREMENT = 4, LEFT_MARGIN = 0,
LINES_PER_PAGE = 56, HEADING_LENGTH = 70,
CHARACTERS_PER_LINE = 78;
private PrintWriter file;
private int linesOnPage;
private int pageNumber;
private int indentation;
private String buffer = "";
private String heading;

// The constructor initializes the private instance variables.
// It constructs a page heading containing the input file name.

public Output(String fileName)
throws FileNotFoundException, IOException
{
file = new PrintWriter(new FileWriter(fileName + "_.c"));
linesOnPage = LINES_PER_PAGE;
pageNumber = 1;
indentation = LEFT_MARGIN;
heading = fileName;
for (int i = 0; i < HEADING_LENGTH - fileName.length(); i++)
heading += ' ';
}

// Closes the output file.

public void close()
{
file.close();
}

// outputToken outputs the token string, adjusting spacing
// specified by the spacing word.
```

```
public void outputToken(String token, int spacing)
{
if (buffer.length() + token.length() > CHARACTERS_PER_LINE)
{
outputLine(buffer);
buffer = "";
}
if ((spacing & Lexer.SUPPRESS_LEADING_SPACE) != 0)
if (buffer.length() > 0 &&
buffer.charAt(buffer.length() - 1) == ' ')
buffer = buffer.substring(0, buffer.length() - 1);
buffer += token;
if ((spacing & Lexer.SUPPRESS_TRAILING_SPACE) == 0)
buffer += ' ';
}

// outputDirective prints out a compiler directive starting at the
// left margin.

public void outputDirective(String directive)
{
outputLine(directive);
}

// outputError prints out error messages.

public void outputError(String error)
{
file.println(error);
}

// indent increments the indentation variable.

public void indent()
{
```

```
indentation += INDENT_INCREMENT;
}

// unindent decrements the indentation variable.

public void unindent()
{
indentation -= INDENT_INCREMENT;
}

// endLine calls outputLine to write out a line. If the parameter
// forceNewPage is true, then newPage is called.

public void endLine(boolean forceNewPage)
{
if (forceNewPage && (linesOnPage > 0))
newPage();
if (buffer != "")
outputLine(buffer);
buffer = "";
}

// skipLine skips a line.

public void skipLine()
{
outputLine("");
}

// endPage sets linesOnPage to force a call to newPage.

public void endPage()
{
linesOnPage = LINES_PER_PAGE;
}
```

```
// newPage does a form feed and prints a new page heading.

public void newPage()
{
file.println("\f" + heading + "PAGE " + pageNumber++);
linesOnPage = 0;
}

// outputLine fills up the number of spaces in the margin
// and prints a line from the buffer.
// It then increments lines per page.

public void outputLine(String line)
{
String buffer = "";

if (linesOnPage >= LINES_PER_PAGE)
newPage();
for (int space = 0; space < LEFT_MARGIN + indentation; space++)
buffer += ' ';
buffer += line;
file.println(buffer);
linesOnPage++;
}
}
```

Module 2: Imperative Languages—Data Control and Data Types

Topics

1. Data Control
2. Scalar Data Types
3. Structured Data Types
4. Pointers and References

I. Data Control

Before we discuss the issues that relate to the different kinds of data types, we begin with a variety of issues that concern data types in general. These issues are sometimes referred to as *data control*. Many of these data-control issues are central to the design of a compiler's symbol table.

In module 1, we discussed the syntax of identifiers or names, which in most languages consist of some combination of letters and digits and perhaps also a few special characters—underscores being the most common. The identifier token is unique among the tokens of programming languages in that it is the only one that does not have a predefined meaning. Most lexemes that are identifiers are given meaning through declarations, which are then stored in the symbol table. There are, however, some lexemes that syntactically are identifiers that do have predefined meanings. These lexemes include reserved words, keywords, and predefined names.

Languages vary greatly with regard to the number of reserved words they contain. Making a name a reserved word emphasizes the importance of the concept associated with that word. Among popular imperative languages, Ada has the most

reserved words. At the other extreme is LISP, with none. Deciding which features of the language are important enough to warrant reserved words is a key decision that influences how easy a language is to learn.

When we examine particular language features, we may occasionally consider the number of reserved words associated with that feature. Concurrency is a good example. Ada's implementation of concurrency involves a dozen or more reserved words, whereas Java has only one. In module 4, we will examine the implications of that design decision.

A. Binding

A commonly used term in programming language design is *binding*. Among the important issues related to binding is when it occurs. These binding times are referred to as either *static binding* or *dynamic binding*

We will examine two kinds of binding, the binding of variables to types and the binding of variables to memory addresses.

Variable-to-Type Binding

We begin with a discussion of the binding of variables to types. The first variation among languages is whether variable declarations are required or not. We first consider languages that do not require variable declarations.

Implicit Declarations

In some cases, the type of the variable is determined by using a naming convention that implicitly specifies the type. Such *implicit declarations* are now rare among compiled languages, but Fortran still uses such a convention. In Fortran, variables that begin with the letters *I* through *N*, if not otherwise declared, are taken to be integers. This convention is based on the practice in mathematics of using letters, such as *i* and *j*, as subscripts.

A criterion that we should use to judge the quality of a language design decision, one that we did not mention in module 1, is how that decision influences error detection. All programmers recognize that the earlier in the software development process an error occurs, the easier it is to fix. In particular, compilation errors are much easier to fix than logic errors because logic errors are never reported and

must be uncovered by debugging. Although allowing implicit declarations makes programs easier to write because one is spared having to declare variables, and perhaps easier to read because its type is apparent from its name, there is a significant disadvantage, which involves the late detection of errors. Suppose one mistakenly writes the following line of Fortran code:

```
ICOUNT = ICOUMT + 1
```

A typographical mistake, like this one, is easy to make. In a language that requires explicit declaration of all variables, this line would produce a compilation error, specifically the static semantic error *undeclared variable*. In Fortran, however, this mistake results in a logic error. Because logic errors are so much more costly to fix than compilation errors, this language design decision can have an adverse effect by increasing the cost of writing such Fortran programs.

Another drawback to such implicit typing is that it does not extend well. If a language allows user-defined types, there is no easy way to have it apply to such types. Because of these two drawbacks, most modern compiled languages do not use naming conventions as a way to specify type.

Another way the type of a variable can be determined when it is implicitly declared is referred to as *type inference*. Although rarely used in compiled languages, there is one instance in Ada where type inference is used—the Ada for statement, as illustrated in the following example:

```
for (I in 1..10)
  Put(I);
```

Loop-control variables inside for loops are implicitly declared. In fact, if the variable I is declared in an outer block, it is a different variable from the I inside the loop. The reason for making these declarations implicit was so that the scope of the variable could be guaranteed to not extend beyond the body of the loop. In this case, it is not the name that is used to conclude that I is an integer but the type of

the constants, which specify the lower and upper bounds of the loop-control variable. Although Ada prohibits any real types in this context, it allows any discrete type, which includes enumeration types.

In both the Fortran and Ada examples of implicit declarations, the binding is static—it happens at compile time. Although implicit declarations are rare in compiled languages, they are common among interpreted languages. Because interpreted languages are not compiled, the binding must be dynamic. Some interpreted languages, such as LISP, are essentially typeless. In such languages, no variable-type binding ever occurs.

Explicit Declarations

Next, let's consider languages that have explicit declarations. As we have seen in compiled languages, the variable-type binding is generally static even when variables are implicitly declared. The same is certainly true for declared variables. There is, however, a case where binding becomes dynamic—in object-oriented languages. In such contexts, we generally refer to objects rather than variables. Consider the following Java declaration:

```
Object obj = newObject();
```

In this situation, the type of obj is not determined until run time, so the binding is dynamic. What can be checked at compile time is that the method newObject, sometimes called a *factory method*, returns a type that is either the declared type or any type derived from it.

Variable-to-Storage Binding

The next kind of binding that we consider is the binding of a variable to a memory address. Unlike variable-type binding, the division between static and dynamic binding does not sufficiently distinguish the various cases because there are cases in which some aspect of the address can be determined at compile time but another aspect at run time. So, we consider the various cases along a spectrum from most static to most dynamic.

The first case is generally referred to simply as *static variables*. All variables in the early versions of Fortran fall into this category. Global variables and local variables prefaced with the storage class specifier static in C or C++ are other examples of static variables. The addresses of such variables are determined at compile time, but even in this case, we must qualify the exactness of this determination. Most compilers produce what is called *relocatable object code*, which as the name suggests, can be loaded at different locations in memory. Some compilers produce what are sometimes called *core images*, which contain absolute addresses, but because of the inflexibility of such object code, it is not a common practice.

Furthermore, in a machine that has virtual memory, we can never be sure of the absolute address of a variable because it can change throughout the execution of a program.

One advantage of a language that allows only static variables is that once the program is compiled, we can with certainty know how much memory the program will require when it runs, because the memory requirements of such programs are constant—O(1). As soon as a language allows any more dynamic kind of binding, we introduce the possibility of programs that require an unbounded amount of memory when they run.

Any programming language that allows recursion must allow a more dynamic form of variable-storage binding. One reason is that the local variables of any recursive subprogram can have an unlimited number of instances—one for each activation of the subprogram. Although we observed in module 1 that recursion properly belongs to the domain of declarative languages, most modern imperative languages support it. To be able to support recursion, it is necessary for programming languages to allow stack-dynamic variables. The name suggests that such variables are allocated on the compiler's run-time stack.

It is possible to some degree to compute the address of a stack-dynamic variable at compile time. Its address can be computed relative to the beginning of the activation record in which it resides. The code below illustrates how the relative addresses of such variables can be computed in a Java method.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
void m()
{
    int i;
    double d;
    byte b;
    ...
}
```

The actual address is the sum of the address of the activation record, which can only be determined at run time, plus the relative address inside the activation record, which can be computed at compile time.

In languages, such as C, C++, Java, and C#, that do not allow nested subprograms, only the local variables of the current subprogram can be accessed, so the address relative to the beginning of the subprogram is all that must be determined at compile time. But in languages that allow nested subprograms, such as Algol, Pascal, and Ada, another component of address information must be determined at compile time—which activation record the local variable is in. The Ada subprogram shown below illustrates this point.

Move your mouse over the highlighted part of the program to see an explanation of that part.

```
procedure One is
    X: Integer;
    procedure Two is
       procedure Three is
       begin
           X := 1;
       end Three;
    begin
        null;
    end Two;
begin
    null;
end One;
```

When we discuss scope rules later in this module, we will elaborate on why local variables of other subprograms are accessible when subprograms are nested. The distance, measured in levels of nesting, is still not sufficient information to locate the actual activation record. In module 3, we will return to this issue and further explain what other information must be maintained at run time to locate the actual activation record.

The most dynamic binding between variables and memory addresses is *heap-dynamic* binding. The memory for variables bound in this fashion comes from the heap. No address information whatsoever can be determined for such variables at compile time. It is done exclusively at run time.

There are two varieties of such bindings. The first variety is explicit and involves a specific instruction coded by the programmer to cause memory allocation. The reserved word new is used in several imperative languages, including C++, Ada, Java, and C#, to request such explicit allocation. Languages that provide explicit heap allocation have either pointers or references associated with this kind of allocation. Some have both. Pointers and references introduce the possibility of aliasing in the language.

Historically, languages that had explicit heap allocation required explicit heap deallocation. This requirement exists in Pascal, C, and C++. Explicit deallocation in C++, for example, is accomplished with the reserved word delete. Ada began a movement toward implicit deallocation despite explicit allocation. Ada compilers are given the option to perform what is usually called *automatic garbage collection*. In Java and C#, automatic garbage collection is required.

Automatic garbage collection was first included in languages such as LISP that use *implicit heap-dynamic* binding—the other variety of heap dynamic binding. In such languages, the programmer never explicitly requests memory allocation. Instead, certain operations, by their nature, must perform it. When we discuss LISP in module 5, we will explain what causes implicit allocation of memory from the heap.

B. Type Checking

In module 1, we introduced static semantics and the rules that define whether a program is meaningful. We noted that it is the job of the semantic-analyzer phase of the compiler to check whether a program adheres to these rules. Many of the rules involve type checking. Checking done at compile time is called *static type checking*. When done at run time, it is called *dynamic type checking*. Compiled languages favor the former.

Types must be checked in a variety of different places in the syntax of most languages. Languages that check types in every possible context are said to be *strongly typed languages*.

Type Checking Expressions

The first place that type checking must occur is inside expressions. It must be verified that the types of the operands are valid for that particular operator. In most languages, the remainder operator is defined only for integral types, so attempting to use it in such languages with floating-point operands would be an error.

The other check that must be made is whether, for binary operators, the types of the two operands are compatible. If they are compatible but not identical, type coercion must be performed.

We will elaborate shortly on the way in which type compatibility is determined for user-defined types. For now, let's consider the typical rules for primitive types. In most languages, all numeric types are compatible, and when not identical, the "smaller" type is coerced to the "larger" one.

What varies among languages is whether character types are considered compatible with numeric types. In the C-family of languages, they are considered compatible, but in Pascal and Ada, they are not. A similar variability exists with whether Boolean and enumeration types are considered compatible with numeric types.

When coercion promotes a numeric expression to a "larger" type, usually no unexpected results occur, but some kinds of coercion do produce unexpected results. In C and C++, unsigned types, which we encountered in module 1 as an illustration of orthogonality, are such an example. The rule in those languages is that when unsigned and signed types are mixed, the signed type is converted to an unsigned type, which can lead to unexpected results. Consider the following C++ code segment:

```
signed int s = -1;
unsigned int u = 0;
cout << s + u << endl;
```

The output will be the largest possible unsigned integer. The problem with coercion between signed and unsigned types is that the set of values of neither one is a subset of the other, so no matter which direction coercion is performed, unexpected results can occur.

Type Checking Assignments

Type checking must also be performed on assignment statements. The variable on the left side of the assignment must be checked against the type of the expression on the right. In some languages, the type compatibility rules on assignments are the same as for expressions, which is the case in both C and C++. As a result of this design decision, unexpected results can occur on assignments apart from those associated with unsigned types. Consider the following C++ code segment:

```
int i = 50000;
short s = i;
cout << s << endl;
```

Assuming that the type short is two bytes, the output will be −15536, which is clearly an unexpected result. The problem in this case is that the variable on the left side of an assignment cannot be coerced to another type, and the space reserved for the variable s is not large enough to store 50000. Consequently, the high-order bits are lost, which produces this unexpected result.

When we discuss assignment statements in module 3, we will describe how the rules for type checking on assignments differ in Java.

Type Checking Subprogram Calls

Many languages also perform type checking on subprogram calls. Two checks can be made. The type of the actual parameters can be checked against their corresponding formal parameters. The calls to subprograms that return values are expressions. Consequently, these calls can be placed wherever an expression is allowed, so the return type of the expression can be checked to ensure it is placed in a compatible context.

Failing to perform such type checking of actual against formal parameters can result in logic errors, or worse, may even compromise the compiler's run-time stack at execution time. This lack of type checking was a serious problem in the early versions of C. We will explore how separate compilation is related to this problem and the way in which C++ solves it, when we discuss subprogram parameters in module 3.

Type Checking Control Statements

Type checking may also be appropriate on some control statements. In many languages, such as Ada and Java, the type of the expression embedded in an if or while statement must be Boolean. The switch statement in the C family of languages also must be type checked. The expression in that statement must be an integral type. Languages with case statements often have similar rules.

C. Type Compatibility among User-Defined Types

Until now, in our discussion of type checking, we have discussed only type compatibility among primitive types. Most new languages allow user-defined types, and some allow a greater freedom in defining types than others. Ada allows perhaps the greatest variety of user-defined types. In Ada, the user can define new integer types, real types, enumeration types, array types, record types, access types, and others.

In addition to the wide range of user-defined types, Ada also allows explicit derived types and subtypes. To illustrate this range of possibilities, let's consider three variations of the way in which one might define an integer type for the days in a month. The first approach employs a user-defined integer type. That definition is shown below:

```
type DaysInMonth is range 1..31;
```

The type DaysInMonth is not type compatible with the predefined type Integer or any other user-defined integer type. However, it is compatible with all integer literals within its range.

The second possibility is to define the type explicitly as a derived type, as follows:

```
type DaysInMonth is new Integer range 1..31;
```

The reserved word new indicates an explicit derived type. Like the previous type, it is incompatible with the predefined type Integer or any other user-defined integer type.

The third possibility is to use subtypes as follows:

```
subtype DaysInMonth is Integer range 1..31;
```

When a subtype is used, the subtype is compatible with the original type. Pascal has a similar concept referred to as *subranges*.

Clearly, two variables of different kinds of types are never compatible in Ada, so for example, no array variable is ever compatible with a record variable. Therefore, when compatibility exists, it is only between variables of the same kind. Ada uses

name type compatibility. Because Ada requires almost all types to be named, it is easy to implement name type compatibility. There are, however, some cases in which anonymous types are permitted. Arrays, tasks, and protected objects can be anonymous.
When name compatibility is used and anonymous types are permitted, some additional rules must be defined to accommodate these kinds of types. Consider the following example of arrays with anonymous types:

```
Array1, Array2: array(1..5) of Float;
```

Even in the above situation, Ada considers Array1 and Array2 to be of different types.

In the C family of languages, user-defined types are more limited. In C, structures, unions, and enumeration types can be given names. Arrays and pointer types are not named. Although a typedef statement is included, it does not really create a new type in the sense that Ada type declarations do. C++ added classes to the previous list, although it is not necessarily true that every class name is actually a type name. The names of classes with exclusively static members are not really types. Java dropped structures and unions but added interfaces.

Because many types in C are anonymous, it uses structure type compatibility for the unnamed types, and name type compatibility for the named ones.

One final issue that affects type compatibility is inheritance. In most languages that support inheritance, types in an inheritance hierarchy are compatible with any ancestor type but not any descendant types. The former conversions are often referred to as *upcasts*, and the latter are referred to as *downcasts*.

D. Scope and Lifetime

Although scope and lifetime are related concepts, they are not the same. The scope of a declaration is some portion of the program in which it is declared. *Lifetime* refers to a time period during the execution of a program when the declared variable, constant, and so forth, exist.

Scope Rules

The scope rules of most modern programming languages have their roots in Algol, which was the first block-structured language. Algol and its descendants, Pascal and Ada, allow nested subprograms. None of the languages in the C family do, but all of them allow nested blocks.

In languages that do not permit nested subprograms, such as C, the scope rule for function names is simple. The scope of all function names includes the whole program, so any function in a C program can call any other function in that program. When nested subprograms are permitted, a subprogram can only call other subprograms whose names it can see. Consider the following Ada program:

```
procedure Outer is
    procedure Inner1 is
    begin
        null;
    end Inner1:
    procedure Inner2 is
    begin
        null;
    end Inner2;
begin
    null;
end Outer;
```

In the above program, Outer can call itself and both Inner1 and Inner2. Inner1 can call itself and Outer, but not Inner2. Inner2, like Outer, can call all three.

Regardless of whether the languages support nested subprograms, the scope rules for local names in most languages are the same. The scope of a declaration is from the declaration to the end of the block in which it is declared. Generally, a name is visible through its scope.

Most languages allow a name to be declared in a block and again declared in a block nested inside it. When a name is redeclared in such a fashion, it will not be visible throughout its scope. Consider the following program segment in C or C++:

The Scope of the Outer x (highlighted)	Where the Outer x Is Visible (highlighted)

`{`
`  int x;`
`  {`
`  int x;`
 	
`  x = 1;`
`  }`
`  x = 2;`
`}`
	`{`
	`  int x;`
	`  {`
	`  int x;`
	
	`  x = 1;`
	`  }`
	`  x = 2;`
	`}`

There are other cases where the scope and visibility of identifiers differ, for example, when a class is declared inside a method in a language like Java.

When names are redeclared in this fashion, some languages allow them to be accessed if they are qualified. In Ada, by prepending a name with the block or subprogram name in which it is defined, it is possible to access a name that would otherwise be hidden. In C++, when a global name is hidden by a local of the same name, prepending the name with the scope resolution operator :: makes the global name accessible.

Java is one of the few languages that disallows duplicate local declarations with overlapping scopes as illustrated by the above example. It does allow duplicate local names within the same method if the scopes are disjoint. It also allows instance variables and local variables with the same name inside a class. In that case, the instance variable is hidden by the local variable. Java uses the term *shadowing* to describe such situations.

In Ada, package specifications are separate from package bodies. In C++, function bodies, unless they are inline, are separated from the class definition. In Java, however, the method bodies are embedded in the class definition. Consequently,

the scope rules must be stated in a slightly different way. In a Java class, from the perspective of scope, it is as though all method bodies follow all data declarations and method signatures.

The scope rules that we have described until now are called *static scope rules*. The adjective *static*, as usual, suggests the fact that with such scope rules, scope can be determined at compile time. Although early versions of LISP used dynamic scope rules, few languages use them now. With dynamic scope rules, the local variables of all activation records on the compiler's stack are potentially accessible. Furthermore, depending upon the sequence of subprogram calls, a nonlocal name may reference different activation records if different execution paths are taken.

Lifetime

As we mentioned earlier, the lifetime of a variable is the time period during the execution of the program when it exists. C and C++ are good languages to examine to understand the difference between scope and lifetime. The lifetime of a local variable, which is stack-dynamic by default, is the lifetime of the function in which it is declared. Its life begins when the function is called and ends when the function completes. Consequently, the local variables of a function that is called multiple times have multiple lifetimes. By contrast, global variables, which are allocated as static variables, have lifetimes that are the lifetime of the program.

The connection between scope and lifetime is that local variables have function-wide scope and function-long lifetimes, whereas global variables have program-wide scope and program-long lifetimes. Because of this connection, the distinction between them is sometimes lost. What highlights the difference is the fact that C and C++ allow static local variables, which are a hybrid. They have function-wide scope but program-long lifetimes. Consider the following example in C++.

Move your mouse over the highlighted parts of the method for an explanation of those parts.

```
#include <iostream>
using namespace std;

int global = 0;

void increment()
{
    auto int local_1 = 0;
    static int local_2 = 0;

    cout << global++ << " " << local_1++ << " " << local_2++ <<
}

int main()
{
    for (int i = 1; i <= 4; i++)
        increment();
    return 0;
}
```

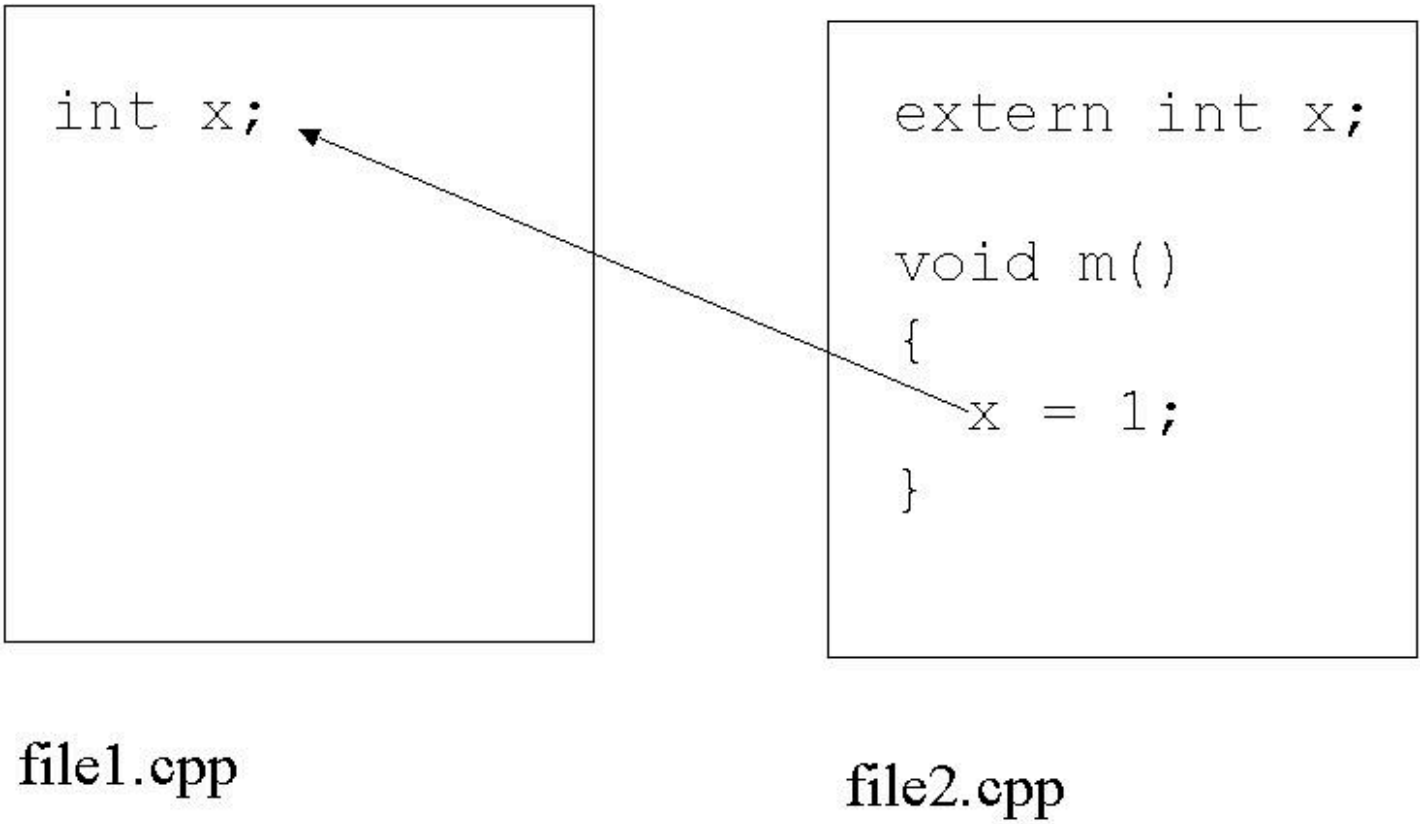
Once you understand the lifetimes associated with each of these three kinds of variables, you should be able to predict the output of this program, which is shown below.

0 0 0
1 0 1
2 0 2
3 0 3

Because global variables and static local variables both have program-long lifetimes, they behave in the same way in this program.

Another aspect of the storage class specifier static in C and C++ warrants comment. Because global variables are allocated by default as static variables, labeling them static would be redundant, much as labeling local variables auto is redundant. Consequently, the designers of C elected to have the storage class specifier static have a different meaning when applied to global variables. To understand its meaning, it is necessary to elaborate on the scope rules for global variables. We had indicated earlier that global variables have program-wide scope. That is true, but global variables can be referenced outside their source file only if they are imported using a declaration containing the storage class specifier extern. Figure 2.1 illustrates how such cross-file references are made.

Figure 2.1
Referencing a Global Variable of a Different Source File



If static is applied to the declaration of x in file1.cpp in the above example, the global variable x will no longer be accessible from file2.cpp. In that case, the linker will generate an error indicating that there is an external reference that cannot be resolved. So, static has the effect of limiting the scope of the global variable.

In C and C++, static applied to local variables extends their lifetime, but when applied to global variables, it limits their scope. We have gone through the effort of explaining this complicated situation primarily because it exemplifies a very poor language-design decision. Ascribing different meanings to the same reserved word in different contexts is never a good language-design choice. The fact that the meaning of static is so difficult to explain underscores this fact. What adds to the confusion is that scope and lifetime are themselves often confused, so having the same reserved words pertain to scope in one context and lifetime in another context compounds the confusion.

If static having two meanings in C were not bad enough, C++ added a third meaning in a third context. When applied to member variables of a class, it specifies that there is only one copy of the variable per class rather than one copy per object, effectively altering its lifetime. Member variables labeled static have a class-long lifetime rather than an object-long lifetime. To its credit, Java adopted this third meaning but dropped the original two.

We conclude with one final example in C that illustrates how important it is to understand variable lifetimes. This function accepts a C string as a parameter and copies it to a local string variable, changing each character to uppercase. It then returns a pointer to that converted string.

```
#include "ctype.h"

#define LENGTH 10

char *to_upper_case(char string[LENGTH])
{
    int index;
    char result[LENGTH];

    for (index = 0; index < LENGTH; index++)
        result[index] = toupper(string[index]);
    return (result);
}
```

No C compiler will produce any compilation errors for the above program. If your compiler is helpful, it may generate a warning, however, indicating that you are returning the address of a local variable. Anyone who would write such a program clearly does not understand the lifetime of stack-dynamic variables. The source of the problem is that this function is returning a pointer to a local string whose lifetime will end as soon as the function completes.

The behavior of such functions would confuse anyone who does not understand the way stack-dynamic variables are allocated. If the calling function immediately displayed what was returned, the function would appear to work because stack-dynamic variables are not erased when their lifetimes end. But if another function were called and afterward the converted string was displayed, garbage would most likely be output. In that case, the local string would have been overwritten by the local variables of the function last called.

II. Scalar Data Types

Now that we have completed our discussion of the issues related to data types, which we called *data control*, we turn our attention to the specific data types that are included in most programming languages. We begin with scalar data types,

which are typically predefined.
As is the case with so many of the comparisons we make with imperative languages, the approach taken by the C family of languages differs with regard to scalar data types from that taken by Ada and its ancestors. So, in the discussion that follows, we will explain both approaches.

A. Integer Types

In the C family of languages, there is a collection of predefined integer types of different sizes. As we mentioned earlier, in C and C++, these sizes are machine independent, but in Java, the language defines them. In C and C++, the integer types include short, int, and long. Java adds byte to that list. In all three languages, char can be used for integer data.

Although Ada also has predefined integer types of different sizes, as we saw earlier, it is possible to define new derived and subtypes for integer data. This ability allows the programmer to distinguish between integers used for a specific purpose and integers in general. In our earlier example, making DaysInMonth a derived type—either implicitly or explicitly—makes it type incompatible with the predefined type Integer and any other integer type. Both derived types and subtypes make run-time range checking possible, which improves the reliability of programs. Ada does not allow the type Character to be used for numeric data.

B. Real Types

The approach taken for real-valued numeric data in the C family is much the same as the approach taken for integer data. It provides several predefined sizes—all stored in floating-point format. C and C++ have three sizes: float, double, and long double. Java includes only the first two.

Ada's approach is quite different. First, Ada allows both floating-point and fixed-point representations. Although there are predefined floating-point types, the programmer is allowed to define new floating-point types unique to a particular application. An example of such a user-defined floating-point type definition is shown below:

```
type MyFloatType is digits 10 range 0.0..100000.0;
```

Floating-point representations allocate some number of bits for the mantissa and some for the exponent. The digits qualifier specifies the amount of required precision. By multiplying the request digits by the number of bits needed per digit, which is between 3 and 4, the number of bits needed for the mantissa can be computed.

In a similar fashion, the range qualifier could be used to compute the number of bits required for the exponent. Although this is an interesting language feature, user-defined floating-point types have limited practical value because, to be efficient, the floating-point type must be implemented in the hardware. The best the compiler can do with such information is to select the smallest hardware-defined floating-point type that is large enough to accommodate this definition. Having the compiler make the selection, however, does avoid problems with the portability of programs between machines with different floating-point hardware implementations.

Ada also allows fixed-point types to be defined. C, C++, and Java have no comparable types, but C# does have a type called decimal that we will discuss shortly. Let's consider an example of a fixed-point type in Ada:

```
type Dollars is delta 0.01 range 0..100_000;
```

The delta qualifier specifies the required amount of accuracy. One can determine how many bits are needed to the right of the decimal to provide this level of accuracy.

A key difference between fixed- and floating-point types is how the model numbers for such type definitions are distributed along a number line. With fixed-point types, they are evenly distributed. Suppose, for example, the fixed point were placed with four bits to the right. A model number would appear at every $1/16^{th}$ mark along the number line. With floating-point types, the model numbers are closely clustered near zero and more widely separated the farther from zero they are.

An inherent problem with any binary representation of real numbers, whether fixed or floating point, is the possibility of representation errors. This problem is particularly troublesome in business applications. One way to avoid such representation errors is to use binary-coded decimal.

Not surprisingly, COBOL has a decimal type, which uses BCD, as does C#.

C. Nonnumeric Types

Most languages provide some predefined scalar types for nonnumeric data such as character data. We have already observed how some languages allow character data to be used for integer data, but other languages do not.

Boolean Data

The other predefined scalar data type is Boolean. Again, some languages differentiate between integer data and Boolean data; others do not. In C and C++, the type bool is just a renaming of int. An integer value can be used as Boolean data, with 0 being treated as false and everything else being treated as true. Java, Ada, and its ancestor languages, by contrast, have true Boolean types.

Not distinguishing between Boolean and integer data can lead to some unfortunate misinterpretations of C and C++ programs. The following if statement in C++ illustrates this problem:

```
if (1 <= x <= 10)
    cout << "Between 1 and 10";
```

In this case, the attempt is to determine whether x is between 1 and 10, but the expression in the if condition will always evaluate to true because the first subexpression:

```
1 <= x
```

will evaluate to 0 if it is false and evaluate to 1—the nominal representation for true—otherwise. In either case, that result is less than or equal to 10, so the expression as a whole is true.

Experienced programmers know the proper way to write the intended expression is:

```
1 <= x && x <= 10
```

What makes this misinterpretation so troubling is the fact that writing 1 ≤ x ≤ 10 is perfectly good mathematical notation for expressing the desired condition. So, we have a situation that we previously described as one of the worst results in language design—the ability to write something that would be reasonably expected to mean one thing and have it be interpreted to mean something else.

In languages that have a true Boolean type, such expressions produce semantic errors—specifically, a type incompatibility between a Boolean and an integer.

Enumeration Types

In addition to providing these two predefined nonnumeric types, many languages allow the user to define nonnumeric types called *enumeration types*. In Ada, the boolean and character types are consider predefined enumeration types.

Pascal was among the first languages to allow enumeration types, so it is not surprising to find them in Ada. Although they were not a part of the earliest versions of C, they were subsequently added and, therefore, are also in C++. They were dropped originally in Java and only recently added in version 5.0.

Allowing the user to define an enumeration type has several advantages. It allows names to be used for the literals in place of numbers. One could argue that the same advantage could be achieved using named constants, which is true. However, a second advantage that simply naming constants does not achieve, is conveying that some set of constants constitutes a type. The final advantage is that checking can be done to ensure that variables of such types are never assigned improper values. Not all languages that include enumeration types provide the last advantage. C and C++ do not. The following program segment illustrates this fact:

```
enum PrimaryColors {RED, YELLOW, BLUE};
PrimaryColors color = (PrimaryColors) 10;
```

Although enumeration types are not implicitly compatible with integers, an integer can be type cast to any enumeration type, which is what is being done in the above code. Neither a compilation nor run-time error results, yet the variable color will contain an improper value.

In Ada, such explicit conversion of integers to enumeration types is prohibited. This lack of type-safety is likely one reason that Java initially chose to omit enumeration types. When Java incorporated enumeration types into the language in version 5.0, it was done in a way that is type-safe. In Java, enumeration literals are objects, not integers. Every enumeration type is implicitly derived from the predefined class Enum. Because enumeration literals are objects, it is possible to include user-defined data and methods. A Java enumeration type definition for primary colors, which illustrates these capabilities, is shown below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
import java.awt.*;
enum PrimaryColors
{
    RED(Color.RED),
    YELLOW(Color.YELLOW),
    BLUE(Color.BLUE);

    private Color color;

    private PrimaryColors(Color color)
    {
        this.color = color;
    }
    public Color getColor()
    {
        return color;
    }
}
```

All enumeration types in Java inherit several methods from Enum. The first is a method name, which returns the enumeration literals as a string, so RED.name() returns "RED". The second is ordinal, which returns an integer corresponding to the position of the literal in the type definition—RED.ordinal() being 0. Enum implements the Comparable interface so a compareTo method is defined that compares literals based on their ordinal values. Finally, a method values returns an array of all literals, which is useful in the *for-each* style for loop that was added in version 5.0 and that we will discuss in module 3.

Although Ada does not allow the user to associate new data or subprograms with enumeration types, there is a technique for accomplishing what most of the aforementioned methods inherited from Enum provide. We shall now discuss this technique in the broader context of discrete types.

D. Type Attributes

Associated with Ada's categorization of data types is the syntax of type attributes. Each category of types has an associated set of attributes. One such category is discrete types. Other languages sometimes refer to such types as *ordinal* or *integral* types. Included in the discrete-type category are integer types and enumerated types. Attributes can be either constants or subprograms. Let's consider the Ada implementation of our previous enumeration type of primary colors as an example:

```
type PrimaryColors is (Red, Yellow, Blue);
```

For this type, there is an attribute constant PrimaryColors'First whose value is Red. For every discrete type, the attribute 'Image is a function that returns the string representation of the enumeration literal, so for example, PrimaryColors'Image(Red) returns "Red". Similarly, there is an attribute function 'Pos that returns the position or ordinal value—PrimaryColors'Pos(Red) returns 0.

In Ada, each type category has an associated set of attributes for all types belonging to that category. These attributes are provided by the compiler and do not have to be defined by the programmer. In our upcoming discussion of arrays, we will see some of the attributes defined for array types.

III. Structured Data Types

Next, we consider structured data types, which are data types whose variables contain a collection of values. Arrays and records are the two fundamental kinds of structured data types. Both appeared early in the history of programming languages—Fortran being the first major high-level language to support arrays and COBOL being the first to support records.

A. One-Dimensional Arrays

Arrays are collections of elements of the same type. Although they may seem more restrictive than records that allow collections of different types, their generality lies in their selection mechanism. Any collection must have some mechanism for selecting individual elements within that collection. Subscripting or indexing is the technique used by arrays. What makes this selection mechanism so powerful is that array subscripts can be expressions that contain variables.

This flexibility does have an unfortunate consequence, though: the subscripts can be outside the defined bounds of the array. One important design consideration of any language that supports arrays is whether bounds checking on subscripts will be performed. Because subscripts can be expressions that contain variables, if this checking is done, it must be done at run time. Any such run-time checking adversely affects the execution speed of a program. Failing to make such checks, however, can lead to highly unreliable programs. C and C++, which emphasize speed, do not perform such checking. Ada and Java, which value reliability above all, do perform subscript-bounds checking.

Array Allocation

Like any variables, array variables can be allocated in static memory, on the compiler's run-time stack, or on the heap. With array variables, another consideration is whether the arrays' bounds are values that can be determined at compile time. With statically allocated arrays, the bounds must be determinable at compile time, and with arrays allocated on the heap, it makes little difference whether or not they are determinable at compile time.

It is with arrays allocated on the stack that languages differ. In C and C++, the size of stack-dynamic arrays must be able to be determined at compile time. Consequently, the following attempt to write a function in C++ that reverses an array by using a local copy fails to compile.

```
void copy(int array[], int length)
{
    int copy[length];

    for (int i = 0; i < length; i++)
        copy[i] = array[i];
    for (int i = 0; i < length; i++)
        array[i] = copy[length - 1 - i];
}
```

The error produced by many compilers is a message that indicates that the size of an array must be a constant, which leads many novices to modify the function heading as follows:

```
void copy(int array[], const int length)
```

That modification does not fix the error because by *constant*, the error means the size of an array must be able to be determined at compile time—not that it cannot have its value changed. Allowing such arrays is undesirable because it is difficult for a compiler to calculate the address of local variables relative to the beginning of the activation record—a process we described earlier in this module when we introduced stack-dynamic allocation. Such calculations can be made only if the size of the variables is known.

Ada does allow stack-dynamic arrays to contain variables. Our above failed attempt to write such a function in C++ can be accomplished in Ada.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure Main is
    type IntegerArray is array (Integer range <>) of Integer;

    procedure Copy (Original: in out IntegerArray) is
        Copy: IntegerArray(Original'Range);
    begin
        for I in Original'Range loop
            Copy(I) := Original(I);
        end loop;
        for I in Original'Range loop
            Original(I) := Copy(Original'Length - 1 - I);
        end loop;
    end Copy;

    Original: IntegerArray(1..5);
begin
    Original := (1, 2, 3, 4, 5);
    Copy(Original);
end Main;
```

Array Aggregates

Next, we consider the extent to which various languages support array constants or array aggregates. In the C family of languages, array aggregates can be used for initialization only when an array is declared. In C or C++, we can write:

```
int array[] = {1, 2, 3, 4, 5};
```

But if we attempt to use the aggregate in an assignment, a compilation error will result, as seen below:

```
int array[5];
array = {1, 2, 3, 4, 5}; // does not compile
```

With strings that are character arrays, such assignments are possible if the array aggregates are written as string literals.

In our previous Ada example, we saw that Ada was more flexible. It allows array aggregates to be used in assignments. It provides another variation on the array aggregate syntax, called *positional association*. The array aggregate assignment used in the previous program could be written using positional association as follows:

```
Original := (5 => 5, 1 => 1, 4 => 4, 3 => 3, 2 => 2);
```

When positional association is used, order is no longer important. Ranges, choices, and the reserved word others adds to this flexibility, as illustrated in the following example:

```
Mixture: IntegerArray(1..50) := (1..10 => 1, 20 | 30 | 40 => 2,
    others => 0);
```

In the example above, the values in subscripts 1 through 10 are initialized to 1, the values in subscripts 20, 30, and 40 are set to 2, and all the other subscripts are initialized to 0.

Array Assignment and Operations

Another area in which there is considerable variation among languages is on the number of operations that are predefined on whole arrays.

Let's first examine assignment. In C and C++, attempts to assign one whole array to another result in a compilation error as the following code illustrates:

```
int array1[] = {1, 2}, array2[] = {3, 4};
array1 = array2;
```

The assignment is not permitted because array names are treated as constant pointers, which cannot be modified. The situation in Java is different, however. Array assignments are permitted, but like any object assignment, they are shallow copies.

Whole array assignments are permitted in Ada. The following lines illustrate such an assignment.

```
Array1: IntegerArray(1..2) := (1, 2);
Array2: IntegerArray(2..3) := (3, 4);
Array1 := Array2;
```

The rule on assignment requires that the types and lengths match, as is true in the above example. Unlike Java, the copy is deep—the actual values are copied.

None of the languages in the C family have any other predefined operations on whole arrays. Ada does, however. It permits concatenation, slicing and comparison for equality, and inequality for one-dimensional arrays.

B. Multidimensional Arrays

One fundamental language design issue regarding multidimensional arrays that language designers must consider is whether to implement multidimensional arrays as rectangular arrays or arrays of arrays or both.

Rectangular Arrays

When a language implements multidimensional arrays as rectangular arrays, a block of memory is allocated whose size is the product of the length of each dimension multiplied by the size of each array element.

When array references are made, it is necessary for the compiler to generate code that will compute the position of each array element. That code must compute the location of the array element using a *storage allocation function* that maps two dimensions in one. Two kinds of mappings—*row-major* and *column-major*—are typically used. Figure 2.2 illustrates how the elements of a two-dimensional array with two rows and three columns would be enumerated according to both orders.

Figure 2.2
Example of a 2 × 3 Array
Enumerated in Row-Major and Column-Major Orders

Example Of A 2 × 3 Array Enumerated in Row and Column Major

Row Major Order

0	1	2
3	4	5

Column Major Order

0	2	4
1	3	5

The storage allocation function for row-major order is shown below, assuming array subscripts always begin at 0, as they do in the C family of languages:

$\Lambda(i, j) = (i \times \#\text{columns} + j) \ \text{size}$

Making the lower bound 0 avoids having to subtract it from the corresponding subscript. Notice that the number of columns is involved in the calculation. When we discuss array parameters in module 3, we will explain how that affects multidimensional array parameters in C and C++.

Most languages that implement rectangular arrays use row-major order. Fortran is the one exception. The formula for column major, shown below, is similar to that for row major, but instead involves the number of rows.

$\Lambda(i, j) = (j \times \#\text{rows} + i) \times \text{size}$

Arrays of Arrays

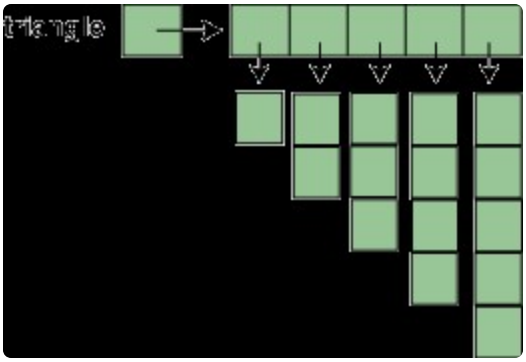
In some languages, such as Java, multidimensional arrays are exclusively arrays of arrays. In most languages that support arrays of arrays, those arrays can be ragged. In languages that do not explicitly support ragged arrays but do contain pointers, ragged arrays can be created using an array of pointers.

To illustrate this idea, consider the following two-dimensional Java array that creates a triangular array. The first row has one column, the second has two, and so on.

```
int [][] triangle = new int[5][];
for (int row = 0; row < triangle.length; row++)
    triangle[row] = new int[row + 1];
```

Figure 2.3 illustrates the ragged array that is created by the above code.

Figure 2.3
A Ragged Array



In the previous example, as with any two-dimensional array, triangle.length refers to the number of rows. triangle[0].length refers to the number of columns in the first row.

Some languages support both rectangular arrays and arrays of arrays. Ada and C# are such languages. In Ada, the arrays of arrays cannot be ragged.

C. Strings

We have elected to treat strings separately from arrays because in some languages, strings are actually arrays, and in other languages, they are objects. In C, strings are arrays of characters with the special provision that they are terminated with the null character. String operations in C are accomplished with calls to predefined functions. Many of these operations, such as strcpy, which is used to deep copy strings, are prone to cause memory to be overwritten. Such overwriting occurs when the destination string is smaller than the source string. Because no size information is kept at run time for any C arrays—including strings—it is impossible for such functions to check whether sufficient space has been allocated.

C++, being a superset of C, includes the C-style strings but also has a predefined class named string, which is far safer to use. Because C++ allows operator overloading, the assignment operator, all relational operators, and the + operator are all overloaded in the string class. The + operator is used to define concatenation.

Furthermore, C++ treats [] as an operator, and the string class overloads that operator, so that strings can be subscripted like arrays. It is even possible to put a subscripted string variable on the left-hand side of an assignment. In module 3, we will explain what feature of C++ enables this unusual ability, in effect, to place a function call on the left side of an assignment.

As elegant as the C++ implementation of the string class is, there is one shortcoming—C++ must also support C strings. Consequently, the type of string literals in C++ is char*, not string. This fact leads to some unfortunate misinterpretations. If two string literals are compared with any of the relational operators, this comparison is treated as pointer comparison, not a call to the overloaded relational operator defined in the string class. Shown below is an even more obscure example of the kind of misinterpretation that can result:

```
string s = "a" + 'b';
```

This is another example of the worst possible interaction of various language features that produce a program that appears to do something reasonable but is misinterpreted in a highly unreasonable way. Most programmers would expect s to contain the string "ab"—the result of having concatenated the string literal "a" with the character literal 'b'. Instead, what results is garbage.

One must understand a number of features of C++ to explain how this statement is interpreted. First, as we mentioned, string literals in C++ have type char*—not string. Next, characters are treated as integers, so the character literal is interpreted as the integer corresponding to its ASCII value. Next, the + operator is interpreted as pointer arithmetic, not concatenation—a topic we will discuss shortly. That pointer arithmetic produces an address viewed as type char*. To understand why the types of opposite sides of the assignment are not incompatible, one must be aware of yet another feature of C++—the fact that certain constructors may be implicitly called to perform type coercion. The fact that the string class has a constructor with the following prototype:

```
string(char *);
```

explains why this assignment is not rejected as a static semantic error.

Although all the languages in the C family often implement features in the same way, with strings, that is not the case, so Java's approach differs from that of both C and C++. First, in Java, the type of string literals is String. Consequently, the Java code equivalent to our earlier C+ code shown below behaves as expected.

```
String s = "a" + 'b';
```

Unfortunately, Java does not support operator overloading, so assigning Java strings results in shallow copies. Fortunately, Java strings are immutable, so shallow copies do not cause problems. Problems do arise with the inability to overload the relational operators. The compareTo method must instead be used in much the same way strcmp is used with C strings. Furthermore, when comparing strings, if the == operator is mistakenly used, a shallow comparison results, which is seldom what is desired.

In Ada, the predefined type String is an unconstrained array type of characters. Because strings are arrays, all array operations are defined for strings, which include subscripting, slicing and comparing for equality, and inequality. Because Ada permits operator overloading, the remaining four relational operators can be overloaded for strings.

D. Records

Records are the other fundamental structured data type. As we mentioned earlier, they are more general than arrays in the sense that they can collect data elements of different types. But because the components are named, not numbered, the selection mechanism is less powerful. Most languages use the dot notation for selecting a record component. Because names are essentially constants, every record component reference can be resolved at compile time, unlike subscripted arrays.

As we noted in our introduction to structured data types, COBOL was the first language to include records. Except for early versions of Fortran and more recently Java, most other languages have them. In both Pascal and Ada, these structured data types are called *records*, but in C, C++, and C#, they are referred to as *structures*, using the reserved word struct to define them.

It is worthwhile to explore the evolution of structures in the C family. In C, which is not object-oriented, structures are limited to data, making them true records. In C++, which is object-oriented, structures and classes are nearly identical—both allowing data members and member functions. The only difference between structures and classes is whether the members are public or private by default.

We have been emphasizing how important the use of reserved words is in highlighting important language features. C++'s use of two different reserved words—struct and class, is a poor design because the meanings of these two reserved words are so similar. Leaving structs as they were in C would have been a better language-design choice. In C#, by contrast, structures and classes are fundamentally different.

The designers of Java made the decision to drop structures altogether. Because a class can have only data and no methods, it can be reduced to a simple record structure. Of course, in well-designed object-oriented programs, having a class with only data but no methods is seldom a good design unless, perhaps, the class is nested inside another class.

E. Unions and Variant Records

C and C++ have unions, which syntactically look very similar to structures. What they represent is different, however. A whole structure variable contains *all* the components, whereas a whole union variable contains *only one* of the components. The union implementation in these languages is referred to as a *free union*. It is one area where no type checking is performed and the type system can be compromised. The following code segment illustrates how the types can be compromised.

```
union
   {
       int i;
       double d;
   } u;

   u.d = 1.0;
   cout << u.i;
```

The code above outputs garbage, yet no error is produced.

Another kind of union is a *variant record*, also called a *discriminated union*. In a variant record, a tag field or discriminant contains a value indicating which variant is applicable. Pascal has variant records, but types can still be compromised because the tag field, as it is called in Pascal, can be changed without changing the remaining fields, producing a problem similar to the one above in C++.

Ada solves this problem. Consider the following variant-record definition in Ada. Move your mouse over the highlighted parts of the code segment to see the explanations of those parts.

```
procedure Main is
    type MaritalStatus is (Single, Married);
    type Person(Status: MaritalStatus := Single) is
    record
        Name: String(1..4);
        Age: Integer range 0..100;
        case Status is
            when Married =>
                Spouse: String(1..4);
            when Single =>
                null;
        end case;
     end record;

    John: Person := (Status => Single, Name => "John", Age => 2
begin
    -- John marries Jane
    John := (Status => Married, Name => "John", Age => 25,
        Spouse => "Jane");
end Main;
```

Ada allows the discriminant to be changed only for unconstrained variant records. Furthermore, when the discriminant is changed, all the record components must be changed as is done in the above example.

Neither Java nor C# has either unions or variant records because inheritance is the approach that would be used in those languages to implement such data relationships. In the above example, SinglePerson and MarriedPerson would be classes derived from a base class Person.

IV. Pointers and References

Any language that has explicit heap-dynamic memory allocation must have either pointers or references—sometimes both.

A. Declarations

We begin our discussion with an illustration of the syntax for declaring pointers and references.

We chose C++ to illustrate declarations because it is unusual among languages in that it has both pointers and references. Both must be explicitly declared as illustrated below.

Move your mouse over the highlighted parts of the code segment to see the explanations of those parts.

```
int x = 5;
int &rx = x;
int *px = &x;
```

C, by contrast, supports only pointers, as do both Pascal and Ada. Ada calls them *access types* and uses the reserved word access in their type declarations. Java has only references, but unlike C++, they are implicit references and apply only to objects.

B. Operations

Next, let's consider the operations that typically accompany pointers. References have no associated operators, regardless of whether they are explicit as in C++ or implicit as in Java.

Dereference and Address Operators

The one operation that every language that has pointers needs is a dereference operator. In Pascal, the symbol ^ is used for this operator. In C and C++, the symbol * is used. Ada uses a more unusual syntax—.all—which can be omitted when

dereferencing pointers to arrays or records.
Less common is the inverse of the dereference operator—the address operator—that creates a pointer from an ordinary variable by providing its address. Such operators are present only in languages such as C or C++ that allow pointers to point to stack-dynamic variables. In those languages, the symbol for that operator is &. We saw that operator used in the pointer initialization in our previous example.

We have commented on the undesirable practice of using the same reserved word to mean different things in different contexts. In C++, the symbols * and & are used in a similar fashion. They are overloaded in a way that their meaning as unary operators is unrelated to their meaning as binary operators. As unary operators * and & mean *dereference* and *determine address*, respectively. As binary operators, they represent *multiplication* and *bitwise and*, respectively.

Compounding the overuse is that these symbols are also used in type specifiers to specify *pointer* and *reference* types, respectively. In that latter context, they are not ordinary operators, but type operators. One might argue that their uses as type modifiers and unary operators are related, but they are certainly not identical. Needless to say, such overuse of symbols can make it challenging to master the syntax of pointers and references in C++.

Pointer Arithmetic

Pointer arithmetic is an unusual feature to find in languages that have pointers. C and C++ are the only popular high-level languages that contain this feature. Pointer arithmetic is most useful with array parameters or dynamically allocated arrays. Because the array and pointer notation is interchangeable in C and C++, one can avoid pointer arithmetic entirely. A common practice in C was to use pointer arithmetic with array parameters to improve the efficiency of a function by eliminating the need for the multiplication that is generated whenever a subscripted array element is accessed.

To understand what multiplication is eliminated, examine the following translation that the compiler must make whenever it encounters a subscripted one-dimensional array element:

`a[i]` is translated by the compiler to `a + i × sizeof`(the array elements)

Consider the following three equivalent implementations of a simple function to sum all the elements of an integer array.

First, consider the version in which the parameter is declared and referenced as an array. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_1(int array[], int length)
{
    int sum = 0;
    for (int i = 0; i < length; i++)
        sum += array[i];
    return sum;
}
```

Next, let's look at the same function rewritten declaring the parameter as a pointer. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_2(int* array, int length)
{
    int sum = 0;
    while (length-- > 0)
        sum += *array++;
    return sum;
}
```

We provide the third version to illustrate subtraction of one pointer from another. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_3(int* array, int length)
{
    int sum = 0;
    int* start = array;
    while (array - start < length)
      sum += *array++;
    return sum;
}
```

It is worth noting that a good optimizing compiler can perform these programmer-encoded optimizations.

The three pointer-arithmetic operations are summarized in table 2.1.

Table 2.1
The Three Kinds of Pointer-Arithmetic Operations

Operation	Computation
pointer + integer	pointer + integer × size
pointer - integer	pointer - integer × size
$pointer_1$ - $pointer_2$	($pointer_1$ - $pointer_2$) / size

Because the behavior of pointer arithmetic depends on the type of the object being pointed to, pointers to different types are not type compatible in C and C++.

The fact that subtraction of pointers is defined but addition is not defined may seem strange at first. There are other instances, however, in which only the three arithmetic operations in the table above are meaningful, for example the arithmetic of time and addresses. The correspondences are illustrated in table 2.2.

Table 2.2
Correspondence of Pointer, Time, and Address Arithmetic

Pointer Arithmetic	Time Arithmetic	Address Arithmetic
pointer	time	relative address
integer	duration	absolute address

In the case of time, subtracting two times produces a duration, not a time. Adding two times is meaningless. If you have studied assembly-language programming, you understand the difference between relative and absolute addresses. The arithmetic of absolute and relative addresses behaves in the same way as pointer arithmetic. Subtracting two relative addresses produces an absolute address, but adding two relative addresses is meaningless.

C. Dangling References and Garbage

Now let's consider some problems that can arise in any language that has pointers. As we noted earlier, pointers in C and C++ can point to variables that are allocated on the stack. This capability can lead to problems like the one we illustrated earlier in our discussion of lifetime. In that case, the programmer returned a pointer to a local variable, which created a dangling reference or dangling pointer.

Dangling pointers can be created in other ways in C and C++ as well. Consider the following code segment:

```
int *x;
*x = 5;
```

Although the above code is seriously in error, no error is generated at compile time. At run time, if an error occurs, it is likely to be generated by the operating system—most likely a memory-protection error—although there is certainly no guarantee that any error will result. The problem in this case is that the pointer has never been initialized, yet it is being dereferenced.

There is yet a third way to create a dangling pointer in C and C++, which is illustrated by the following C++ code segment:

```
int *px = new int, *py = px;
delete px;
```

At this point, the pointer py is left dangling.

A related issue concerning pointers and any memory allocated on the heap is the possibility of creating *garbag*e. Another term that is less often used but is perhaps more descriptive is *dangling object*. Figure 2.4 illustrates how easily garbage can be created.

Figure 2.4
Creating Garbage

0:00 / 0:30

In a language such as C or C++ that does not have automatic garbage collection, it is the responsibility of the programmer to ensure that garbage is not created. To prevent the formation of garbage, explicit deallocation is needed. The C++ code segment in figure 2.4 is shown below. The command to explicitly deallocate the memory is highlighted.

```
int *p = new int;
int *q = new int;
*P = 5;
*q = 6;
delete p;
p = q;
```

In languages that do implement automatic garbage collection, such as Java and C#, the user is freed from this responsibility, and garbage will never accumulate endlessly.

D. Pass by Reference

One benefit of allowing pointers to point to the compiler's run-time stack is that they can be used to pass parameters by reference as illustrated in the following C program.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <stdio.h>

void swap(int *value1, int *value2)
{
    int temp;

    temp = *value1;
    *value1 = *value2;
    *value2 = temp;
}

void main()
{
    int value1 = 10,value2 = 20;

    swap(&value1, &value2);
    printf("value1 = %d, value2 = %d\n", value1, value2);
}
```

Including the syntax of references in C++ does simplify significantly the syntax of pass by reference when references are used in place of pointers. Consider the previous C program, rewritten in C++, using reference parameters in place of pointers.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

void swap(int &value1, int &value2)
{
    int temp;

    temp = value1;
    value1 = value2;
    value2 = temp;
}

int main()
{
    int value1 = 10,value2 = 20;

    swap(value1,value2);
    cout << "value1 = " << value1 << " value2 = " << value2 <<
    return 0;
}
```

Another interesting consequence of including references in the language is the ability to write functions that return references. In module 3, when we discuss subprograms, we will give some examples of functions that return references and discuss how useful this feature can be.

Module 3: Imperative Languages—Control Flow

Topics

1. Expressions
2. Statements
3. Subprograms

I. Expressions

We begin our discussion of control flow in imperative languages with the lowest level of control flow—the expression level.

A. Expression Evaluation

Fortran first introduced nested expressions in high-level languages. When an infix expression contains more than one operator, one important question is how that expression should be parsed and, therefore, how it should be evaluated. In module 1, we emphasized the importance of designing a grammar that would produce the correct parse tree. Now we consider the rules of programming languages that specify the kind of parse tree that is required. Every language allows expressions to be fully parenthesized, including a pair of parentheses for each operator. When an expression is fully parenthesized, the parentheses parse the expression, thereby specifying the parse tree and how the expression is to be evaluated.
Most languages, however, allow parentheses to be omitted. When they are omitted, other rules are needed to specify how to correctly parse an expression. These rules are the precedence and associativity rules. Many books characterize these rules as defining the order of operations. Unfortunately, that characterization causes many

students to misunderstand the purpose of these rules. A common misunderstanding that arises by thinking that precedence defines the order of operations is illustrated by the following expression:

```
(x + y) - (a * b)
```

Some students conclude that the multiplication operator is evaluated before the addition operator because the former has higher precedence. That conclusion is incorrect because this expression is sufficiently parenthesized so that the precedence rules are irrelevant in this case.

The best way to characterize the purpose of the precedence and associativity rules in programming languages is to say their role is to determine the way the expression should be parsed when parentheses do not fully specify the correct parse tree.

Precedence Rules

Once you realize that nested expressions in programming languages are intended to mirror mathematical formulas, it will not surprise you that these expressions borrowed the precedence rules that are inherent in mathematical formulas. For example, in mathematics, when we write a formula such as $xy + ab$, we know it is interpreted as $(xy) + (ab)$ because multiplication has higher precedence than addition. What has *not* been copied from mathematics is the implied multiplication of mathematical formulas, in part, because single-letter variable naming is generally discouraged.

Languages vary greatly with regard to the number of levels of precedence, with a language such as Pascal having as few as 4 levels and languages such as C and C++ having as many as 15. Just as the number of precedence levels varies, so does the relative precedence of operators. Among the arithmetic, relational, and logical operators, however, there is a natural precedence order that most languages observe, ranking the arithmetic operators as the highest among the three groups and logical operators the lowest. Pascal is an exception to this general order. Consequently, in Pascal, writing the following expression without parentheses is an error:

```
(x > y) AND (y > z)
```

The parentheses are required because, in Pascal, the logical operators have higher precedence than the relational operators.

Associativity Rules

The associativity rules are needed in the absence of parentheses when there are two or more operators of equal precedence, as illustrated by the following expression containing integer constants:

```
4 - 1 - 1
```

Because subtraction is not associative, the result will be different depending on how the subexpressions are grouped. Grouping from left to right produces a value of 2, whereas a right-to-left association gives 4.

The four fundamental arithmetic operators associate left to right in almost all languages that define associativity rules. The one arithmetic operator that varies, in languages that have it, is exponentiation. In Visual Basic it is left associative, in Fortran, it is right associative, and in Ada it is nonassociative. By *nonassociative*, we mean that Ada requires parentheses in expressions that have adjacent exponentiation operators.

Subexpression Evaluation Order

One additional issue that programming languages must define is whether the left subexpression or the right is evaluated first, or whether this evaluation order is left unspecified. Upon initial reflection, it may not seem that this order matters, which is true in most cases. Consequently, most languages leave this order unspecified for most operators, to allow the optimizer phase of the compiler the freedom to make the optimal choice. As we shall discuss shortly, short-circuit logical operators, when present, are an exception. Java is also an exception. In Java, the left subexpression is always evaluated before the right.
Expressions in which this order matters are expressions that contain side effects. Side effects are caused in expressions either by assignment expressions or function calls that modify global variables or parameters or perform input/output.

Figure 3.1 provides an example of a C or C++ expression containing an assignment operator whose value depends upon the relative order of evaluation of the two subexpressions. By selecting one of two buttons that are provided, you can specify which order is used. After you step through the explanations when the Left-Before-Right button is selected, select the Right-Before-Left button to step through those explanations.

Figure 3.1
Evaluating Expressions with Side Effects

0:00 / 1:14

What you should have noticed is that the result of the expression will be different depending upon which order was followed. The results are different because this expression contains a variable whose value is changed by an assignment expression and the same variable is used again in another part of the expression.

Using a function call to a function that either modifies a parameter or a global variable can create a similar kind of expression. The following C++ program contains a function that modifies its parameter, which is also contained in another part of the expression.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int doubleParameter (int* parameter);

int main()
{
    int x = 10, y = 10, left, right;
   left = doubleParameter(&x) + (x / 2);
    right = (y / 2) + doubleParameter(&y);
    cout << left << " " << right << endl;
    return 0;
}

int doubleParameter (int* parameter)
{
    return (*parameter) *= 2;
}
```

When run with Microsoft Visual C++, this program produces 30 25 as its output. Other compilers could behave differently. When side effects are present, as they are in this case, the result is that addition is no longer a commutative operation, as this program illustrates.

Expressions without side effects are called *pure expressions*. It is difficult in imperative languages to guarantee that expressions will always be pure. Ada comes closest to achieving this goal. It has no assignment expressions. Function parameters must be in parameters, which prevents the kind of program written above in C++. The first Ada reference manual, written in 1979, stipulated that

functions were prohibited from modifying global variables to help ensure the purity of function calls. This stipulation was dropped by the time that version of Ada was finalized in 1983.
Pure expressions are extremely important in functional languages, so we will return to this idea in module 5.

B. Operators

Next, let's consider the various kinds of operators that make up expressions.

Arithmetic Operators

All imperative languages have the standard four arithmetic operations and use the symbols:

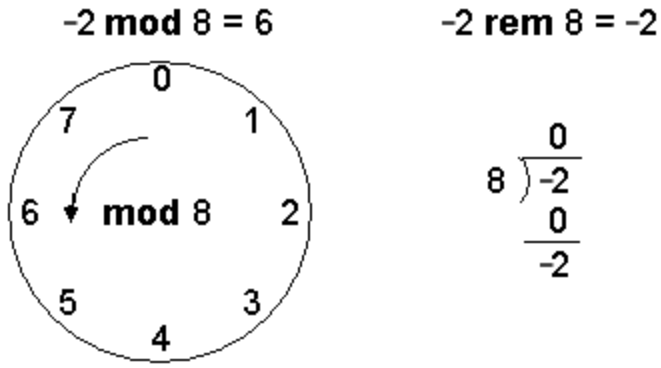
+
–
*
/

There is some disparity among languages, however, with integer division, the remainder operation, and exponentiation.

Pascal is unusual among languages, having an operator, DIV, for integer division. Using the / operator with two integer operands produces a real result, whereas the DIV operator produces integer division. In most other languages, one must type cast one of the integer operands to a floating-point type to avoid integer division, which otherwise would be the default.

The remainder operator is another varying feature. The C family uses the % symbol. Pascal uses the reserved word MOD. In most languages, Java being an exception, this operation is valid only with integer operands. Ada is unique in that it provides two operators, rem for remainder and mod for modulus, which produce different results with negative values. Figure 3.2 contains an example that illustrates this distinction.

Figure 3.2
Comparing the mod and rem Operators of Ada



We have already discussed some of the differences among languages with regard to the exponentiation operator when we described operator associativity. What we did not mention is that not all languages have an exponentiation operator. Fortran and Ada do have one and use the symbol **. The C family of languages, by contrast, has no operator defined for exponentiation, and relies instead on library functions.

Relational Operators

All languages have the standard six relational operators. What varies somewhat are the symbols used for equality and inequality.

- In Pascal and Ada, = is used for equality because = is not the symbol for assignment.
- In the C family, == is used because = is the symbol for assignment. (In module 1, we discussed the problem associated with mistakenly using = when == is meant in the C languages.)

The symbol for inequality varies more than the symbol for equality:

- In Pascal, <> is used.
- In Ada, /= is used.
- In the C family, != is used.

We summarize these facts in table 3.1 below.

Table 3.1
Equality and Inequality Symbols

	Equality	Inequality
Pascal	=	<>
Ada	=	/=
C family	==	!=

A problem is associated with relational operators in languages such as C and C++ that lack a true Boolean type. In such languages, expressions like:

```
1 <= x <= 10
```

are misinterpreted, as we discussed in module 2.

Another misinterpreted expression containing a relational operator is the following expression in C++:

```
"abc" < "def"
```

Rather than comparing the string values, the addresses of the two literals are compared instead, which is clearly not what is intended. The cause of this misinterpretation is the fact that strings in C++ are C-style strings whose type is char*. In module 2, we saw another example of how this fact can cause misinterpretation of the + operator.

Finally, a similar misinterpretation can occur in Java strings when the == operator is used rather than the equals method. The cause for the misinterpretation in Java is different, however. It is caused by the fact that Java does not overload the == operator for strings to perform deep comparison—comparison of values rather than references. Java disallows user-defined operator overloading as well—a topic we will discuss shortly.

Logical Operators

In addition to some variation in the symbols used, the other aspect of logical operators that tends to vary among languages is whether the operators short-circuit or not. Short-circuiting of logical operators is based on the following two logical identities that are true regardless of the value of *x*.

false AND $x \equiv$ false
true OR $x \equiv$ true

Logical expressions containing short-circuiting operators are unique in that the left subexpression must always be evaluated before the right, regardless of whether this rule applies to other expressions in a language. The reason is that when short-circuiting does occur, the right operand is never evaluated.

Ada has both short-circuit and non-short-circuit logical operators. The operators "and then" and "or else" short-circuit, whereas and and or do not short-circuit. Java has both kinds of logical operators also—&& and || short-circuit—& and | do not short-circuit. For clarity, we have summarized these in the table below.

Table 3.2
Short-Circuit and Non-Short-Circuit Operators
in Ada and Java

	Short-Circuit Operators	**Non-Short-Circuit Operators**
Ada	and then or else	and or
Java	&& ||	& |

In C and C++, the bitwise operators cannot be used reliably as non-short-circuiting operators because those languages treat 0 as false and nonzero values as true. Consequently, the following attempt will fail:

```
10 & 01 = 00
```

Two values that are regarded as true ANDED together should produce a true result but clearly do not.

Conditional Expression Operator

The languages in the C family have a conditional expression operator, but Pascal and Ada do not. It is an unusual operator in two ways. First, it is a ternary operator, meaning it has three operands. Second, it involves two symbols, the ? and the : symbols. The only case in C and C++ where an if statement cannot be used in place of this operator is with macros, as illustrated by the following example.

```
#define MAX(x,y) x > y ? x : y
```

Because this operator is rather cryptic, one might reasonably have argued for dropping it from Java, which does not permit macros, but the Java designers clearly believed otherwise.

Lest anyone doubt that overuse of this operator can lead to hard-to-read programs, consider the following use of nested conditional expression operators to determine the largest of three values:

```
x > y ? x > z ? x : z : y > z ? y : z
```

The lack of parentheses adds to the difficulty in interpreting this expression and requires that one know that it is a right-associative operator.

Bitwise Operators

Including bitwise operators in a language is a low-cost addition because every machine has hardware instructions to perform such operations. C, which was intended as a language to replace assembly language, understandably included them. All the languages in the C family kept them. There are six bitwise operators in C, as shown in the following table:

Table 3.3
The Bitwise Operators of C

Bit-Wise Operator	Meaning
&	and
|	or
^	exclusive or

<<	left shift
>>	right shift
~	one's complement

Because Java removed unsigned types, it added a seventh operator:

```
>>>
```

In Java, the >> operator performs an arithmetic right shift, which is what occurs in C when signed types are used. The >>> operator performs a logical right shift, which is what occurs in C with unsigned data.

The Comma Operator

Although a comma is a punctuation symbol in most programming languages, in C and C++, it is an operator when it appears in certain contexts. It is most commonly used in for loops, in which two subscripts are needed. For example:

```
for (int i = 0, j = n - 1; i < n; i++, j--)
    ...
```

It is most noteworthy, however, because of the potential problems it can create. Consider the following assignment:

```
x = a * b,
     + c;
```

Beginners occasionally misunderstand the role of a comma to be a symbol required when a statement is continued on the next line. Such a misunderstanding might lead one to write an assignment like the one above. Unfortunately, the above statement is not a compilation error; it is a logic error because the value of the expression is only a * b. The comma is viewed as the comma operator and the + as a unary plus.

The Java developers wisely chose to exclude this operator.

Assignment Operators

In languages that regard the assignment symbol as an operator, which includes all languages in the C family, assignments are expressions. By contrast, in Pascal and Ada, assignments are statements. When assignments are expressions, they can be nested wherever an expression is permitted that enables statements such as the following commonly used C loop:

```
while ((ch = getchar()) != '\n')
    ...
```

By nesting the assignment to the call to the function that reads in the next character, one is able to avoid duplicating that call—first in a priming read before the loop and subsequently at the end of the loop body.

Another benefit of assignments being expressions is that assignments can be chained together, as illustrated by the following assignment:

```
x = y = 1;
```

One must also realize that the assignment operator is right associative to understand the way the above statement is interpreted.

Making assignments expressions does have two drawbacks:

1. It contributes to the misinterpretation of statements in which the assignment operator is mistakenly used where the equality operator was intended. We discussed that issue in module 1.

2. It introduces side effects in expressions, an issue we discussed earlier in this module, that can lead to different expression values depending upon the order in which the subexpressions are evaluated.

Although Pascal and Ada have only a single assignment symbol, :=, languages in the C family have a whole collection of assignment operators. In addition to the simple assignment operator, any of the binary arithmetic or bitwise operators can be

combined with the assignment operator to produce operators like +=. Furthermore, these languages also have increment and decrement operators, which are also assignment operators that can be used either in prefix or postfix.
The numerous assignment operators are an example of feature multiplicity—a characteristic of language features that we introduced in module 1. Specifically, in the C family of languages, there are, for example, four different ways to increment a variable such as *x*. These four possibilities are:

```
x = x + 1;
x += 1;
x++;
++x;
```

Providing so many different ways to accomplish the same task can lead to programs that might be difficult to read. In this case, the three operators +, +=, and ++ have different degrees of applicability. Specifically, the + operator is most general and ++ the least general. So, if programmers adopt the style of using the least general operator that can be applied in a given case, the readability of programs can be enhanced. Choosing the least general operator does not, however, help you decide whether to use the increment operator in prefix or postfix when the meanings do not differ because neither is more applicable than the other, so some convention is needed to establish consistency.

Finally, we consider type checking of assignments. In module 2, we mentioned that in C and C++, unexpected results can occur when variables of a larger type are assigned to variables of a smaller type—assigning a variable of type long to one of type short, for example. Java requires explicit type casts when narrowing assignments are performed to call attention to the potential consequences of such assignments.

C. Operator Overloading

Most languages overload some operators—the arithmetic operators being the most commonly overloaded. Arithmetic operators are generally defined to perform integer operations when applied to integers and to perform floating-point

operations when applied to floating-point numbers. What is less universal is whether languages allow user-defined overloaded operators. Both C++ and Ada allow such overloading. Java does not.
Proponents argue that allowing user-defined overloading enables data types that appear to be built-in types to be added to a language. Detractors claim the misuse of this language feature can lead to programs that are hard to read.

II. Statements

Now we consider the next higher syntactic level of control flow—the statement level.

A. Fully Bracketed Statement Syntax

We begin with a general issue related to control-statement syntax, which is the presence or absence of a fully bracketed syntax—a syntax that provides both an opening and a corresponding closing reserved word for each statement. Many newer languages have such a syntax. Ada employs this syntax, as does Fortran, beginning with Fortran 77. The languages in the C family do not.

A fully bracketed syntax has three benefits. First, requiring an end if terminator on if statements eliminates the dangling else problem that we discussed in module 1 in conjunction with our discussion of ambiguous grammars. Consider parallel if statements in C++ and Ada to see why in C++ ambiguity is possible, but in Ada, it is not.

Move your mouse over the highlighted parts of the program segments to see the explanations of those parts.

Table 3.4
Comparison of C++ and Ada if Statements—
When Another if Is Nested in the if Part

C++ Version	Ada Version

`if (x != 0)`
`if (x > 0)`
`cout << "x is positive";`
`else`
`cout << "x is zero";`
	`if x /= 0 then`
	`if x > 0 then`
	`put("x is positive");`
	`end if;`
	`else`
	`put("x is zero")`
	`end if;`

The second benefit of a fully bracketed syntax is that because all statements are explicitly terminated, the inconsistency of when compound statements are required is eliminated. Consider parallel if statements in C++ and Ada containing a single statement. Notice that when a second statement is added, the C++ if statement must be modified to contain a compound statement. No such change is needed in Ada.

Table 3.5
Comparison of C++ and Ada if Statements—
When Two or More Statements Are Nested in an if Statement

C++ Version	Ada Version

`if (x < 0)`
`{`
`x = -x;`
`cout << x;`
`}`
	`if x < 0 then`
	`x = -x;`
	`put(x);`
	`end if;`

The third benefit to a fully bracketed syntax is that because each kind of statement has a unique closing reserved word, as Ada does, error recovery can be improved. Consider a parallel missing closing delimiter in C++ and a corresponding error in Ada.

Move your mouse over the highlighted parts of the program segments to see the explanations of those parts.

Table 3.6
Comparison of C++ and Ada Statements—
Error Recovery

C++ Version	Ada Version

`while (x != 0)`
`{`
`  cin >> x;`
`  if (x < 0)`
`  {`
`  x = -x;`
`  cout << x;`
`}`
	`while x /= 0 loop`
	`  get(x);`
	`  if x < 0 then`
	`  x = -x;`
	`  put(x);`
	`end loop;`

B. Selection Statements

Next, we consider the first of two fundamental categories of control statements—selection statements. Selection statements allow the control flow of a program to be broken into multiple paths. There are two fundamental kinds of selection statements. The first—the if statement—is the more general of the two. The second is more restrictive. It is called a case statement in some languages and a switch in others.

Let's begin with the if statement. We have already considered one fundamental variation in the syntax of this statement, which is whether it requires a closing reserved word. Another variation is whether the statement syntactically is restricted to at most two paths, or whether it allows the control flow to be separated into any number of paths.

The if statement of the C family of languages and the if statement in Pascal allow at most two paths. To achieve the effect of more than two paths, nested if statements are needed. By contrast, Ada's if statement allows any number of paths because it

allows any number of elsif clauses. Compare a code segment that determines a letter grade from a numeric grade using C++ with a comparable code segment in Ada.

Table 3.7

Comparison of C++ and Ada Selection Statements

C++ Version	Ada Version

`if (grade >= 90`
`  cout << "A";`
`else`
`  if (grade >= 80)`
`  cout << "B";`
`  else`
`  if (grade >= 70)`
`  cout << "C";`
`  else`
`  if (grade >= 60)`
`  cout << "D";`
`  else`
`  cout << "F";`
	`if grade >= 90 then`
	`  put("A");`
	`elsif grade >= 80 then`
	`  put("B");`
	`elsif grade >= 70 then`
	`  put("C");`
	`elsif grade >= 60 then`
	`  put("D");`
	`else`
	`  put("F");`
	`end if;`

Proper indentation generally reflects nesting, which is what we have done in both cases. Notice what occurs in the C++ code, however. The last cases become very deeply indented. In languages with two-way if statements, programmers may opt to indent deeply nested if statements the way we have shown them indented in Ada because that indentation better reflects the meaning of the statement.

Next, let's consider the other kind of selection statement. We have already observed that it is a less general statement than the if. Consequently, languages would not restrict a programmer's problem-solving ability by including only an if statement. Including both kinds of statements also introduces feature multiplicity, but because one statement is more restrictive than the other, with good programming style guidelines, such multiplicity need not compromise readability.

Regardless of whether we are considering the switch statement used by the C family of languages or the case statement of Pascal and Ada, the restrictions on these statements are much the same. The type of the expression that is used for selection must be an integral type, or in Ada's terminology, a *discrete type*. The expressions that define the cases must be constant expressions—meaning that their values must be able to be computed at compile time. Unlike the if statement, these statements do not use logical expressions for path selection.

Comparing the switch statement to Ada's case statement, we find some important differences. The need for break statements at the end of the cases inside switch statements is contrary to the way we expect true structured statements to behave. The switch statement is actually more similar to Fortran's computed goto statement than to the case statements of either Pascal or Ada. Because forgetting to place a break at the end of each case is a common programming error, C# requires each nonempty case to end with a statement that transfers control—usually a break or a return statement.

Another difference is in the ease with which multiple values can be specified for a particular case. With the switch statement, each value must be enumerated. In the Ada case statement, a rich syntax is provided that allows a collection of values to be specified in a very compact way.

C. Iterative Statements

The other major category of control statements is iterative statements. Iterative statements differ fundamentally from selection statements in that they produce backward control flow, which is impossible with any number of selection statements.

But like the selection statements, iterative statements can be divided into two similar groups. Table 3.8 shows those groups for both categories of statements in Ada.

Table 3.8
The Control Statements of Ada

	Discrete Values	Logical Condition
Selection	case	if
Iteration	for	while

In much the same way as the Ada case statement is restricted to selecting among a collection of values of a discrete type, the Ada for statement is restricted to iterating across a range of discrete values. Neither the case or for statements involve logical conditions, but both the if and while statements do. The for statement is guaranteed to never produce an infinite loop because the loop control variable cannot be modified within the loop. The same, of course, cannot be said of the while loop.

We have confined the preceding discussion to Ada because although the C family of languages has a for loop, it is not more restrictive than the while statement. In the C family, any for statement could be changed to a while and vice versa. This kind of feature multiplicity is less desirable because it becomes more difficult to clearly establish a good set of style rules concerning when to use either statement.

The syntax of the C family's for statement allows some clever, although perhaps obtuse uses. Consider the following C for statement, which performs a string copy that uses an assignment expression as its termination condition.

Move your mouse over the highlighted parts of the statement to see the explanations of those parts.

```
for (i = 0; s[i] = t[i]; i++)
    ;
```

Such statements illustrate the expressive power of the syntax, but also the difficulty of reading the statements that result.

Both C# and version 5.0 of Java have included variations of a for statement that are more restrictive than the while statement. (In C#, it is called a foreach statement.) These statements are useful for iterating across data structures for which an iterator is provided. In Java, this statement can be used, for example, to iterate across an array, as shown below:

```
int[] array = {1, 2, 3, 4, 5};
for (int value: array.values())
    System.out.println(value);
```

These iterators are read-only, so a similar construct cannot be used to assign values to every element of an array.

One other aspect of iterative statements that varies among languages is whether the language provides loops that allow the condition that governs whether the loop will continue or stop to be placed either at the beginning or the end of the loop. The languages in the C family do provide such a choice with the while statement containing the condition at the beginning and do-while statement containing the condition at the end. Loop statements that have the condition before the loop body may never execute, whereas those with the condition placed at the end are guaranteed to execute at least once. Pascal has both a while and a repeat-until statement, giving the programmer the choice of placing the condition at the beginning or the end. The Pascal while statement differs from the repeat-until, in that the condition for the while statement is a continuation condition, whereas the condition in the repeat-until is a termination condition. Ada does not provide any special syntax comparable to a do-while or repeat-until statement.

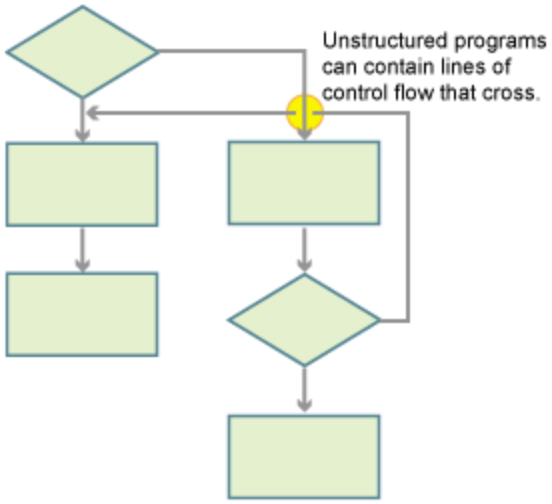
One final decision language designers must make in conjunction with iterative statements is whether multiple exits are permitted. Pascal, which was developed when structured programming was becoming widely accepted, took the approach of prohibiting multiple entries and exits. Although it is universally agreed that multiple entries are undesirable, the view on multiple exits has changed since that time. In the C family of languages, a break statement is used to exit from the middle of a loop. In Ada, the statement is called an exit statement. Java, C#, and Ada all allow a loop name to be specified so that an exit can be made from inside nested loops.

D. Structured Programming

Although it is taken for granted today, the introduction of a nested-statement syntax that enabled structured programs to be written was one of the most important milestones in programming language design. In assembly language and early versions of Fortran, programmers had to rely on *goto* statements to implement program control flow.

When programming in assembly and early Fortran, the design diagram used most often was the flowchart. With unstructured languages, the control flow of a program could be very convoluted, so a pictorial representation—the flowchart—helped clarify an otherwise potentially complicated algorithm. The flowchart of an unstructured program can be complicated because the lines of control flow can cross each other, as illustrated in figure 3.3

Figure 3.3
Flowchart of an Unstructured Program



The term *spaghetti code* was often used to describe programs with highly complicated flowcharts. This metaphor was used to compare the lines of control flow of such programs with the strands of spaghetti on a plate.

Once the benefit of structured programming was clearly understood and programmers refrained from using *goto* statements in their programs, the control flow of programs was greatly simplified, so flowcharts became unnecessary.

It is curious that in spite of the fact that it has long been recognized that using *goto*

statements is undesirable, Java is one of the few languages to have eliminated them entirely.

III. Subprograms

Finally, we consider the highest syntactic level of control flow—the subprogram level.

A. Parameter Passing and Return Values

One of the most important features of subprogram design is what mechanism a language provides for exchanging information between the calling subprogram and the called subprogram. We concentrate on two techniques—parameters and return values. Modifying global variables is another technique that can be used in languages that have global variables, but it is generally regarded as a poor design choice. There are really no design issues other than scope, which we have already discussed in module 2, that influence the ability to access global variables. Subprograms that are methods can access instance variables, but we defer specific discussion of methods to module 4, which focuses on object-orientation.

The way parameters are passed is a feature of programming languages that varies more than perhaps any other. One of the reasons for so much variation is that there are interacting and competing issues that any language designer must consider, so no solution is really a perfect one.

The two issues that interact are the semantic roles of parameters and implementation details. Semantically, a programmer must decide which of the following three roles the parameter will play:

1. to send data from the caller to the called subprogram, referred to as *in*
2. to send data from the called subprogram back to the caller, referred to as *out*
3. to send data in both directions, referred to as *in-out*

The implementation details involve how the parameters are passed:

- by value

- by result
- by reference
- by constant reference
- by value-result
- by name

What complicates matters is that these issues are not independent of one another. To illustrate this point, let's consider the relationship between three semantic roles of parameters and two of the implementations. Table 3.9 shows this relationship. The checkmarks mean that the role is supported by that implementation. An x indicates that it is not.

Table 3.9
Comparing Parameter Semantics and Passing Technique

	Value	Reference
in	√	√
in-out	x	√
out	x	√

Passing-by-value, which involves copying the actual parameter to the formal parameter, can be used only to send data into the subprogram, whereas passing by reference can be used for all three.

Because passing-by-reference appears to be more general, one might conclude it should be the only technique needed. Unfortunately, there is a competing issue that one also must consider when judging the generality of the parameter-passing technique shown in table 3.10.

Table 3.10
Comparing the Actual Parameter and the Passing Technique

Actual Parameter	Value	Reference
constant	√	x
variable	√	√
expression	√	x

This table indicates that when pass-by-value is used, the actual parameter can be a constant, variable, or expression, but with pass-by-reference, the actual parameter must only be a variable. When pass-by-reference is used, the rule for what is allowed as an actual parameter is the same rule as what can appear on the left side of an assignment.

If one considers this measure, call-by-value is more general. Call-by-value also has the benefit that it does not create aliasing as does call-by-reference. We will shortly see an example of the consequences of aliasing.

Because there is so much variation among languages, we have chosen seven languages to discuss—no two using exactly the same techniques.

Pascal

Pascal supports both call-by-value and call-by-reference, allowing the programmer to make the choice. What is perhaps most interesting about its implementation is the syntax for reference parameters, which is to require that they be labeled with the reserved word VAR. The choice of this syntax was, no doubt, to underscore the fact that when reference parameters were used, the corresponding actual parameters were required to be variables.

Fortran

Most early versions of Fortran used pass-by-reference. Unfortunately, Fortran has no restriction on the syntax of the actual parameters, as does Pascal. Consequently, it was possible to write Fortran programs that modified constants, which is a serious error. If a constant were passed as the actual parameter and the corresponding formal parameter's value were changed in the subprogram, the result could be that the constant's value would thereafter contain the new value.

Later versions of Fortran tend to use pass-by-value-result, which is sometimes also called *copy-in copy-out* because that name describes exactly how it is implemented. At the point of the call, the value of the actual parameter is copied to the formal parameter, and upon return, the formal is copied back to the actual. With this implementation, overwriting constants is easier to prevent.

We will return to comparing pass-by-reference and pass-by-value-result when we discuss Ada 83.

ALGOL

We have seldom discussed ALGOL in our previous topics concerning imperative languages, but it is noteworthy in this case because one of its parameter-passing techniques is unusual. It uses the familiar pass-by-value but also another technique called *pass-by-name*.

Pass-by-name textually substitutes the actual parameter for the formal parameter. In most cases, pass-by-name behaves exactly like pass-by-reference, but there are some peculiar situations where it behaves differently, illustrated by the following ALGOL procedure.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure OUTER;
    integer INDEX;
    integer array NUMBERS[1:2];
    procedure INNER(PARAMETER)
        integer PARAMETER;
        begin
        INDEX := 1;
        PARAMETER := 10;
        INDEX := 2;
        PARAMETER := 20
        end;
    begin
        INNER(NUMBERS[INDEX])
    end;
```

If you studied the highlighted statements in the above procedure, you saw that with call-by-name, formal parameters could refer to different variables at different times during the execution of the procedure. Implementing call-by-name is expensive

because this textual substitution must be done at run time, so code must be inserted at every point where a formal parameter is referenced that determines to what memory location that formal parameter currently refers.
Although no current widely used language uses call-by-name, C and C++ use compile-time textual substitution with macros, which we will discuss shortly.

C /C++

The parameter-passing technique used by C and C++ is call-by-value. In C, pointers are used to achieve passing by reference. Because array names are constant pointers, arrays are effectively passed by reference. As we illustrated with some examples in module 2, C++ added references, which provides a syntactically simpler technique for passing by reference.

C++ has two techniques for achieving in semantics. The first is to use call-by-value. The other is to use call-by-constant-reference. The latter technique avoids having to copy the actual to the formal parameter, which for large objects may be an efficiency benefit. Strictly speaking, using constant reference is slightly different from using call-by-value because modifying the value of the formal parameter is prohibited, which is not true when passing by value. The function prototype of a function that uses constant reference is shown below:

```
void f(const LargeObject& object);
```

The const modifier can also be used with pointer parameters. Unlike references, with pointer parameters, there are two things that we might wish to prohibit from changing:

1. the value of the pointer
2. the value the pointer points to

To illustrate these two possibilities, consider the following two C++ functions.

Move your mouse over the highlighted parts of the functions to see the explanations of those parts.

```
void f1(int* const p)
{
    p = new int;
    *p = 5;
}
void f2(const int* p)
{
    p = new int;
    *p = 5;
}
```

In much the same way that the references of C++ can be used to pass parameters by reference, they can also be used to return references to achieve what we might call *return-by-reference*. The following C++ program illustrates an integer wrapper class that contains two member functions that use return-by-reference.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

class Int
{
public:
    Int(int value)
    {
        this->value = value;
    }
    int& get()
    {
        return value;
    }
    friend ostream& operator<<(ostream& out, const Int& anInt)
    {
        return out << anInt.value;
    }
private:
    int value;
};

int main()
{
    Int i(5);
    i.get() = 6;
    cout << i << endl;
    return 0;
}
```

The ability to return references makes it possible for the C++ string class to overload the subscripting operator so that a subscripted string can be placed on the left side of an assignment to modify one character in a string.

Java

The parameter-passing rules in Java are really quite simple—everything is passed by value. It is often mistakenly written that Java objects are passed by reference, but such statements are really a misunderstanding of pass-by-reference. It is because all objects in Java are allocated on the heap and all object names contain references that such statements are made. When Java objects are passed as parameters, a copy of the reference is made, which is passing by value.

When pointer parameters are used in C++, we saw that const can be applied to either the pointer or the value being pointed to. In much the same way, the meaning of *in-out* semantics might be interpreted in two ways when reference parameters are used. It might mean either:

1. a change to a parameter's value will be preserved upon return to the caller

   or

2. a change to a parameter's reference will be preserved upon return to the caller

To illustrate the difference, consider the following Java program. Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
import java.awt.*;

class JavaParameters
{
    public static void main(String[] args)
    {
        Point point1 = new Point(100,100),
            point2 = new Point(200, 200);
        zeroPoints(point1, point2);
        System.out.println("point1 = " + point1 +
            " point2 = " + point2);
    }
    private static void zeroPoints(Point first, Point second)
    {
        first.x = first.y = 0;
        second = new Point(0, 0);
    }
}
```

Java object parameters can be viewed as *in-out* with regard to their values, but not with regard to their references.

Primitives are allocated on the stack, so when primitive parameters are used they cannot even be *in-out* with regard to their values. Consequently, it is impossible to write a swap method in Java that swaps two ints, for example.

C#

By contrast to Java, C# does allow pass-by-reference. It does so by adding the reserved word ref to the parameter declaration. So, if the Java program we just examined were rewritten in C#, and the parameter second were labeled as a ref parameter, both points would be zeroed out when the program was run.

C# also allows parameters to be labeled with out to designate *out* semantics. In that case, the parameter need not be initialized prior to the method call.

Ada

Although Ada predates C# by about two decades, the two have similar approaches to parameter passing. In Ada, procedure parameters are designated with one of three reserved words—in, in out, or out—that correspond to the three semantic roles of parameters. Omitting any of the three reserved words will cause a default to the in mode. In Ada 83, in parameters could not be written to, and out parameters could not be read from. In Ada 95, the latter restriction was dropped.

Ada's design philosophy was that the programmer should specify the semantics, and the compiler should decide on the appropriate implementation that supports it. We have already explained how some parameter-passing techniques such as call-by-value can accommodate only certain semantic roles, whereas others can accommodate any of the three. In particular, both call-by-reference and call-by-value-result can be used to implement *in-out* semantics.

When we were discussing the technique of constant reference in C++, we noted that it could be used in place of call-by-value because it might be more efficient given that it avoids making a copy. The efficiency tradeoffs between techniques involving copying compared to those that don't are actually more complicated.

In Ada 83, the compiler is free to choose whether to use call-by-reference or call-by-value-result when array parameters are designated as in out. The reasoning is that when the array is small but the number of array references is large, making a copy is preferable. Keep in mind that when call-by-reference is used, referring to array elements is less efficient because it requires an additional level of indirection. On the other hand, if the array is large and the number of references small, it is better to avoid copying the array. In theory, Ada compilers could have performed an efficiency analysis and chosen the more efficient implementation for that program. In practice, it is unlikely that any did.

Although allowing the compiler the freedom to choose the more efficient parameter-passing technique seemed like a good design decision, one unfortunate result was that certain kinds of programs—those that involved an undesirable kind of aliasing—would behave differently depending upon which technique was used.

Figure 3.4 provides an example of an Ada program containing this kind of aliasing. By selecting one of the two radio buttons provided, you can specify which

parameter-passing technique is used.

Figure 3.4
An Ada 83 Program that Behaves Differently,
Depending upon the Chosen Parameter-Passing Technique

0:00 / 1:54

Ada 83 declared such programs illegal, but compilers did not detect this illegality nor did they produce a compilation error. In recognition of the limited practical value of this design decision and the accompanying problems, in Ada 95, call-by-reference is used exclusively in such programs.

Unlike the C family of languages, Ada distinguishes between functions and procedures. Because function calls are expressions whereas procedure calls are statements, Ada places additional restrictions on function parameters, as we mentioned earlier. In Ada functions, the parameters can only be in parameters to minimize side effects.

B. Other Parameter Issues

There are several other characteristics of parameters aside from the way they are passed that we consider next.

Default Parameters

The first of these characteristics is whether a language allows default parameters. Both C++ and Ada allow such parameters, but Java does not. In Ada, default parameters are allowed only with in parameters. Default parameters can be very useful with constructors, as the following C++ example illustrates.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
class Time
{
public:
    Time(int hours, int minutes = 0, int seconds = 0)
    {
        this->hours = hours;
        this->minutes = minutes;
        this->seconds = seconds;
    }
private:
    int hours, minutes, seconds;
};

int main()
{
    Time noon(12);
    Time eightTwenty(8, 20);
    Time oneOneOne(1, 1, 1);
    return 0;
}
```

To write a class in Java equivalent to the one above would require including three separate constructors.

Default parameters can interact with another feature that we will discuss in greater detail shortly—subprogram overloading. As an illustration of this interaction, suppose an Ada program had two overloaded procedures with the following specifications:

```
procedure Some_Procedure(Formal_1: in Integer;
    Formal_2: in Boolean := False);

procedure Some_Procedure(Formal_1: in Integer;
    Formal_2: in Integer := 0);
```

Next, suppose the program included the following procedure call:

```
Some_Procedure(1);
```

Clearly, such calls are ambiguous because there is insufficient information to determine which one is intended. Compilers can detect such ambiguities, and Ada compilers will generate a compilation error in such cases. Such feature interactions are far less serious than the ones that result in a program being accepted by a compiler but misinterpreted.

Parameter Association

In most languages, the association between the actual and formal parameters is positional association—the first actual matches the first formal and so on. In Ada, the programmer can choose whether to use positional association or named association. To illustrate these two possibilities in Ada, consider the following procedure specification:

```
procedure Another_Procedure(Formal_1: in Integer;
    Formal_2: in out Boolean; Formal_3: out Float);
```

Using positional association, the procedure call would be written as follows:

```
Another_Procedure(Actual_1, Actual_2, Actual_3);
```

Using named association, it might be written as follows:

```
Another_Procedure(Formal_2 => Actual_2, Formal_1 => Actual_1,
    Formal_3 => Actual_3);
```

We have intentionally rearranged the parameter order to illustrate the fact that it is possible to use any order when named association is used, not because it is a recommended practice. The situation in which named association has the greatest benefit is when the actual parameter is a literal constant. In such cases, the presence of a formal parameter name enhances the readability of the call. An alternative in languages without named association would be to adopt the style of always using named rather than literal constants.

Array Parameters

Some issues are particular to parameters that are arrays. To understand the first of these issues, we begin by discussing a problem that exists with Pascal array parameters. In Pascal, array type definitions are required to include the specific array bounds. Consequently, an array of 10 integers can never be of the same type as an array of 5 integers. Furthermore, Pascal requires that the type of the actual parameter and the formal parameter match. The consequence of these two features is that in Pascal, one cannot write a subprogram that processes arrays of different lengths. This is a serious language-design problem because it requires unnecessary code duplication.

Ada's solution to this problem was to introduce unconstrained array types. We saw an example of an unconstrained array type in module 2 as a part of our discussion of arrays, but let's consider another example.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
with Text_IO; use Text_IO;
procedure Main is
type IntegerArray is array (Integer range <>) of Integer;

    function SumArray (FormalArray: in IntegerArray) return Int
        Sum: Integer := 0;
    begin
        for I in FormalArray'Range loop
            Sum := Sum + FormalArray(I);
        end loop;
        return Sum;
    end SumArray;

    ActualArray_1: IntegerArray(1..5);
    ActualArray_2: IntegerArray(1..3);
begin
    ActualArray_1 := (1, 2, 3, 4, 5);
    Put(Integer'Image(SumArray(ActualArray_1)));
    ActualArray_2 := (1, 2, 3);
    Put(Integer'Image(SumArray(ActualArray_2)));
end Main;
```

Ada solves the problem with array parameters in Pascal by decoupling the specific array bounds from the array type.

Let's consider how this same problem would be written in C++. We examined a number of variations of a similar function in conjunction with pointer arithmetic in module 2, but let's now consider both the function and an accompanying main function.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int sum_array(int array[],int length);

int main()
{
    int actual_1[] = {1, 2, 3, 4, 5};
    int actual_2[] = {1, 2, 3};

    cout << sum_array(actual_1, sizeof(actual_1) / sizeof(int))
        << endl;
    cout << sum_array(actual_2, sizeof(actual_2) / sizeof(int))
        << endl;
    return 0;
}

int sum_array(int array[], int length)
{
    int sum = 0;

    for (int i = 0; i < length; i++)
        sum += array[i];
    return sum;
}
```

Although it is possible to write functions that process arrays of different lengths, the approach used in C and C++ can be the source of problems, as we noted in module 2 in our discussion of the C string functions. If the wrong length information is provided to the function—the worst case being a length greater than the actual length—memory outside the array bounds will be accessed. Recall that no bounds checking is done in these languages, so in functions that modify arrays, memory outside the array can be overwritten.

In case you think this problem could be remedied by eliminating the length parameter and replacing length in the for loop with sizeof(array) / sizeof(int), you must realize that sizeof is evaluated at compile time. There is no way at run time to determine the size of an array because no size information remains at run time.

Finally, let's look at the way this same program would be written in Java.

Move your mouse over the highlighted part of the program to see the explanation of that part.

```
class ArrayParameters
{
    public static void main(String[] args)
    {
        int[] actual_1 = {1, 2, 3, 4, 5};
        int[] actual_2 = {1, 2, 3};

        System.out.println(sumArray(actual_1));
        System.out.println(sumArray(actual_2));
    }
    private static int sumArray(int[] array)
    {
        int sum = 0;

        for (int i = 0; i < array.length; i++)
            sum += array[i];
        return sum;
    }
}
```

There is one final issue regarding array parameters that requires discussion—when parameters are multidimensional arrays. For programming languages such as C and C++ that do not store the array dimensions at run time, there is some difficulty in writing functions that process arrays with dimensions of any length. Consider the following attempt at a function prototype with a two-dimensional array parameter.

```
int sum_matrix(int matrix[][],int rows, int cols);
```

The above prototype will not compile. Although the first dimension can be omitted, the second and subsequent ones cannot. The reason the number of columns cannot be omitted is because the compiler needs this information to compute the location of array elements.

The number of columns is a part of this formula, but the number of rows is not. A solution to this problem in C and C++ is for the programmer to encode this formula, which the compiler would otherwise generate, and write the function as though it were taking a one-dimensional array as a parameter, as shown in the program below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int sum_matrix(int matrix[], int rows, int cols);

int main()
{
    int actual[][3] = {{1, 2, 3}, {4, 5, 6}};

    cout <<  sum_matrix(actual[0], 2, 3) << endl;
    return 0;
}

int sum_matrix(int matrix[], int rows, int cols)
{
    int sum = 0;

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            sum += matrix[i * cols + j];
    return sum;
}
```

This problem with passing multidimensional arrays in C and C++ does not exist in Java or Ada because in both languages the sizes of the dimensions are stored at run time.

Parameter Type Checking

In module 2, we introduced the topic of parameter type checking and explained how the type of actual parameters must be checked against the type of formal parameters and how type coercion might be performed in languages that allow it when the types do not match, but are compatible.

Now, we consider languages in which such type checking is impossible in some cases and explore the consequences in those cases. Specifically, we refer to early versions of C. Independent compilation was allowed, but in the early versions, it was possible to call a function in another source file without providing a function prototype. Actually, a limited prototype that specified only the return type was required for functions that did not return an int. Without such prototypes, the compiler would be unable to generate correct object code for calls to such functions.

These prototypes were not required to specify the types of parameters, however. If the type of actual and formal parameters were the same, the program would execute properly. But if they were different, no coercion would be possible, and the resulting program would behave in an unpredictable fashion. This language design was especially troublesome during the maintenance phase of a program, in which the type of one of the function's parameters might change, and forgetting to make the appropriate change on every call produced logic errors that were difficult to find.

Because of the serious nature of the problems that can result from a lack of parameter type checking, the latest version of C and all versions of C++ require that a prototype be present for every function that is called.

Subprogram Parameters

Like recursion, in many ways, subprogram parameters properly belong to the domain of functional programming, not to imperative programming. When we discuss functional programming in module 5, we will see what an essential role subprogram parameters play.

Nonetheless, most imperative languages support this feature in some form. Pascal does. C and C++ allow function pointer parameters. Ada 83 required that generic subprograms be used for this purpose, but Ada 95 allows a more direct implementation of subprogram parameters. C# has a somewhat unique feature, called *delegates*, for this purpose. Java has no direct way to accomplish this task.

In Java, a method must be wrapped in a stateless object to achieve what amounts to passing it as a parameter. Consider the following Java example that illustrates creating a stateless object whose primary role is to pass a method as a parameter.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
class MethodParameters
{
    interface IntegerSequence
    {
        int sequence(int i);
    }

    public static void main(String[] args)
    {
        int sum = sumTerms(
            new IntegerSequence()
                {
                    public int sequence(int i)
                    {
                       return i;
                    }
                },
            5);
        System.out.println("Sum = " + sum);

    }

    private static int sumTerms(IntegerSequence f, int n)
    {
        int sum = 0;

        for (int i = 0; i < n; i++)
            sum += f.sequence(i);
        return sum;
    }
}
```

There are two important design issues that any languages that allow subprogram parameters must consider. The first is the degree to which the parameters are type checked. Without some type checking, serious errors can occur at run time. If type checking is performed, the next consideration is how closely the signatures must match.

The other design consideration is what referencing environment to give to the formal subprogram parameter. Which of the following should it be?

1. the environment of the call to the formal subprogram parameter
2. the environment of the actual subprogram parameter definition
3. the environment of the call that passed the subprogram parameter

One final consideration is whether functions can return other subprograms as their return values. Compiled languages generally do not allow this capability, although C and C++ do allow pointers to functions to be returned. Interpreted languages more often have this ability. JavaScript, for example, allows newly created subprograms to be returned. This ability is extremely important in functional programming, so we will return to it in module 5.

C. Other Issues

There are two other issues pertaining to subprograms, but not specifically parameters, that we need to consider. They are overloading and macros. We will discuss each of these below.

Overloading

We have seen some examples of overloaded subprograms earlier in this module, but now we plan to define *overloading* and explore the extent to which various languages support it.

A language allows subprogram overloading if distinguishable subprograms of the same name are permitted. What constitutes *distinguishable* differs somewhat among languages. In C++ and Java, the parameters and their types must be different. In Ada, the return type of functions can also be used to distinguish overloaded functions.

Overloading is sometimes referred to as a kind of polymorphism, but it is very different from the polymorphism associated with inheritance. The key difference is that overload resolution can be done at compile time, whereas the resolution of true polymorphism must occur at run time. We will explore this latter kind of polymorphism in module 4 in conjunction with our discussion of object-orientation.

Among languages that allow subprogram overloading, there is variation as to whether they allow operators to be defined as functions and thus allow user-defined operator overloading. Ada, C++, and C# allow such operator overloading, but Java does not.

By allowing user-defined operator overloading, it is possible to create new data types in a language that appear to be built-in data types. To illustrate this point, let's consider the definition of a new data type for complex numbers defined in C++. (Actually, the C++ standard template library contains a class very similar to this one.) First, we examine the class definition that would be placed in a file complex.h.

Move your mouse over the highlighted parts of the class definition to see the explanations of those parts.

```
class Complex
{
public:
    Complex(double real, double imaginary);
    friend istream& operator>>(istream& in, Complex& complex);
    friend ostream& operator<<(ostream& out, Complex complex);
    friend Complex operator+(Complex left, Complex right);
    friend Complex operator-(Complex left, Complex right);
    friend Complex operator*(Complex left, Complex right);
private:
    double real, imaginary;
};
```

Unlike Java, the bodies of the member bodies of a class are defined separately. Those function bodies would be in a file called complex.cpp, shown below.

```
#include <iostream>
using namespace std;

#include "complex.h"

Complex::Complex(double real, double imaginary)
{
    this->real = real;
    this->imaginary = imaginary;
}

istream& operator>>(istream& in, Complex& complex)
{
    char i;
    in >> complex.real >> complex.imaginary >> i;
    return in;
}

ostream& operator<<(ostream& out, Complex complex)
{
    if (complex.imaginary >= 0)
        out << complex.real << " + " << complex.imaginary << "
    else
        out << complex.real << " - " << -complex.imaginary << '
    return out;
}

Complex operator+(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real + right.real;
    imaginaryResult = left.imaginary + right.imaginary;
    return Complex(realResult, imaginaryResult);
}
```

```
Complex operator-(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real - right.real;
    imaginaryResult = left.imaginary - right.imaginary;
    return Complex(realResult, imaginaryResult);
}

Complex operator*(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real * right.real -
        left.imaginary * right.imaginary;
    imaginaryResult = left.real * right.imaginary +
        left.imaginary * right.real;
    return Complex(realResult, imaginaryResult);
}
```

Critics of user-defined operator overloading claim that the misuse of this feature can lead to programs that are hard to read.

Macros

Although macros are technically not subprograms, they play a role similar to that of subprograms, so it is appropriate to discuss them now. Macros are most commonly found in assembly language. But C, which was intended to be used as a language to replace assembly, and consequently C++, both allow macros. In these languages, they are translated by the preprocessor, which makes a preliminary pass over the source code. Java dispensed with the preprocessor, so it also dropped the possibility of macros.

In our earlier discussion of the call-by-name parameter-passing technique, we mentioned that the macros use a similar technique—performing a textual substitution of the formal parameters with the actual parameters. The primary

difference is that with macros, this substitution is done at compile time. Consider the following C or C++ macro:

```
#define SQUARE(x) x * x
```

Notice that, like functions, macros have parameters and a body. The body is generally much shorter than the average function body, consisting most often of a single expression rather than a sequence of statements. There are tradeoffs between the use of macros and functions. Using macros can make a program execute faster because the overhead of function calls is eliminated, but they can enlarge programs because the body of the macro replaces every macro call. It is because of this tradeoff that macros tend to be short.

Programmers who use textual substitution must be aware of several problems. For example, consider the following invocation of the previously defined macro.

```
y = SQUARE(z + 1)
```

The code that will result from the textual substitution is the following:

```
y = z + 1 * z + 1;
```

Because of precedence, the result is $2z + 1$, not the intended $(z + 1)^2$. This problem can be avoided by adding parentheses around both instances of x in the macro definition as follows:

```
#define SQUARE(x) (x) * (x)
```

Actual parameters that have side effects can present another problem. Suppose we invoke our macro as follows:

```
z = 2;
y = SQUARE(++z);
```

Because two substitutions will be performed, z will be incremented twice, not once, as was undoubtedly intended.

D. Implementing Subprograms

The complexity of implementing subprograms increases as we move from languages that do not support recursion, to ones that do, and, finally, to those that allow nested subprograms.

Languages without Recursion

When languages do not allow recursion, like early versions of Fortran, static data can be used exclusively. In such languages, it is relatively simple to implement the linkage between the calling and called subprogram. The only real complexity is considering that a subprogram may be called from possibly many different places. Therefore, at the point of the call, it is necessary to save the return address so that once that called subprogram completes, it knows where to return.

Languages with Recursion

As we discussed in module 2, languages that support recursion must allocate local data as stack-dynamic data because there can be multiple activations of such subprograms. Each time a recursive subprogram is called, another instance of its activation record must be pushed onto the compiler's run-time stack. That activation record must contain the return address, the parameters passed to the subprogram, space for local variables, and space for any temporary variables. For function calls, the returned value is also placed on the stack. It is customary to also include a pointer to the top of the activation record of the calling subprogram, called a *dynamic link*, to simplify popping the activation record instance when the subprogram completes.

Languages That Allow Nested Subprograms

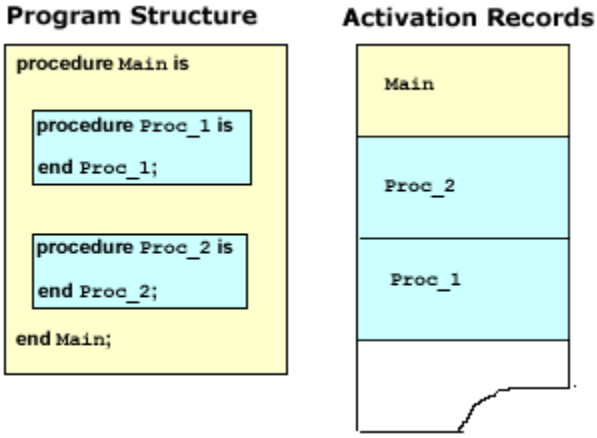
The greatest complexity in implementing the subprogram linkage is in languages that allow nested subprograms. Recall that none of the languages in the C family allow such nesting, but Pascal and Ada do. In module 2, in our discussion of stack-dynamic data, we mentioned that the compilers for such languages must determine two kinds of variable address information:

1. the relative address in the activation record
2. which activation record the variable is in

The latter value is referred to as the *static depth*. As we mentioned in module 2, this information alone is insufficient to locate the activation record at run time.

Figure 3.5 illustrates an Ada procedure containing two nested procedures, and the corresponding run-time stack of the compiler assuming that when the program runs, MAIN called PROC2 and PROC2 called PROC1.

Figure 3.5
An Ada Program with Nested Procedures
and the Compiler's Run-Time Stack



According to the scope rules of Ada, Proc_1 can access the local variables of Main but not Proc_2. Because Proc_1 is nested directly in Main, Main is considered to have a static depth of 1 away from Proc_1. The problem, however, is that the activation record for Main is two records away on the stack. Following and counting the dynamic links does not lead to the correct activation record.

There are two common solutions to this problem. The most common is to include what are called *static links* between activation records. The static link in the activation record for Proc_1 would point to the activation record for Main. In this way, following the static links and counting until the static depth was reached would lead to the correct record.

The other solution is to use what are called *displays*. A display is an array of pointers to activation records. In this example, when Proc_1 was executing, the pointer in position 0 of the display would point to the activation record for Proc_1, and the pointer in position 1 would point to the activation record for Main. So, the static depth would be used as an index into the display to find the correct activation record.

Languages that Allow Nested Blocks

All the languages in the C family allow nested blocks despite the fact that they do not allow nested subprograms. Nested blocks do not have any of the complexities of nested subprograms because blocks are not callable, and so can be encountered only when they are reached during program execution.

Module 3: Imperative Languages—Control Flow

An Expression Interpreter

The case study for this module incorporates two of the language features that we discussed—expressions and assignments. The program interprets fully parenthesized arithmetic expressions that contain either literal values or variables. The variables must then subsequently be assigned values.

The grammar for the language that this interpreter accepts is defined by the following grammar:

<program> → <exp> , <assigns> ;
<exp> → (<operand> <op> <operand>)
<operand> → <literal> | <variable> | <exp>
<assigns> → <assigns> , <assign> | <assign>
<assign> → <variable> = <literal>

The regular expressions defining the three tokens are the following:

<op> [+-*/]
<variable> [a-zA-Z][a-zA-Z0-9]*
<literal> [0-9]+

So, if you were to enter the following expression:

$(x + (y * 3))$, $x = 2$, $y = 6$;

the interpreter would respond:

Value = 20

The interpreter itself is written in C++. The complete program consists of 10 classes. We will present 7 of them. Your instructor may ask you to complete this program, perhaps enhance it, and add some error checking as one of the programming projects.

We begin with the main function and one subordinate function, which are contained in module3.cpp. The main function reads in the program, calls upon the static function parse of the SubExpression class to parse it, and builds an arithmetic expression tree. It then calls the subordinate function parseAssignments to parse the assignments and enter them into the symbol table, and then evaluates the expression and displays the result. That code is shown below:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

#include "expression.h"
#include "subexpression.h"
#include "symboltable.h"
#include "parse.h"

SymbolTable symbolTable;

void parseAssignments();

int main()
{
Expression* expression;
char paren, comma;
cout << "Enter expression: ";
cin >> paren;
expression = SubExpression::parse();
cin >> comma;
parseAssignments();
```

```
cout << "Value = " << expression->evaluate() << endl;
return 0;
}

void parseAssignments()
{
char assignop, delimiter;
string variable;
double value;
do
{
variable = parseName();
cin >> ws >> assignop >> value >> delimiter;
symbolTable.insert(variable, value);
}
while (delimiter == ',');
}
```

The arithmetic expression tree is built using an inheritance hierarchy. At the root of the hierarchy is the abstract class Expression. The class definition for Expression is contained in the file expression.h, shown below:

```
class Expression
{
public:
virtual double evaluate() = 0;
};
```

This abstract class has two subclasses. The first of these is SubExpression, which defines the node of the binary arithmetic expression tree. The class definition for SubExpression is contained in the file subexpression.h, shown below:

```
class SubExpression: public Expression
{
public:
SubExpression(Expression* left, Expression* right);
```

```
static Expression* parse();
protected:
Expression* left;
Expression* right;
};
```

As is customary in C++, the bodies of the member functions of that class are contained in the file subexpression.cpp, shown below:

```
#include <iostream>
using namespace std;

#include "expression.h"
#include "subexpression.h"
#include "operand.h"
#include "plus.h"
#include "minus.h"
#include "times.h"
#include "divide.h"

SubExpression::SubExpression(Expression* left, Expression* right)
{
this->left = left;
this->right = right;
}

Expression* SubExpression::parse()
{
Expression* left;
Expression* right;
char operation, paren;

left = Operand::parse();
cin >> operation;
right = Operand::parse();
```

```
cin >> paren;
switch (operation)
{
case '+':
return new Plus(left, right);
case '-':
return new Minus(left, right);
case '*':
return new Times(left, right);
case '/':
return new Divide(left, right);
}
return 0;
}
```

The SubExpression class has four subclasses. We show one of them—Plus. The class definition for Plus is contained in the file plus.h, shown below:

```
class Plus: public SubExpression
{
public:
Plus(Expression* left, Expression* right):
SubExpression(left, right)
{
}
double evaluate()
{
return left->evaluate() + right->evaluate();
}
};
```

Because the bodies of both member functions are inline, no corresponding .cpp file is required.

The other subclass of Expression is Operand, which defines the leaf nodes of the arithmetic expression tree. The class definition for Operand is contained in the file operand.h, shown below:

```
class Operand: public Expression
{
public:
static Expression* parse();
};
```

The body of its only member function is contained in operand.cpp, shown below:

```
#include <cctype>
#include <iostream>
#include <list>
#include <string>

using namespace std;

#include "expression.h"
#include "subexpression.h"
#include "operand.h"
#include "variable.h"
#include "literal.h"
#include "parse.h"

Expression* Operand::parse()
{
char paren;
double value;

cin >> ws;
if (isdigit(cin.peek()))
{
cin >> value;
```

```
Expression* literal = new Literal(value);
return literal;
}
if (cin.peek() == '(')
{
cin >> paren;
return SubExpression::parse();
}
else
return new Variable(parseName());
return 0;
}
```

The Operand class has two subclasses. The first is Variable, which defines leaf nodes of the tree that contain variables. The class definition for Variable is contained in the file variable.h, shown below:

```
class Variable: public Operand
{
public:
Variable(string name)
{
this->name = name;
}
double Variable::evaluate();
private:
string name;
};
```

The body of its member function evaluate is contained in variable.cpp, shown below:

```
#include <strstream>
#include <vector>
using namespace std;
```

```
#include "expression.h"
#include "operand.h"
#include "variable.h"
#include "symboltable.h"

extern SymbolTable symbolTable;

double Variable::evaluate()
{
return symbolTable.lookUp(name);
}
```

The other subclass of Operand is Literal, which defines leaf nodes of the tree that contain literal values. The class definition for Literal is contained in the file literal.h, shown below:

```
class Literal: public Operand
{
public:
Literal(int value)
{
this->value = value;
}
double evaluate()
{
return value;
}
private:
int value;
};
```

This interpreter uses a symbol table that is implemented with an unsorted list defined by the class SymbolTable. Its class definition is contained in the file symboltable.h, shown below:

```
class SymbolTable
{
public:
SymbolTable() {}
void insert(string variable, double value);
double lookUp(string variable) const;
private:
struct Symbol
{
Symbol(string variable, double value)
{
this->variable = variable;
this->value = value;
}
string variable;
double value;
};
vector <Symbol> elements;
};
```

The bodies of its member functions are in the file symboltable.cpp, shown below:

```
#include <string>
#include <vector>
using namespace std;

#include "symboltable.h"



void SymbolTable::insert(string variable, double value)
{
const Symbol& symbol = Symbol(variable, value);
elements.push_back(symbol);
}
```

```
double SymbolTable::lookUp(string variable) const
{
for (int i = 0; i < elements.size(); i++)
if (elements[i].variable == variable)
return elements[i].value;
return -1;
}
```

Finally, one utility function, parseName, is needed by this program. Its function prototype is the file parse.h, shown below:

```
string parseName();
```

Its body is in parse.cpp, shown below:

```
#include <cctype>
#include <iostream>
#include <string>
using namespace std;

#include "parse.h"

string parseName()
{
char alnum;
string name = "";

cin >> ws;
while (isalnum(cin.peek()))
{
cin >> alnum;
name += alnum;
}
return name;
}
```

Module 4: Object-Oriented and Concurrent Languages

Topics

1. Object-Based Language Features
2. Object-Oriented Language Features
3. Concurrent Language Features
4. Exceptions

I. Object-Based Language Features

The term *object-based language* is sometimes used to refer to languages that provide the syntax for some, but not all, of the features required of true object-oriented languages. Ada 83 falls into this category.

A. Abstraction, Encapsulation, and Information Hiding

As programs grew larger throughout the history of software development, various levels of abstraction developed as a way to manage ever-larger programs. Process abstraction has been incorporated in high-level languages from the very beginning. Data abstraction is a more recent development.

For each level of abstraction in any language that supports it, there is an associated syntactic mechanism of encapsulation and another kind of support for information hiding. Some texts use *encapsulatio*n and *information hiding* as synonyms. Others make what we consider an important distinction between the two, which will be our approach.

- **Encapsulation** involves grouping together a collection of more elementary components into a larger whole. The language needs some syntax to define the boundaries of this grouping.

- **Information hiding** involves restricting the visibility of data. The scope rules of the language generally provide this capability.

The subprogram is the level 1 encapsulation syntax. It supports grouping together data declarations with a collection of statements that are needed to accomplish a particular task. Level 1 encapsulation was the prime focus of structured design.

At the subprogram level, all local declarations are implicitly hidden from other subprograms. In languages that do not support nested subprograms, they are hidden from all other subprograms. When nested subprograms are permitted, the rules are more complicated, as we discussed in the previous modules.

When programming languages adopted syntax to support data abstraction, level 2 encapsulation was introduced. The syntax for level 2 encapsulation in C++, Java, and C# is the *class*. A class groups together data declarations and the methods that operate on them.

At the class level, hiding information is a bit more complicated. To limit the access to instance and class data to only the methods within the class, most languages require using the reserved word private. Such strict hiding of information is sometimes too limiting when inheritance is involved. Consequently, most languages provide some technique for allowing derived classes greater access. We will elaborate on inheritance later in this module.

Table 4.1 summarizes these two levels of abstraction and the associated language features that support them.

Table 4.1
Levels of Abstraction

Level	Abstraction	Encapsulation	Information Hiding

1	process abstraction	subprogram	local data
2	data abstraction	class	private data

Integer Set in C++

To help you better understand data abstraction and the way language features support it, we will examine an example in two languages, beginning with C++. The example we have chosen is to implement a new data type for integer sets. In module 3, we presented a similar example using complex numbers to illustrate operator overloading. Now, we concentrate on aspects related to data abstraction.

One language feature related to data abstraction that varies among languages is whether the specification is separated from the implementation. C++ and Ada make such a separation. As we saw in module 3, it is customary to place the class definition in a header file—a file whose name is the class name with the file extension .h. The bodies of the member function, by convention, are placed in a file whose name is the class name with the file extension .cpp. In all languages, separate source files serve as a kind of an encapsulation mechanism, although their use is dictated by convention, not as a part of the language definition.

Let's consider the class definition for the C++ implementation of an integer set data type whose values can be the integers from 1 to 25.

Move your mouse over the highlighted parts of the class definition to see the explanations of those parts.

```
const int SIZE = 25;

class Int25Set
{
public:
    Int25Set();
    void add(int value);
    bool in(int value) const;
    Int25Set operator&(Int25Set right)  const;
private:
    bool inRange(int position) const
    {
        return (position >= 0 && position < SIZE);
    }
    bool set[SIZE];
};
```

Next, consider the bodies of the functions, which by convention would be placed in a file called int25set.cpp.

Move your mouse over the highlighted parts of the function bodies to see the explanations of those parts.

```
#include "int25set.h"

Int25Set::Int25Set()];
{
    for (int i = 0; i < SIZE; i++)
        set[i] = false;
}

void Int25Set::add(int value)
{
    int position = value - 1;
    if (inRange(position))
        set[position] = true;
}

bool Int25Set::in(int value) const
{
    int position = value - 1;
    return inRange(position) && set[position];
}

Int25Set Int25Set::operator&(Int25Set other) const
{
    Int25Set result;

    for (int i = 0; i < SIZE; i++)
        result.set[i] = set[i] & other.set[i];
    return result;
}
```

Notice that there is really no syntactic encapsulator for the function bodies, which is why using separate files is so important.

Finally, let's examine a simple main function that will use several of the functions. The program creates two sets and then adds a 1 to the first set and a 2 to the second. It then forms their intersection, assigns it to a third set, and confirms that the third set contains neither a 1 nor a 2.

```
#include <iostream>

using namespace std;

#include "int25set.h"

int main()
{
    Int25Set set1, set2, set3;

    set1.add(1);
    set2.add(2);
    set3 = set1 & set2;
    if (!set3.in(1) && !set3.in(2))
        cout << "set3 is empty as it should be." << endl;
    return 0;
}
```

Differences in Java and C#

Although we do not intend to provide the code for this example in Java and C#, we will explain the ways in which that code would differ. The first difference is that neither Java nor C# separates the class definition from the method bodies as C++ does. In both Java and C#, the method bodies are defined within the class definition.

The next difference is with regard to operator overloading. Java does not allow it, so a named method, for example, intersection in place of operator&, would be used instead. Although C# allows operator overloading, it does not allow friend

functions, so a static method must be used instead. Using a static method would require two parameters, just like using a friend function in C++ does.

Integer Set in Ada

Let's now consider the same integer data set implemented in Ada. We begin with the specification. In Ada, specifications are put in a package specification. Unlike classes, packages may define more than a single type although, customarily, packages are used to define a single type. Including other types is a good practice only if they are tightly related to the primary type.

Move your mouse over the highlighted parts of the package specification to see the explanations of those parts.

```
package Int25_Set_Package is
    type Int25 is range 1..25;
    type Int25_Set is private;
    procedure Add (Set: in out Int25_Set; Value: in Int25);
    function In_Set(Set: in Int25_Set; Value: in Int25) return
    function "and"(Left, Right: Int25_Set) return Int25_Set;
private
    type Int25_Array is array(Int25) of Boolean;
    type Int25_Set is
        record
            Set_Array: Int25_Array := (others => false);
        end record;
end Int25_Set_Package;
```

It is customary, although not required, to separately compile package specifications and bodies, placing each in its own source file. When separate compilation is used, the package specification must be compiled before its body and any other compilation unit that uses it.

Next, we examine the package body.

Move your mouse over the highlighted parts of the package body to see the explanations of those parts.

```
package body Int25_Set_Package is
    procedure Add(Set: in out Int25_Set; Value: in Int25) is
    begin
        Set.Set_Array(Value) := true;
    end Add;
    function In_Set(Set: in Int25_Set; Value: in Int25)
        return Boolean is
    begin
        return Set.Set_Array(Value);
    end In_Set;
    function "and"(Left, Right: Int25_Set) return Int25_Set is
        Result: Int25_Set;
    begin
        Result.Set_Array := Left.Set_Array and Right.Set_Array;
        return Result;
    end "and";
end Int25_Set_Package;
```

Finally, for completeness, we include a main procedure that uses this package and the type that it defines.

```
with Text_IO; use Text_IO;
with Int25_Set_Package; use Int25_Set_Package;

procedure Int25_Main is
    Set1, Set2, Set3 : Int25_Set;
begin
    Add(Set1, 1);
    Add(Set2, 2);
    Set3 := Set1 and Set2;
    if not In_Set(Set3, 1) and not In_Set(Set3, 2) then
        Put("Set3 is empty as it should be.");
        New_Line;
    end if;
end Int25_Main;
```

B. Type Parameterization

Clearly there are several limiting factors of the integer set implementation that we just considered. It can be used only for sets of integers and moreover only for integer values between 1 and 25. Of course, it is highly desirable to write code that can be reused in a wide range of situations, enabling it to be supplied by the compiler, saving programmers from having to write essentially the same code again and again.

Ideally, we would like a set type that could hold sets of any kind of elements. If we use the representation we used in the previous examples, we are somewhat limited in how much we can generalize it. Although we could certainly allow the size to be supplied to the constructor in the C++ case and have integer sets of any range, in Ada we could use a generic package and generalize the elements to be any discrete type, which is Ada's restriction on array subscripts.

To increase the reusability of this type and illustrate the way type parameterization is implemented in different languages, let's again consider developing a set type but use a somewhat different representation—a sorted list. If we choose a sorted list,

we are still limited to the type of the components, but less so compared to the previous representation. Now, the only requirement is that the set components be an ordered type so that they can be sorted.
Because type parameterization is accomplished so differently in C++, Ada, and Java, we elect to focus on examples rather than discuss the generalities of parameterized types.

Sorted Set in C++

The syntax used in C++ to parameterize types is *templates*. Because template instantiation is done at compile time and template functions cannot be fully compiled until they are instantiated, it is customary to place both the class definition and the function bodies in the header file.

Shown below is the content of the header file for a C++ implementation of a sorted set that has a template type parameter for the component type and a template constant parameter for the size. In this implementation, we have omitted the set intersection operation to keep the size of this example from becoming too large.

Move your mouse over the highlighted parts of the template class to see the explanations of those parts.

```
template<typename Component, int SIZE>
class SortedSet
{
public:
    SortedSet()
    {
        length = 0;
    }
    void add(Component item);
    bool in(Component item) const;
private:
    Component set[SIZE];
    int length;
};

template<typename Component, int SIZE>
void SortedSet<Component, SIZE>::add(Component item)
{
    if (length < SIZE)
    {
        int i = length - 1;
        while (i >= 0 && set[i] > item)
        {
            set[i + 1] = set[i];
            i--;
        }
        set[i + 1] = item;
        length++;
    }
}

template<typename Component, int SIZE>
bool SortedSet<Component, SIZE>::in(Component item) const
```

```
{
      int low = 0, high = length - 1, middle;
      while (low <= high)
      {
         middle = (low + high) / 2;
         if (set[middle] == item)
            return true;
         else if (set[middle] > item)
            high = middle - 1;
         else
            low = middle + 1;
      }
      return false;
}
```

To illustrate instantiation of a template class, shown below is a simple main function that uses the sorted set. It adds a 1 to the set and then confirms that the set contains a 1.

Move your mouse over the highlighted part of the function to see an explanation of that part.

```
#include <iostream>

using namespace std;

#include "sortedset.h"

int main()
{
    SortedSet<int, 100> set;

    set.add(1);
    if (set.in(1))
        cout << "set1 contains 1 as expected." << endl;
    return 0;
}
```

What is a particular shortcoming in C++ is that there is no way to specify any limitations on what types can be used for the template type parameters. In the above case, if we attempt to instantiate the template class using a type that does not have a > operator, the template class will not compile.

Sorted Set in Ada

Next, let's examine the same implementation of a sorted set, this time using Ada. Type parameterization is accomplished in Ada by using generic packages. Like ordinary packages, the specification and body of generic packages are usually compiled separately. First, let's examine the specification.

Move your mouse over the highlighted parts of the package specification to see the explanations of those parts.

```
generic
    type Component is private;
    with function ">"(Left, Right: Component) return Boolean is
    Size: in Integer;
package Sorted_Set_Package is
    type Sorted_Set is private;
    procedure Add(Set: in out Sorted_Set; Item: in Component);
    function In_Set(Set: in Sorted_Set; Item: in Component)
        return Boolean;
private
    type Sorted_Array is array(1..Size) of Component;
    type Sorted_Set is
        record
            Set_Array: Sorted_Array;
            Length: Natural := 0;
        end record;
end Sorted_Set_Package;
```

Next, we examine the body of that package.

Move your mouse over the highlighted part of the package body to see the explanations of that part.

```
package body Sorted_Set_Package is
    procedure Add(Set: in out Sorted_Set; Item: in Component) i
        I: Integer;
    begin
        if Set.Length <= Size then
            I := Set.Length;
            while I >= 1 and Set.Set_Array(I) > Item loop
                Set.Set_Array(I + 1) := Set.Set_Array(I);
                I := I - 1;
            end loop;
            Set.Set_Array(I + 1) := Item;
            Set.Length := Set.Length + 1;
        end if;
    end Add;
    function In_Set(Set: in Sorted_Set; Item: in Component)
        return Boolean is
        Low: Integer := 1;
        High: Integer := Set.Length;
        Middle: Integer;
    begin
        while Low <= High loop
           Middle := (Low + High) / 2;
           if Set.Set_Array(Middle) = Item then
              return true;
           elsif Set.Set_Array(Middle) > Item then
              High := Middle - 1;
           else
              Low := Middle + 1;
           end if;
        end loop;
      return false;
```

```
    end In_Set;
end Sorted_Set_Package;
```

To conclude this Ada example, consider a simple main procedure that instantiates the generic package.

Move your mouse over the highlighted part of the procedure to see an explanation of that part.

```
with Text_Io; use Text_IO;
with Sorted_Set_Package;

procedure Sorted_Set_Main is
    package Sorted_Int_Set_Package is new
      Sorted_Set_Package(Component => Integer,  Size => 100);
    use Sorted_Int_Set_Package;
    Set: Sorted_Set;
begin
    Add(Set, 1);
    if In_Set(Set, 1) then
       Put ("Set contains 1 as expected.");
       New_Line;
    end if;
end Sorted_Set_Main;
```

The compilation order rules are stricter with generic packages. Both the specification and the body must be compiled before any compilation unit that contains an instantiation of the generic package.

Sorted Set in Java

We conclude our discussion of type parameterization by examining how the sorted set would be implemented in Java. Before version 5.0 of Java, the approach to writing type-independent container classes was to make the component be type Object when possible, or, if necessary, to use a more restrictive interface type. This

approach is possible in Java because all objects are heap-dynamic so the compiler need not concern itself with the actual size of the objects. That issue is especially important in both C++ and Ada, which support stack-dynamic data, because code with type parameterization can never be fully compiled until it is instantiated in those languages.
The one drawback to Java's initial approach was that when objects were removed from a container, to be usable they had to be downcast back to their original type. Downcasts are potentially unsafe and can be checked only at run time. The syntax for type parameterization added in version 5.0 eliminates the need to downcast and enables type checking at compile time.

Shown below is a Java version of the sorted set. Rather than including a method to search for an element in the set, we included a method to extract the element at a specific index so we could illustrate the place where downcasting is no longer needed in the client class.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
class SortedSet <Component extends Comparable>
{
    private  Component[] set;
    private int length, size;

    public SortedSet(int size)
    {
        set = (Component[]) new Comparable[size];
        this.size = size;
        length = 0;
    }
    public void add(Component item)
    {
        if (length < size)
        {
            int i = length - 1;
            while (i >= 0 && set[i].compareTo(item) > 0)
            {
                set[i + 1] = set[i];
                i--;
            }
            set[i + 1] = item;
            length++;
       }
    }
    public Component get(int index)
    {
        if (index < 0 || index >= length)
            return null;
        return set[index];
    }
}
```

To conclude this example, we supply a simple class containing a main method that instantiates a sorted set object.

Move your mouse over the highlighted parts of the class for an explanation.

```
class SortedSetMain
{
    public static void main(String[] args)
    {
        SortedSet<Integer> set = new SortedSet<Integer>(100);

        set.add(1);
        Integer first = set.get(0);
        System.out.println("The first element in the set is " +
    }
}
```

II. Object-Oriented Language Features

For a language to be considered a fully object-oriented language, in addition to providing support for encapsulation and information hiding, it must support inheritance and run-time polymorphism. It is those latter two features that we consider next.

In this section, we have elected to confine our examples to C++ only. We intend to highlight the ways in which a similar program written in Java would differ. We will also mention some differences in C#. Ada 83 does not support inheritance, but it was added in Ada 95. As is often the case when features are subsequently added to a language, the addition is less than seamless. For that reason, we plan to avoid any discussion of Ada 95 in this section.

A. Inheritance

In module 2, we discussed variant records and provided an example of an Ada variant record—a person that had two variants: a single person and a married person. The data relationships implemented with variant records are the same relationships for which inheritance is best suited—relationships characterized by the *is a* predicate. In this case, we have:

- A single person *is a* person.
- A married person *is a* person.

Let's consider the way this example would be implemented in C++ with inheritance, considering only the married variant. In the previous section, as is customary, we separated our programs into multiple files. Recall, however, that doing so is not a requirement. For the sake of brevity and to avoid all the include statements, our examples in this section will be presented as single file programs.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
#include <string>
using namespace std;

class Person
{
public:
    Person(string name, int age);
    void put(ostream& out);
private:
    string name;
    int age;
};

Person::Person(string name, int age)
{
    this->name = name;
    this->age = age;
}
void Person::put(ostream& out)
{
    out << "name = " << name << " age = " << age;
}

class Married: public Person
{
public:
    Married(string name, int age, string spouse);
    void put(ostream& out);
private:
    string spouse;
};
```

```
Married::Married(string name, int age, string spouse)
    : Person(name, age)
{
    this->spouse = spouse;
}

void Married::put(ostream& out)
{
    Person::put(out);
    out << " spouse = " << spouse;
}

int main()
{
    Married ted("Ted", 25, "Sally");
    Person* person = &ted;

    ted.put(cout);
    cout << endl;
    person->put(cout);
    cout << endl;
    return 0;
}
```

Finally let's consider the output of this program, which is shown below:

```
name = Ted age = 25 spouse = Sally
name = Ted age = 25
```

If you are most familiar with Java, this output may surprise you. When the put method is invoked on the pointer person, the put method of the Person class is called as is evident from the output. The type of the pointer is used to determine which function is called, not the type of the object. Consequently, this determination is made at compile time.

B. Polymorphism

In C++, when inheritance is used, run-time polymorphism is not the default as our previous example illustrates. To achieve run-time polymorphism, we must modify very slightly the base class definition of the previous program. We have highlighted the one change, which is the addition of the reserved word virtual to the prototype of the put function. That modified version is shown below:

```
class Person
{
public:
    Person(string name, int age);
    void virtual put(ostream& out);
private:
    string name;
    int age;
};
```

With this simple modification, the output of the program now becomes the following:

```
name = Ted age = 25 spouse = Sally
name = Ted age = 25 spouse = Sally
```

The invocation of the put function on the person pointer now exhibits run-time polymorphic behavior. The type of the object determines which put function is called, not the type of the pointer. When a virtual function is redefined in a derived class, it is called *overriding*.

In C#, the reserved word override must be used when we wish to override virtual methods. Java does not make a distinction between virtual and nonvirtual methods. In Java, all overridden methods exhibit the behavior of virtual methods. No designation using virtual is required, however.

C. Inheritance and Information Hiding

Including inheritance in a language interacts with the desire to keep the representation of a data type hidden. The member functions of derived classes must often access the data members of their base class. In our previous example, the put function in Married cannot access the data member name in the base class Person. To allow such access, declaring it in a region labeled protected would be necessary.

Java uses the same reserved word, but in Java it has a slightly different meaning. In Java, labeling data or methods as protected makes them accessible to any method in a class in the same package and any method in any derived class, regardless of what package it is in. Java has one more access mode, in addition to public, protected, and private. By omitting any of the three modifiers, we give access to any method in any class in the same package.

D. Inheriting Only the Specification

Although when inheritance was initially introduced there was disagreement on how it was best used, today most software designers agree that it should most often be used for *is a* relationships. In those cases, we wish to inherit both the specification and the representation.

There is one other time when inheritance is widely considered appropriate, which is when we wish to inherit only the specification. This kind of inheritance is so important that Java distinguishes it in two ways. The reserved word interface is used to define classes that contain only a specification, and the reserved word implements is the predicate that connects the interface with the class implementing it. In a Java interface, all the methods are public and abstract, although neither modifier is required. Any data contained in an interface must be public, static, and final.

We believe the differences between interfaces and classes are important enough that the use of these other two reserved words, interface and implements, becomes a wise language design decision. Being an older language, C++ makes no such distinction. That does not mean, however, that interfaces and interface inheritance cannot be done in C++. The following example illustrates that it can.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
template <typename Component>
class Stack
{
public:
    virtual void push(Component value) = 0;
    virtual Component pop() = 0;
};

const int SIZE = 100;

template <typename Component>
class ArrayStack : public Stack<Component>
{
public:
    ArrayStack();
    void push(Component value);
    Component pop();
private:
    Component stack[SIZE];
    int top;
};
```

An abstract class lies between an ordinary class and an interface. In Java, such classes are labeled with the reserved word abstract. These classes are permitted to have abstract methods, which then must also be labeled with the reserved word abstract. Unlike interfaces, they can have ordinary methods and variables.

C++ does not require a formal designation of classes that are abstract. The presence of a pure virtual function is all that is required to designate a class as abstract.

In both languages, every abstract method must be defined eventually along every branch of the inheritance hierarchy. Any derived class that does not define all the abstract methods it inherits is itself an abstract class. Although abstract classes can have constructors, only the constructors of the derived classes can call them. No objects of abstract classes can be instantiated.

E. Inheriting Only the Representation

One final use of inheritance is when inheriting only the representation is desired. Not everyone agrees that such uses of inheritance are appropriate. One thing is certain, however; when inheriting only the representation is desired, one should use inheritance only if a language provides the necessary support. C++ provides such support. Java does not.

To illustrate this use of inheritance in C++, we show the way a stack class could be written that is derived from the predefined class vector in the C++ standard template library. A program illustrating its use is presented below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <vector>
using namespace std;

template<typename Component>
class Stack: private vector<Component>
{
public:
    void push(Component item)
    {
        add_back(item);
    }
    Component pop()
    {
        return remove_back();
    }
};

int main()
{
    Stack<int> stack;

    stack.push(1);
    stack.add_back(2);
    return 0;
}
```

Although Java does not provide support for properly inheriting only the representation, the Java Stack class is implemented in just the way we implemented our Stack class in C++ above. As a result, one can write programs like the following one in Java.

Move your mouse over the highlighted part of the program to see the explanation of that part.

```
import java.util.*;

class JavaStackDesign
{
    public static void main(String[] args)
    {
        Stack<Integer> stack = new Stack<Integer>();

        for (int i = 0; i < 10; i++)
            stack.push(i);
        System.out.println(stack.get(4));
    }
}
```

F. Multiple Inheritance

One final design decision that any object-oriented language must make is whether to support multiple inheritance. C++ allows multiple inheritance of classes, but Java does not. Java does, however, allow multiple inheritance of interfaces.

To understand Java's design decision, it is helpful to understand some of the complexity that arises when multiple inheritance of classes is permitted. One complication is when two or more base classes have functions with identical prototypes. In that case, calls to that function become ambiguous. C++ compilers detect such ambiguity and generate compilation errors. Prepending the name with the name of the desired base class resolves the ambiguity.

Another complication occurs when two or more base classes have a common parent. The question becomes whether only one copy of the data members of the common parent should be inherited or whether multiple copies should be inherited. In C++, the presence or absence, respectively, of the reserved word virtual in the inheritance clause allows the programmer to specify this choice. We should note

that the use of virtual in this context is the reuse of a reserved word in a way that has little relationship to how it is used in the previously mentioned context—a language design decision that we have often derided.
These complications and others are eliminated when multiple inheritance is limited to interfaces because interfaces can contain only method signatures and constant data.

III. Concurrent Language Features

Adding support for concurrency in languages is a more recent trend, so older languages such as C and C++ have no such support. Even if you have studied Java, which supports concurrency, it is generally considered an advanced concept so it may be new to you. Nonetheless, it is a concept that you should not find difficult. The primary difference between a sequential program and a concurrent program is that a sequential program has a single thread of control and a concurrent program has several threads of control that run at the same time—or at least appear to.

Among the languages that have support for concurrency, Ada has the richest syntax. For that reason, we have elected to confine our discussion in this section exclusively to Ada examples. In the case study for this module, we have chosen a Java program that involves concurrent execution.

A. Underlying Implementation of Concurrency

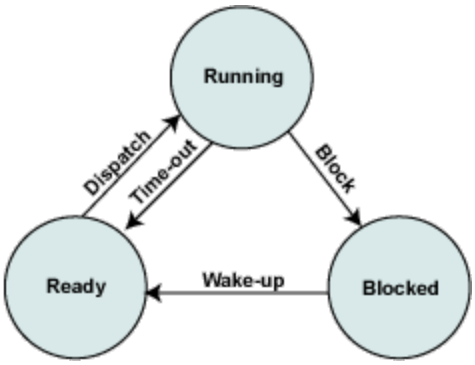
Keep in mind that programs involving concurrency can be written in languages that provide no syntactic support for concurrency by making operating-system calls to implement the required concurrent behavior. When a language does provide syntactic support for concurrency, the programs have the advantage of being operating-system independent.

Although it is not absolutely necessary to understand the underlying operating system implementation in the latter case, it is still somewhat instructive, so we begin with a brief discussion of that topic. There are two fundamental techniques for implementing concurrent execution—physical concurrency and logical concurrency.

Physical concurrency is the simpler of the two implementations. In such a system, a separate processor executes each task. Although the configuration may vary in such systems, each processor generally has some separate memory, and all processors share other memory.

With logical concurrency, a multitasking operating system appears to be simultaneously executing multiple tasks. Let's briefly discuss how a multitasking operating system accomplishes this illusion. To understand the operation of a multitasking operating system, you must understand the role of its scheduler. The scheduler decides which task should run next. To understand the way it decides what should run, you must understand the various states of a task and the events, which cause state transitions to occur. Figure 4.1 illustrates the task states and transitions.

Figure 4.1
Task State Transition Diagram



A task is in running state when it is using the processor. It is in the ready state if it is ready to use the processor but not currently using it. The third state, blocked, requires more explanation. A task is blocked when it is waiting for some event to occur. The most common event a task awaits is I/O completion. Because I/O is a slow process, a multitasking operating system will suspend a task when it makes an I/O request until the request has been completed. Doing so enables another task to use the processor during that interval. The illusion of concurrent execution is achieved by allowing another task to run while a first task is awaiting an event.

There are other events for which tasks can wait. Among them are messages from other tasks, expiration of a timer, or an interrupt from an external device. When a task reaches a point where it must await some event, the task relinquishes the

processor, leaves the ready state, and enters the blocked state.

B. Reasons for Synchronization

The need for synchronization is one of the key differences between programs that involve concurrency and those that do not. Concurrent programs need synchronization for the following two fundamental reasons:

1. to provide the cooperation between tasks that is sometimes needed
2. to avoid an undesirable kind of competition between tasks

Cooperation Synchronization

We begin by discussing cooperation synchronization. Whenever one task must wait until another task has performed some activity before it can proceed, cooperation synchronization is needed. The example we have chosen to illustrate this kind of synchronization will also serve as a means to compare concurrent and sequential programs and to introduce the syntax for Ada tasks.

The example involves the activities one must perform upon arrival at an airport. On arrival at the airport, one must perform two activities. We are intentionally using *activity* in place of *task* to avoid confusion with Ada tasks. The two activities are:

1. pick up luggage
2. obtain a rental car

If we are traveling alone, we must perform these activities in sequence. The order of these activities is unimportant, so we may choose any sequence. First, we implement this problem with a sequential program. It is convenient to assume the existence of a package that implements the details. For obvious reasons, we choose to omit the details of that package. Shown below is the code for performing the two activities.

```
with Airport_Package; use Airport_Package;
procedure Airport_Arrival_1 is
begin
    Pickup_Luggage;
    Get_Rental_Car;
end Airport_Arrival_1;
```

If we are traveling with a friend, the activities can be performed in parallel. One person can perform one of the activities while at the same time the other person performs the other activity. Each task executes a separate subprogram. The fact that the order of these activities is irrelevant is critical to their being able to be done in parallel. In this example, we have a main procedure that contains two subordinate tasks as shown below.

```
with Airport_Package; use Airport_Package;
procedure Airport_Arrival_2 is
     task Luggage_Task;
     task Rental_Car_Task;
     task body Luggage_Task is
     begin
         Pickup_Luggage;
     end Luggage_Task;
      task body Rental_Car_Task is
     begin
         Get_Rental_Car;
     end Rental_Car_Task;
begin
    null;
end Airport_Arrival_2;
```

One important characteristic of Ada tasks that is important to recognize is that they do not need to be explicitly started. In the above program, merely declaring them inside a procedure causes them to begin executing simultaneously when the

composite procedure begins execution. Furthermore, the order in which they are declared is not significant.

Finally, this example illustrates the most rudimentary kind of cooperative synchronization. It guarantees that the Airport_Arrival_2 procedure will not complete until both of its component tasks have completed. Ada has a rich syntax for more sophisticated cooperative synchronization. We will explore some of that syntax shortly.

Competition Synchronization

Competition synchronization is needed when two or more tasks must access common data. Failure to perform proper synchronization can lead to programs that may execute differently depending on the timing of events. Such programs are said to contain a *race condition*.

As an example of a race condition, let's consider a simple two-task program in which both tasks access a shared variable. The specific example that we choose to illustrate this problem is a pair of automated teller machines. The customer's checking account will be implemented as a package with two procedures: Deposit and Withdraw. Both subprograms access the checking-account balance, which is the shared variable. Finally, there are two identical tasks that implement the two ATM machines. Both of these tasks can access the checking-account package. Let us consider the main procedure that contains these tasks.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure Race_Condition is
    package Account is
        procedure Deposit(Amount: in Integer);
        procedure Withdraw(Amount: in Integer; Sufficient_Funds
            Boolean);
    private
        Balance: Integer := 0;
    end Account;
    package body Account is separate;
    task type ATM;
    task body ATM is separate;
    ATM_1, ATM_2: ATM;
begin
    null;
end Race_Condition;
```

Shown below is the body of the Account package, which should require no explanation.

```
separate(Race_Condition)
package body Account is
    procedure Deposit(Amount: in Integer) is
    begin
        Balance := Balance + Amount;
    end Deposit;
    procedure Withdraw(Amount: in Integer; Sufficient_Funds: ou
            Boolean) is
    begin
        if Amount <= Balance then
            Balance := Balance - Amount;
            Sufficient_Funds := True;
        end if;
        Sufficient_Funds := False;
    end Withdraw;
end Account;
```

To complete the programs, consider the body of the ATM task type:

```
with Text_IO; use Text_IO;
separate(Race_Condition)
task body ATM is
    type Transactions is (Deposit_Transaction, Withdraw_Transac
    package Transactions_IO is new Enumeration_IO(Transactions)
    use Transactions_IO;
    package Int_IO is new Integer_IO(Integer); use Int_IO;
    Sufficient_Funds: Boolean;
    Transaction: Transactions;
    Amount: Integer;
begin
    loop
        Put("Enter Transaction Type and Amount: ");
        Get(Transaction);
        Get(Amount);
        case Transaction is
            when Deposit_Transaction =>
                Account.Deposit(Amount);
            when Withdraw_Transaction =>
                Account.Withdraw(Amount, Sufficient_Funds);
                if not Sufficient_Funds then
                    Put("Insufficient Funds"); New_Line;
                end if;
        end case;
    end loop;
end ATM;
```

Remember that there are two instances of this task. It is best to envision each one using a separate console for its input/output.

Next, we outline the various steps leading to a race condition. To fully understand this sequence of events, we must realize that one line of Ada code is actually translated into many lines of machine code. Let us consider the assignment

statement, which is the body of the procedure Withdraw in the package Account. On most machines, this single line would translate into several lines of machine code. Figure 4.2 describes the steps that lead to a race condition.

Figure 4.2

A Scenario That Causes a Race Condition

0:00 / 1:11

The result of the sequence of events in the above figure is that $25 has been withdrawn from two ATM machines, and the balance has changed from $100 to $75 instead of the correct sum, $50.

No bank would be in business long if it allowed such race conditions to occur. Race conditions are a particularly insidious problem for concurrent programs because incorrect results are not likely to occur frequently in programs that contain race conditions. For that reason, testing is not a reliable method for discovering such problems. It is important, then, to be certain that such problems are avoided at the design stage. Although we have described a race condition in the context of a single processor system, the same problem can occur with a multiprocessor system, which contains shared memory.

The possibility of a race condition arises when a task is interrupted during a critical section. In our ATM example, the assignment statement that updates the balance is the critical section. An approach that would prohibit more than one task from having access to a given variable at the same time is an implementation of mutual exclusion. In our ATM example, if the withdraw and deposit operations have mutually exclusive access to the account balance, a race condition will be prevented.

C. Synchronization Techniques

Languages can provide several techniques for synchronization, including the use of monitors, message passing, and semaphores. Although any one of them is sufficient, languages often provide more than one. We will discuss these techniques in the following sections.

Monitors

Monitors provide synchronization that aims primarily to solve competition synchronization problems. Hoare and Brinch Hansen developed them in the mid-1970s. A monitor is a language construct that encapsulates a collection of procedures and variables. The key feature of a monitor is that the execution of the procedures inside the monitor is mutually exclusive. No procedure inside a monitor can begin execution while another procedure is executing.

Although they were not part of Ada 83, Ada 95 added the syntax of protected objects that provide monitor-style synchronization. Now we are ready to return to the ATM example that we introduced earlier. By replacing the package that defined a singleton object used previously with a protected object, we ensure that the procedures Deposit and Withdraw execute in a mutually exclusive way, and no race condition can occur. Consider this modified code. Only the necessary changes are highlighted.

```
procedure No_Race_Condition is
    protected Account is
        procedure Deposit(Amount: in Integer);
        procedure Withdraw(Amount: in Integer; Sufficient_Funds
            Boolean);
    private
        Balance: Integer := 0;
    end Account;
    protected body Account is separate;
    task type ATM;
    task body ATM is separate;
    -- Task body is the same as before
    ATM_1, ATM_2: ATM;
begin
    null;
end No_Race_Condition;

separate (No_Race_Condition)
protected body Account is
    procedure Deposit(Amount: in Integer) is
    begin
        Balance := Balance + Amount;
    end Deposit;
    procedure Withdraw(Amount: in Integer; Sufficient_Funds: ou
        Boolean) is
    begin
        if Amount <= Balance then
            Balance := Balance - Amount;
            Sufficient_Funds := True;
        end if;
        Sufficient_Funds := False;
```

```
    end Withdraw;
end Account;
```

One noteworthy comment is that the use of the reserved word protected in this context is completely unrelated to the way it is used in C++, Java, or C#.

Ada protected objects provide a second level of synchronization that goes beyond what is required of a monitor. This second level accommodates cooperation synchronization and requires the use of a syntax that uses entries rather than procedures and entry barriers to control the synchronization.

To illustrate these two levels of synchronization, we have chosen the classical synchronization problem—a bounded buffer. The bounded buffer is a first-in first-out queue implemented with a fixed-size array. The operations are enqueue and dequeue. There are two tasks that access this buffer—a producer task that reads in characters and enqueues them and a consumer task that dequeues characters and outputs them.

Using a protected object to encapsulate the bounded buffer ensures that the execution of enqueue and dequeue are mutually exclusive. For this problem, however, a second level of synchronization is needed. We must ensure that the producer becomes blocked whenever it attempts to enqueue a character and the queue is full. We must also ensure that the consumer task becomes blocked whenever it attempts to dequeue a character and the buffer is empty. Let's examine the code that solves this problem.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
with Text_IO;
procedure Producer_Consumer_1 is
    Size: constant Integer := 100;
    type Indexes is mod Size;
    type Buffers is array(Indexes) of Character;
    protected Bounded_Buffer is
        entry Enqueue (Char: in Character);
        entry Dequeue (Char: out Character);
    private
        Front, Back: Indexes := 0;
        Filled: Integer range 0..Size := 0;
        Buffer: Buffers;
    end Bounded_Buffer;
    protected body Bounded_Buffer is
        entry Enqueue (Char: in Character) when Filled < Size i
        begin
            Buffer(Back) := Char;
            Back := Back + 1;
            Filled := Filled + 1;
        end Enqueue;
        entry Dequeue (Char: out Character) when Filled > 0 is
        begin
            Char := Buffer(Front);
            Front := Front + 1;
            Filled := Filled - 1;
        end Dequeue;
    end Bounded_Buffer;
    task Producer;
    task body Producer is
        Input_Char: Character;
    begin
        loop
            Text_IO.Get(Input_Char);
```

```
            Bounded_Buffer.Enqueue(Input_Char);
        end loop;
    end Producer;
    task Consumer;
    task body Consumer is
        Output_Char: Character;
    begin
        loop
            Bounded_Buffer.Dequeue(Output_Char);
            Text_IO.Put(Output_Char);
        end loop;
    end Consumer;
begin
    null;
end Producer_Consumer_1;
```

Message-Passing

The second synchronization technique we explore is message passing. This technique can be implemented in various ways. The first variation is whether the communication is synchronous or asynchronous. When we call a friend on the telephone, we are engaging in synchronous communication. We must wait for our friend to answer, before we can have a conversation. When we send a letter using the postal service, we are engaging in asynchronous communication. In addition, electronic mail and leaving messages on telephone-answering machines constitute asynchronous communication.

The second major variation is whether the communication uses symmetric protocol or asymmetric protocol. These protocols are also a part of our everyday experience. Most telephone communication has an asymmetric protocol. The caller selects whom he calls, but the receiver takes calls from anyone. If, however, we have a dedicated line between two parties, when the phone rings, only one person can be calling. This is a symmetric communication protocol.

Now that we have explored the major variations within the message-passing model, we are prepared to discuss the techniques that Ada uses. Ada 83 introduced the first message-passing technique, called *rendezvous*, which is synchronous

communication with an asymmetric protocol. In Ada 95, an asynchronous form was added. We plan to focus on the former technique.

To illustrate rendezvous message-passing, we return to the producer-consumer problem that requires two levels of synchronization and implement it using a task in place of the protected object that we used previously with rendezvous for synchronization.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
with Text_IO;
procedure Producer_Consumer_2 is
    Size: constant Integer := 100;
    type Indexes is mod Size;
    type Buffers is array(Indexes) of Character;
    task Bounded_Buffer is
        entry Enqueue (Char: in Character);
        entry Dequeue (Char: out Character);
    end Buffer;
    task body Bounded_Buffer is
        Front, Back: Indexes := 0;
        Filled: Integer range 0..Size := 0;
        Buffer: Buffers;
    begin
        loop
            select
                when Filled < Size =>
                accept Enqueue (Char: in Character) do
                    Buffer(Back) := Char;
                    Back := Back + 1;
                    Filled := Filled + 1;
                end Enqueue;
            or
                when Filled > 0 =>
                accept Dequeue (Char: out Character) do
                    Char := Buffer(Front);
                    Front := Front + 1;
                    Filled := Filled - 1;
                end Dequeue;
            end select;
        end loop;
    end Bounded_Buffer;
    task Producer;
```

```
    task body Producer is
        Input_Char: Character;
    begin
        loop
            Text_IO.Get(Input_Char);
            Bounded_Buffer.Enqueue(Input_Char);
        end loop;
    end Producer;
    task Consumer;
    task Consumer is
        Output_Char;
    begin
        loop
            Bounded_Buffer.Dequeue(Output_Char);
            Text_IO.Put(Output_Char);
        end loop;
    end Consumer;
begin
    null;
end Producer_Consumer_2;
```

Using rendezvous synchronization, the guards on the accept statements control the cooperative synchronization. The mutual exclusion is achieved by the fact that a task has its own thread of control and can be doing only one thing at a time, so when it is accepting the request of one task, it will not be ready to accept any other requests.

Semaphores

We have saved a discussion of semaphores for last because they are the most primitive synchronization technique and are often characterized as the *goto* form of synchronization control. We highlight the fact that semaphores are a very low-level technique because the potential for misuse is great, and it is easy to create problems such as deadlock. It is for that reason the Ada designers chose not to

implement such a synchronization technique. We will demonstrate that although semaphores are not built into Ada, they can be implemented using both of Ada's synchronization techniques.
There are two types of semaphores—binary semaphores and counting semaphores. We confine our discussion to the former. A binary semaphore can be represented as a Boolean value. Do not be misled, however, into thinking that a binary semaphore is only a Boolean variable.

Its state can be changed only by performing one of two operations—secure or release. The secure operation checks the Boolean value. If it is false, it sets it to true, and the secure operation is complete. If it is already true, however, the task initiating the call is blocked, and the task is queued to await the release. A call to release causes the value to be set to true and causes one of the tasks queued, awaiting release, to be returned to the ready state in order to complete its secure operation. It should be clear that a semaphore is a great deal more than just a Boolean value. Its real essence is defined by its two operations.

First, let's consider how a binary semaphore can be implemented using a protected object.

Move your mouse over the highlighted parts of the protected object to see the explanations of those parts.

```
protected Binary_Semaphore is
    entry Secure;
    procedure Release;
private
    Available: Boolean := True;
end Binary_Semaphore;
protected body Binary_Semaphore is
    entry Secure when Available is
    begin
        Available := False;
    end Secure;
    procedure Release is
    begin
        Available := True;
    end Release;
end Binary_Semaphore;
```

Next, let's examine how a binary semaphore can be constructed using a task and message-passing synchronization.

Move your mouse over the highlighted part of the task to see the explanation of that part.

```
task Binary_Semaphore is
    entry Secure;
    entry Release;
end Binary_Semaphore;

task body Binary_Semaphore is
begin
    loop
        accept Secure;
        accept Release;
    end loop;
end Binary_Semaphore;
```

Figure 4.3 is a finite-state diagram that illustrates the behavior of this task. Notice that no flag is needed because the state of the object captures when the semaphore is or is not available.

Figure 4.3
Finite-State Diagram for a Binary Semaphore Task

IV. Exceptions

Exceptions are erroneous conditions that are unexpected during the normal execution of a program and that require special handling when they do occur. It is particularly important that languages that are designed to handle critical applications provide this capability. Ada, which was developed with embedded system development in mind, was one of the first languages to include it.

In our discussion of exceptions, we will consider the way exceptions are declared, the way they are thrown, the way they are caught, and whether the language provides syntax for subprograms to designate exceptions that they might throw. In

our discussion, we plan to compare three languages, Ada, C++, and Java.

A. Declaring Exceptions

Although only Ada and Java have some predefined exceptions, all three languages allow the user to define additional ones. What we consider first is the way these user-defined exceptions are declared.

In Ada, exception is a reserved word, and we declare exceptions by including this reserved word, as illustrated below:

```
MyException: exception;
```

In C++, there is no distinction between exceptions and any other kind of object, so in C++, any object can be treated as an exception object.

In Java, Exception is a predefined class, and all user-defined exception classes must be defined as classes derived from this class. The parent class of Exception is Throwable, which defines the broadest class whose objects can be thrown. In Java, an exception class declaration might look as follows:

```
class MyException extends Exception
{
}
```

Although exception classes can have instance methods and methods like any other class, they often have neither.

B. Throwing Exceptions

When an exceptional condition is detected, an exception is thrown. In Ada, a raise statement is used to accomplish this objective. Such a statement might look as follows:

```
raise MyException;
```

In both C++ and Java, a throw statement is used. In both languages, exception names are classes, so an instance of the class must be thrown. These instances are typically anonymous. In Java, it might look as follows:

```
throw new MyException();
```

Because C++ permits stack-dynamic objects, the exception instance is often created with a call to the constructor as follows:

```
throw MyException();
```

C. Catching Exceptions

When an exception has been thrown, ideally it will be caught within the program. In Ada, exception handlers can be associated with any begin-end pair. Such pairs might delimit blocks, subprograms, or packages, among others. A block that has such a handler might look as follows:

```
begin
    -- code that might raise an exception
when exception_name =>
    --code to handle the exception
end;
```

Any number of when clauses may be provided to catch various exceptions.

Both C++ and Java require that a try statement be used to catch exceptions. Its code looks as follows:

```
try
{
    // code that might throw an exception
}
catch (ExceptionClass exceptionObject)
{
    // code to catch exception
}
```

As with Ada, any number of catch blocks may be provided. One difference, however, results from the fact that in C++ and Java, exceptions are objects. A catch block will catch any exception of the specified exception class or any class derived from it.

In each of these languages, exceptions are propagated if they are not caught. When nested blocks or nested try statements are used, the exceptions propagate progressively to the next outermost block or statement. Once they reach the subprogram level they are propagated according to the dynamic chain of subprogram calls. If an exception is never caught, the program terminates.

One final difference among languages is that the Java try statement allows an optional finally block. When present, the code inside this block is executed last and is executed regardless of whether or not an exception is thrown.

D. Designating Thrown Exceptions

One final issue is whether the language allows the exceptions that might be thrown in a method to be included after its signature. Ada does not allow such a designation, but both C++ and Java do. There are two differences, however. First, in C++, the reserved word throw is reused for this purpose, whereas Java included a new reserved word throws. In Java, such a designation might look like the following example in which we highlighted the throws clause:

```
void push(Object item) throws StackFull
```

The second difference is that Java distinguishes between unchecked exceptions and checked exceptions. If a checked exception might be thrown in a method and that method does not catch it, it must be designated in the throws clause of that method.

Return to top of page

Module 4: Object-Oriented and Concurrent Languages

A Java Program Involving Concurrency

In our discussion of concurrency in the commentary, we focused exclusively on how it is implemented in Ada. So that you have an opportunity to see the implementation in one other language, for our case study, we have chosen to present a Java example that uses concurrency.

Our example is a program that merges two files of integers. Three tasks, or *threads*, as they are called in Java, are used. There are two input threads: one for reading from each input file, and an output thread that writes to the output file. In addition, there is one additional object that is akin to an Ada protected object that performs the synchronization and ensures that the output thread always receives the next smallest integer.

We begin with the code for the synchronizing object.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
class Protector
{
private int[] values = new int[2];
private boolean[] present = new boolean[2];
public Protector()
{
present[0] = present[1] = false;
}
synchronized public void submit(int i,
int value)
{
```

```
while (present[i])
try
{
    wait();
}
catch (InterruptedException exception)
{
}
values[i] = value;
present[i] = true;
notifyAll();
}
synchronized public int extractSmaller()
{
int smaller;
while (!present[0] || !present[1])
try
{
wait();
}
catch (InterruptedException exception)
{
}
if (values[0] < values[1])
smaller=0;
else
smaller=1;
present[smaller]=false;
notifyAll();
return values[smaller];
}
```

The submit method is called by the input threads to provide a value, and the extractSmaller method is called by the output thread to obtain one.

Let's consider next the class that defines the two input threads.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
import java.io.*;

class Input extends Thread
{
private int i;
private Protector protector;
private BufferedReader inFile;
public Input(int i, String fileName, Protector
protector) throws FileNotFoundException
  {
this.i = i;
this.protector = protector;
inFile = new BufferedReader(new FileReader(fileName));
}
public void run()
{
String line;
int value;
try
{
   line = inFile.readLine();
while (line != null)
{
value = Integer.parseInt(line);
protector.submit(i, value);
line = inFile.readLine();
}
protector.submit(i, Integer.MAX_VALUE);
inFile.close();
}
```

```
catch (IOException exception)
{
}
}
}
```

Next, let's examine the Output class shown below, which should require no additional explanation because it uses the same kind of language constructs as Input.

```
import java.io.*;

class Output extends Thread
{
private PrintWriter outFile;
private Protector protector;
public Output(String fileName, Protector protector)
throws IOException
{
this.protector = protector;
outFile = new PrintWriter(new BufferedWriter(
new FileWriter(fileName)));
}
public void run()
{
int value;
while (true)
{
value = protector.extractSmaller();
if (value == Integer.MAX_VALUE)
break;
outFile.println(value);
}
```

```
    outFile.close();
}
}
```

Finally, we examine the class that contains the main method.

Move your mouse over the highlighted part of the class to see the explanation of that part.

```
import java.io.*;

class Merge
{
public static void main(String[] args) throws
FileNotFoundException, IOException
{
Protector protector = new Protector();
Input input1 = new Input(0, "input1.txt", protector);
Input input2 = new Input(1, "input2.txt", protector);
Output output = new Output("output.txt", protector);

output.start();
input1.start();
input2.start();
}
}
```

Return to top of page (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc330/2225/modules/m4-module-4/s4-case-study.html?ou=693193#pagetop)